

Take these ingredients and apply them to

Learning for vision

Big questions:

1. How do you represent the input and output?
2. What is the objective?
3. What is the hypothesis space? (e.g., linear, polynomial, neural net?)
4. How do you optimize? (e.g., gradient descent, Newton's method?)
5. What data do you train on?

Case study #2: Image classification

- 1. How do you represent the input and output?**
- 2. What is the objective?**
3. Assume hypothesis space is sufficiently expressive
4. Assume we optimize perfectly
5. Assume we train on exactly the data we care about

Image classification

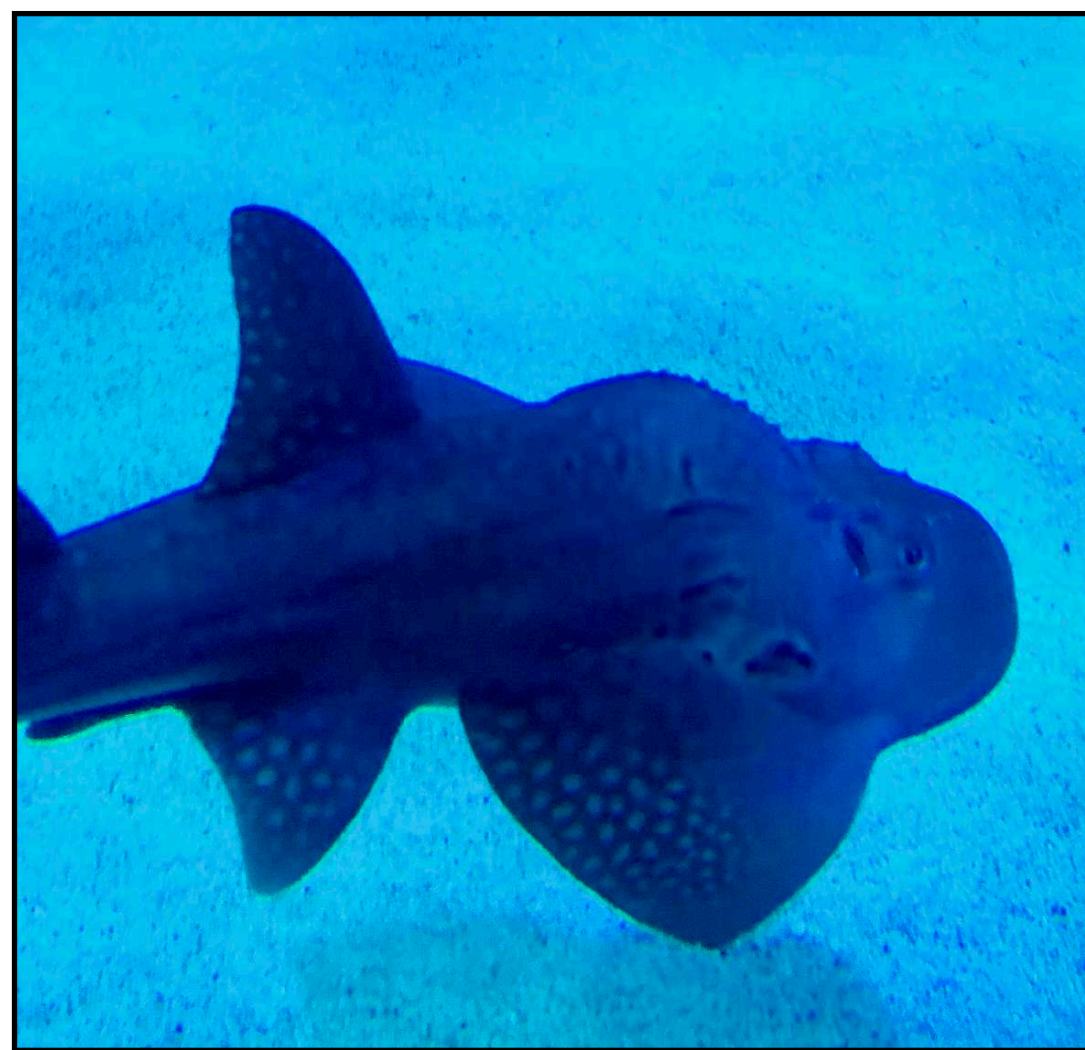
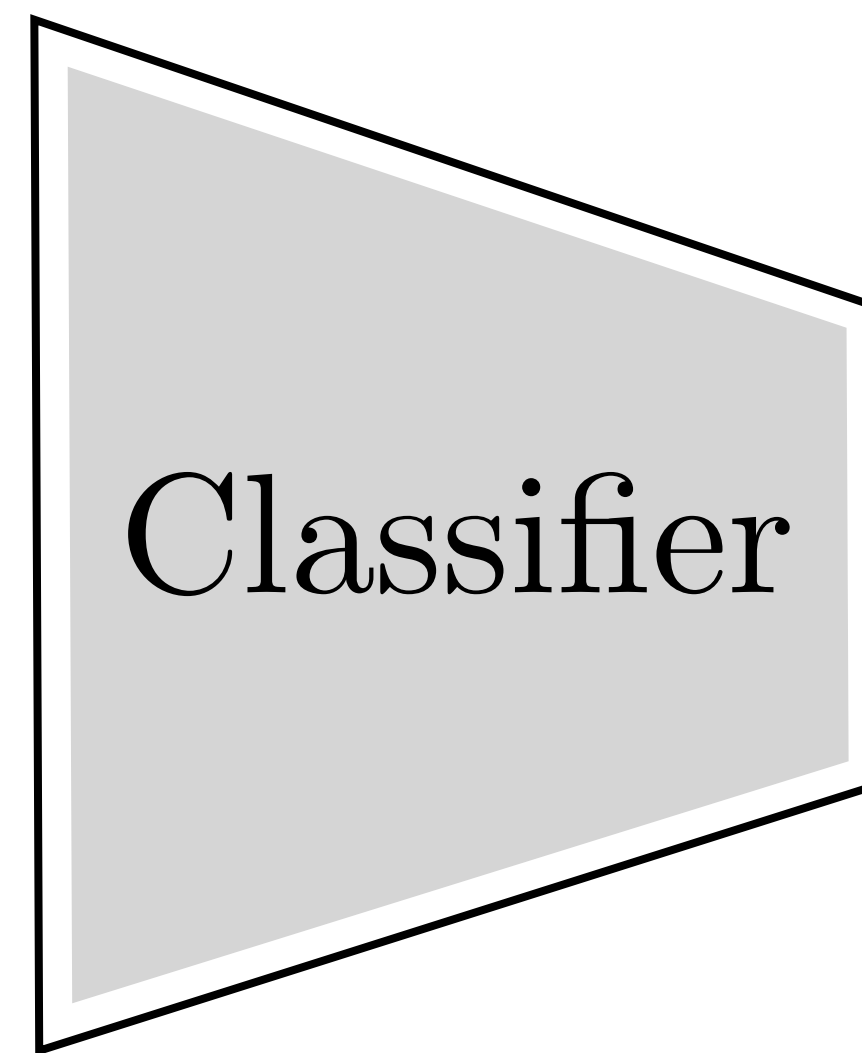


image x



Classifier



Guitarfish

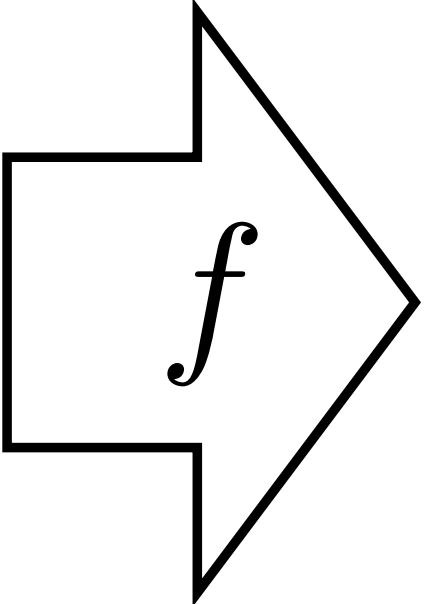
label y

$\mathbf{x}$



$\mathbf{y}$

Guitarfish



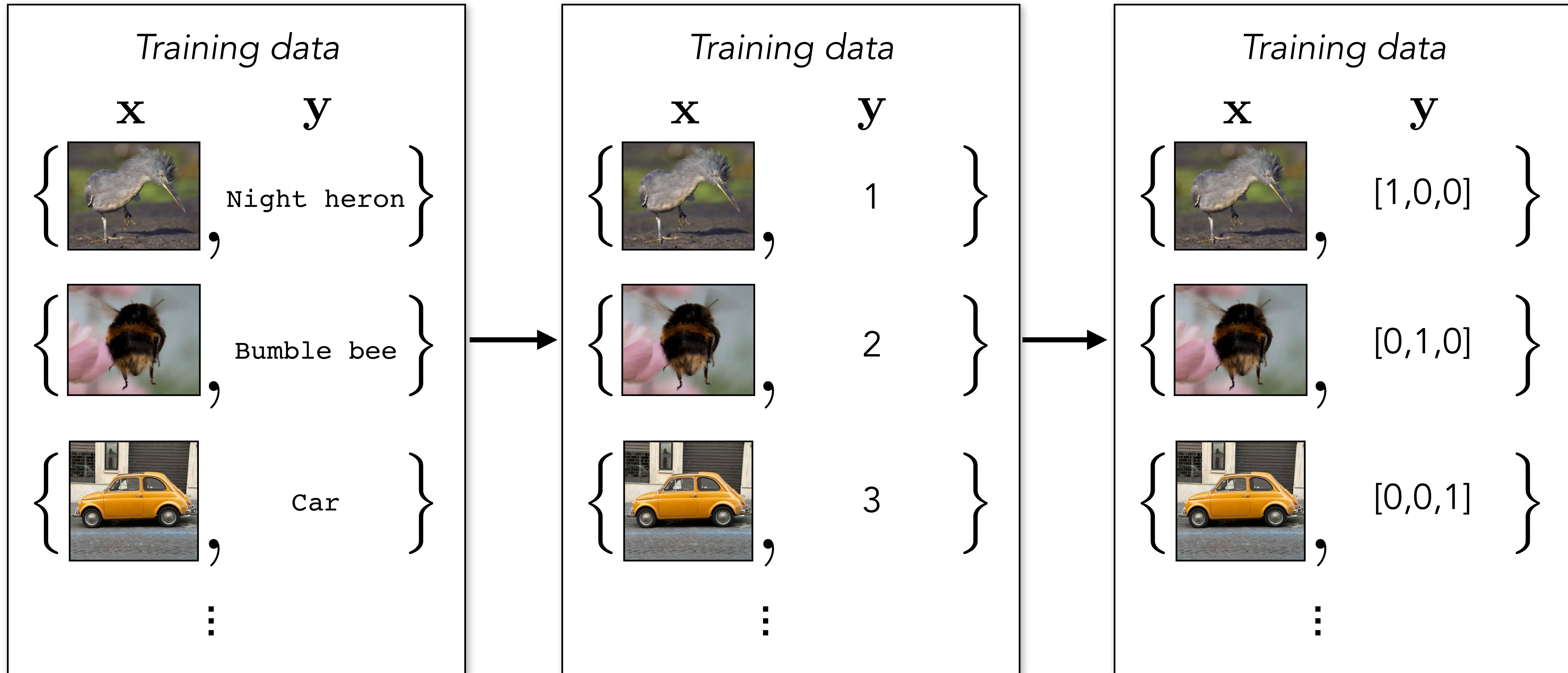
$$\arg \min_{f \in \mathcal{F}} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}), \mathbf{y}^{(i)})$$

Training data

$\mathbf{x}$	$\mathbf{y}$
	Night heron
	Bumble bee
	Car
$\vdots$	

How to represent class labels?

One-hot vector



What should the loss be?

0-1 loss (number of misclassifications)

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \mathbb{1}(\hat{\mathbf{y}} \neq \mathbf{y}) \quad \leftarrow \text{discrete, NP-hard to optimize!}$$

Cross entropy

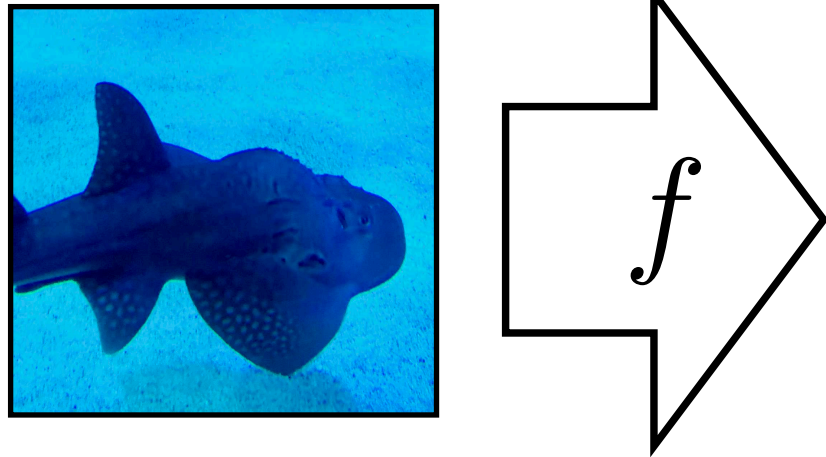
$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad \leftarrow \text{continuous, differentiable}$$

One-hot labeling of the kth class -log estimated probability of the kth class

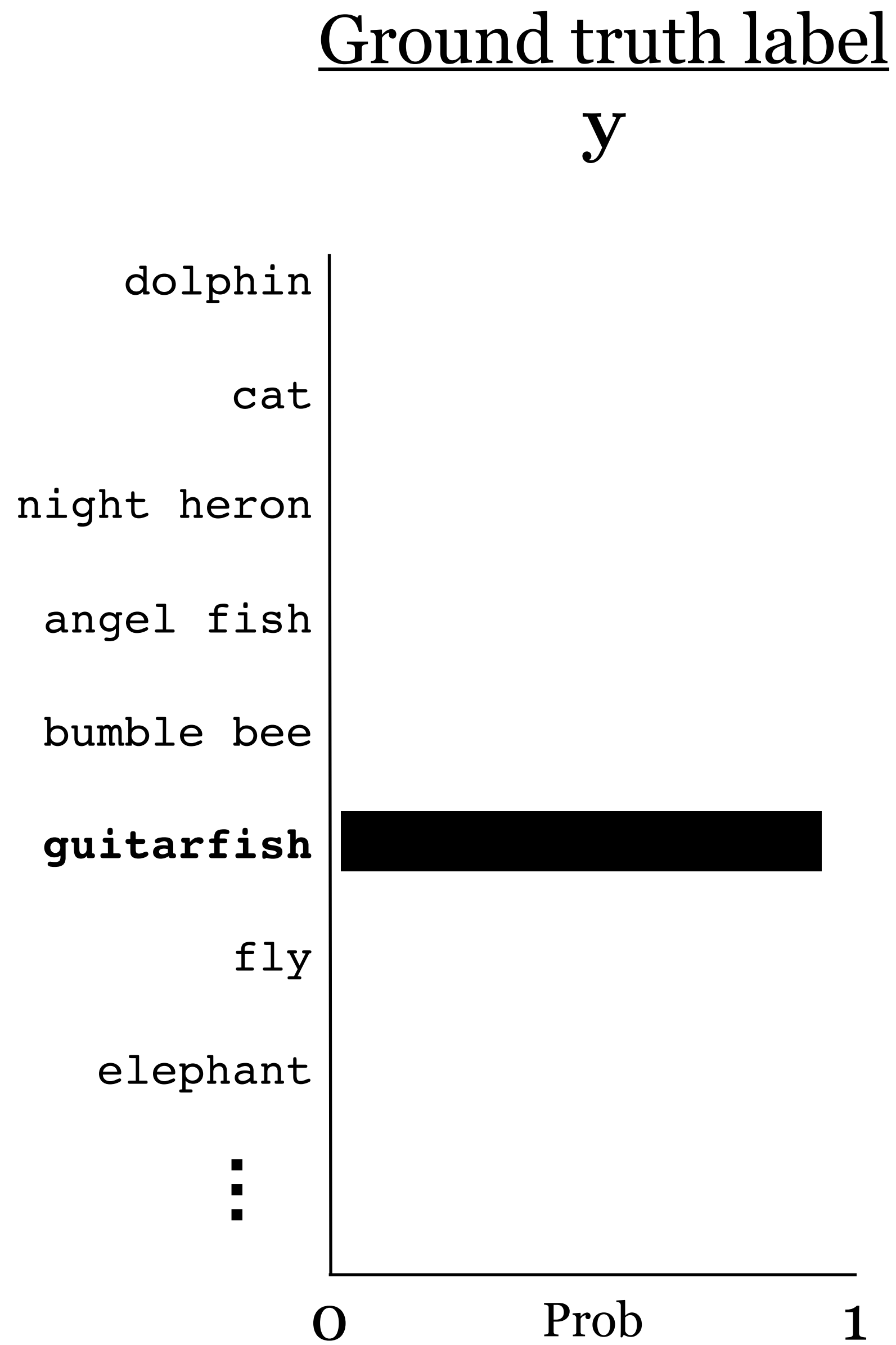
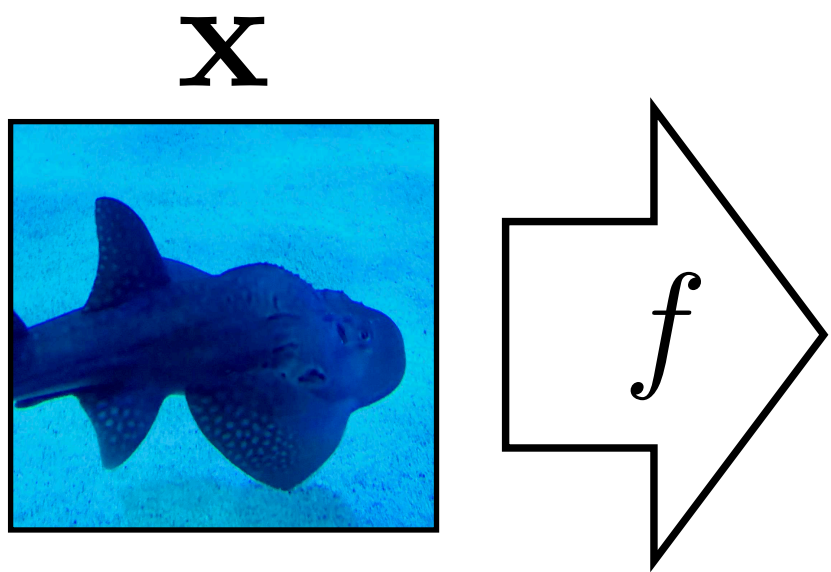
Ground truth label

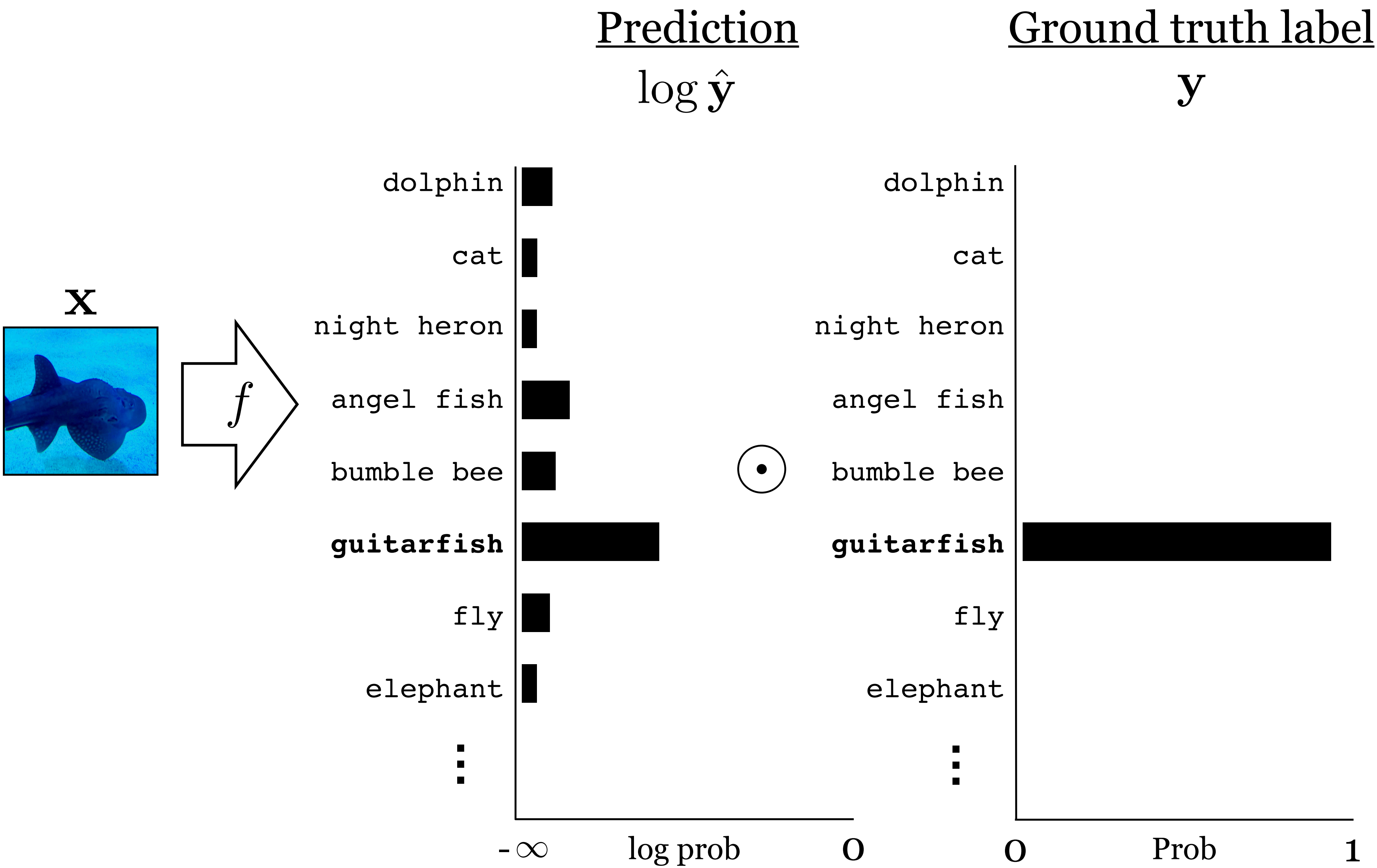
y

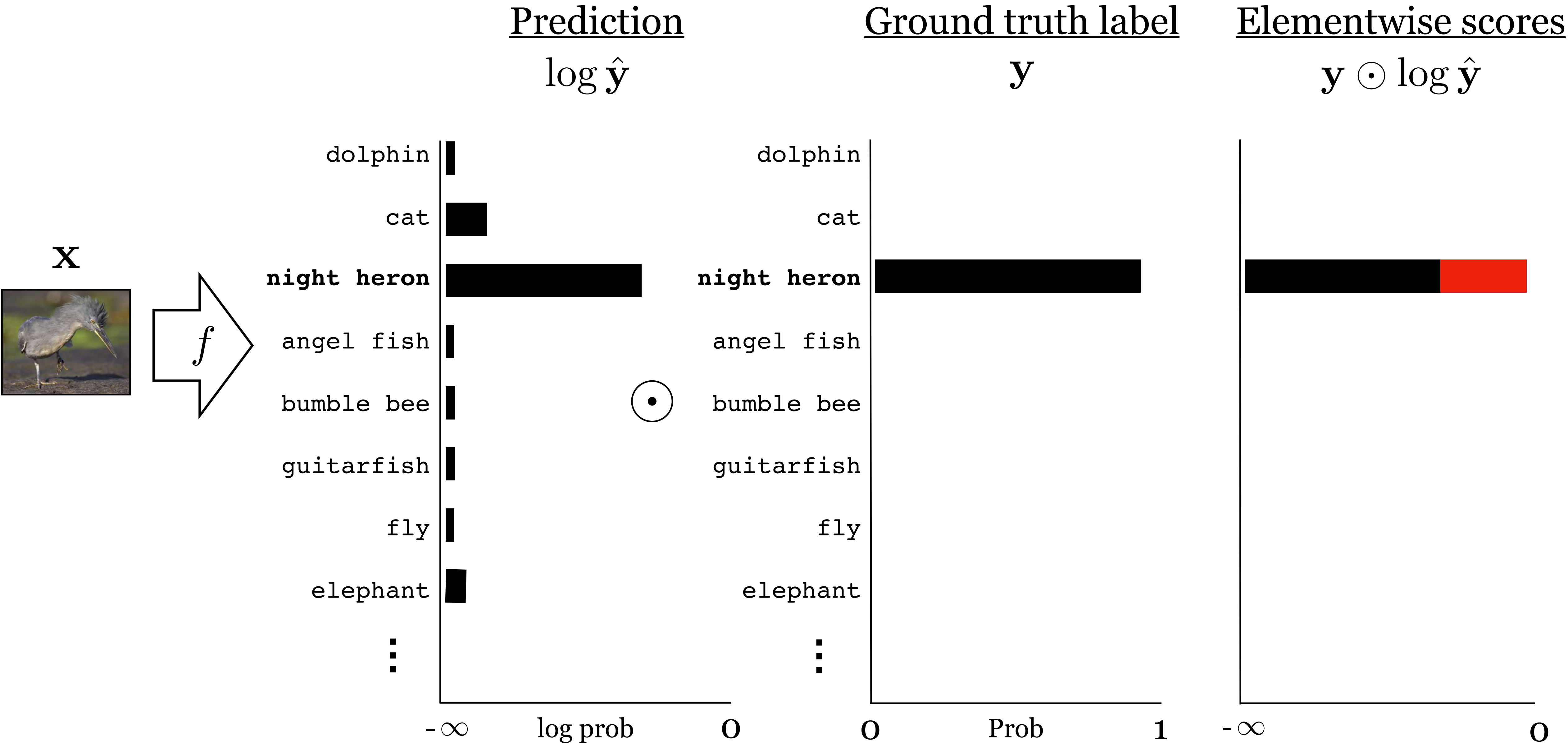
x

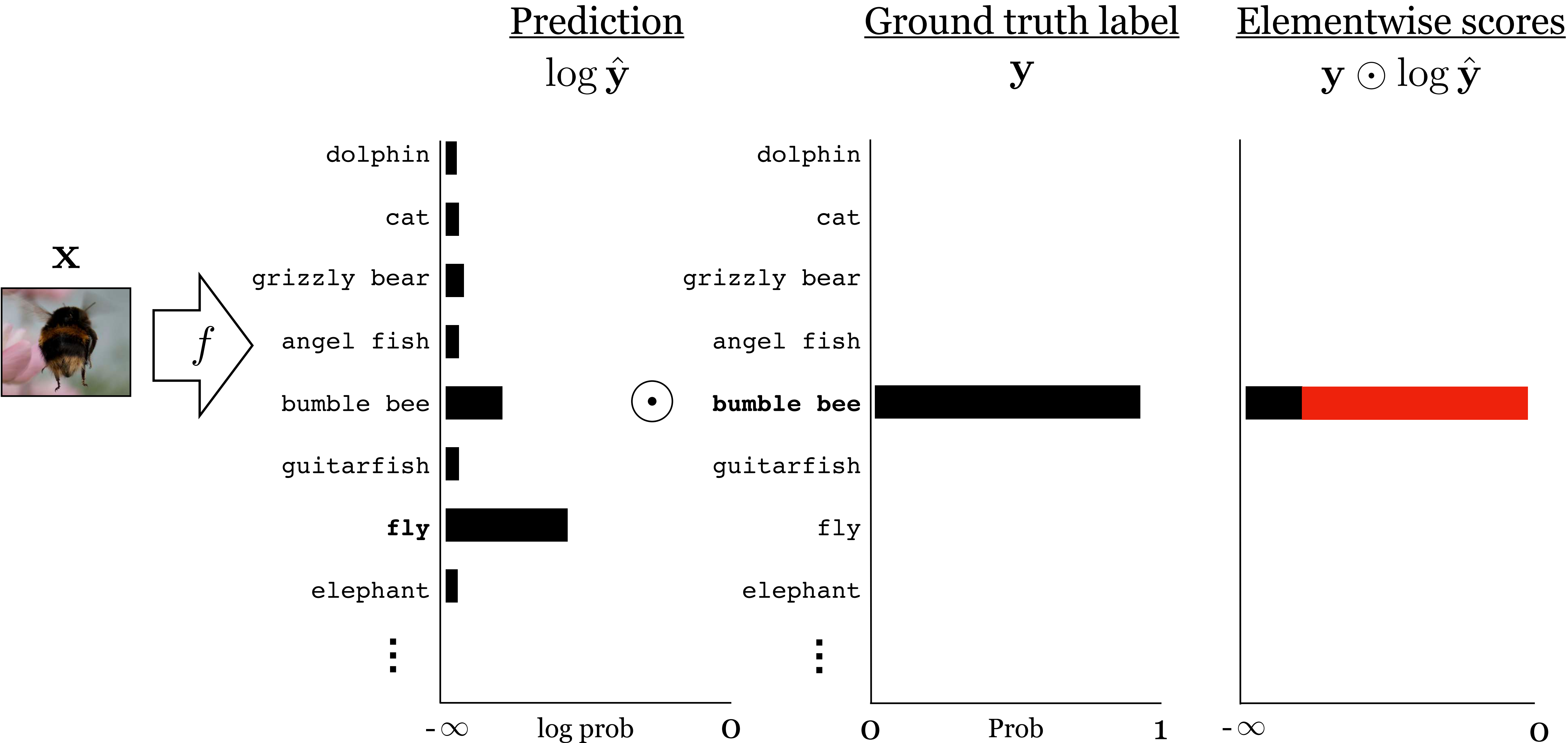


[0,0,0,0,0,1,0,0,...]









Softmax regression (a.k.a. multinomial logistic regression)

$$f_{\theta} : \mathcal{X} \rightarrow \Delta^{K-1}$$

← **Parameterized function, f :** from an input image to a probability mass function.

$$\mathbf{z} = z_{\theta}(\mathbf{x})$$

← **logits:** vector of K scores, one for each class

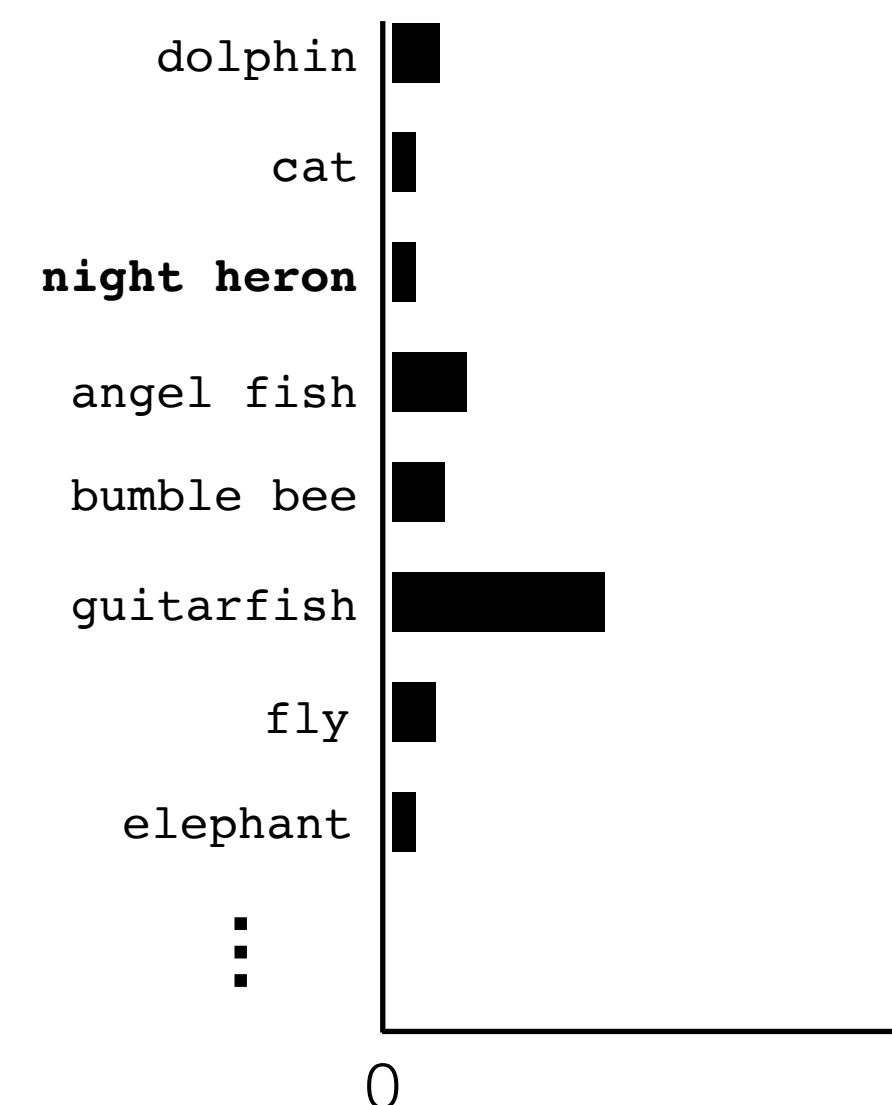
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

← squash into a non-negative vector that sums to 1
— i.e. **a probability mass function!**

$$\hat{y}_j = \frac{e^{-z_j}}{\sum_{k=1}^K e^{-z_k}}$$

$$\hat{\mathbf{y}} = f_{\theta}(\mathbf{x}) = \text{softmax}(z_{\theta}(\mathbf{x}))$$

$$\hat{\mathbf{y}} =$$



Softmax regression (a.k.a. multinomial logistic regression)

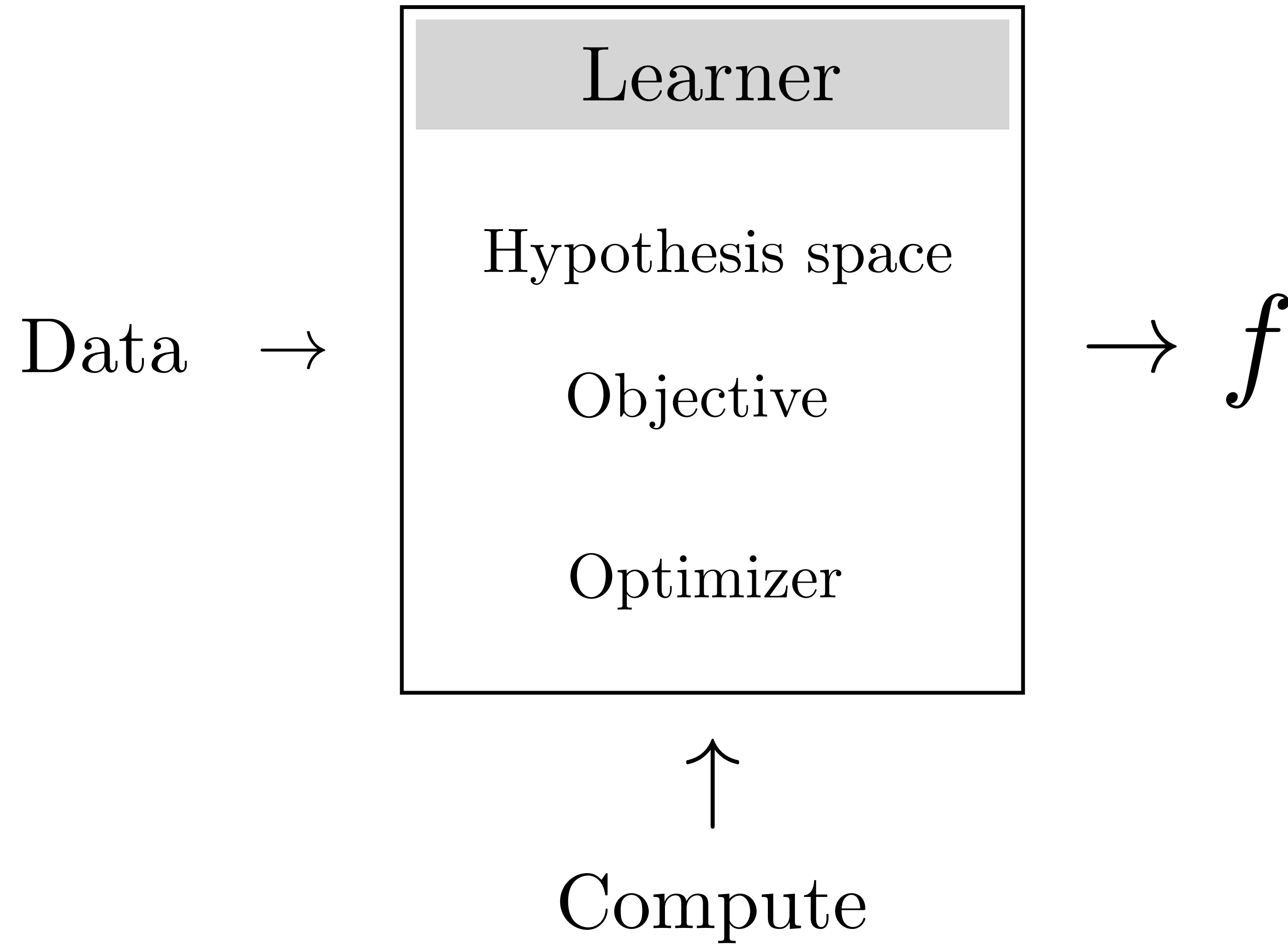
Probabilistic interpretation:

$\hat{\mathbf{y}} \equiv [P_{\theta}(Y = 1|X = \mathbf{x}), \dots, P_{\theta}(Y = K|X = \mathbf{x})]$ $\leftarrow$ predicted probability of each class given input $\mathbf{x}$

$H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$ $\leftarrow$ picks out the -log likelihood of the ground truth class $\mathbf{y}$ under the model prediction $\hat{\mathbf{y}}$

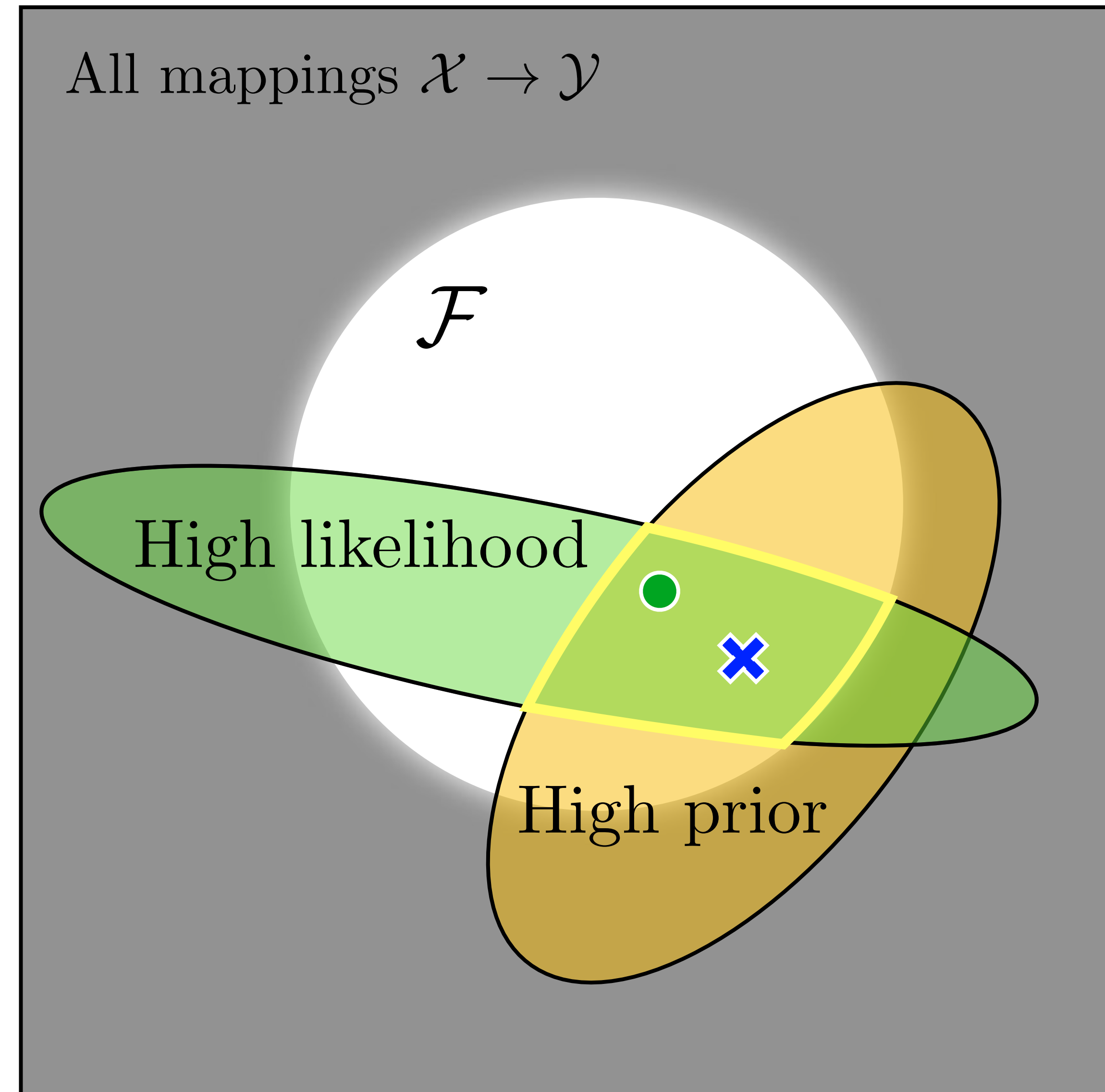
$f^* = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^N H(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$ $\leftarrow$ Make predictions, $\hat{\mathbf{y}}$, that maximize the likelihood of the ground truth classes.

Key Ingredients of learning from data



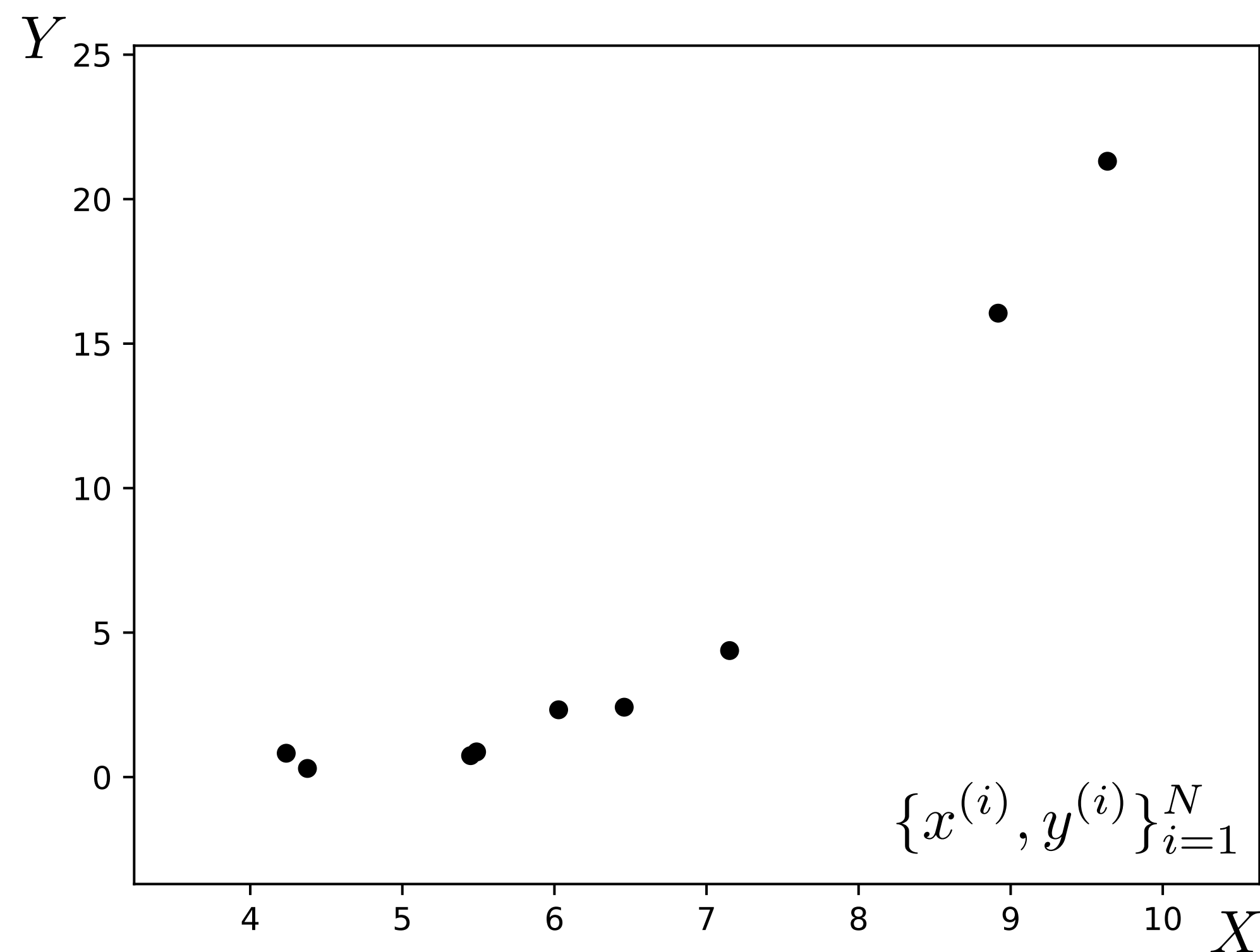
Chapter 11

The Problem of Generalization



Linear regression

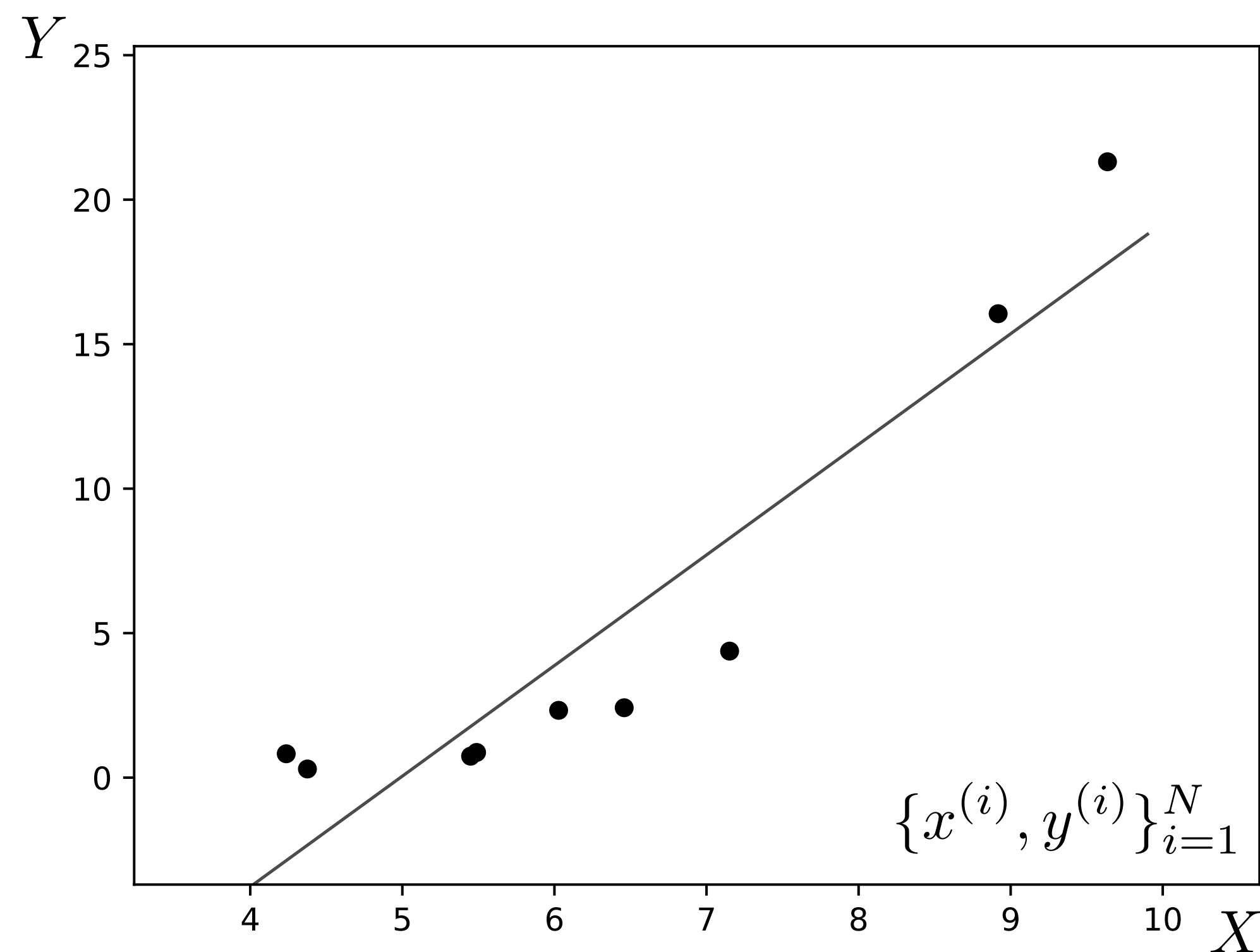
Training data



$$f_{\theta}(x) = \theta_0 + \theta_1 x$$

Linear regression

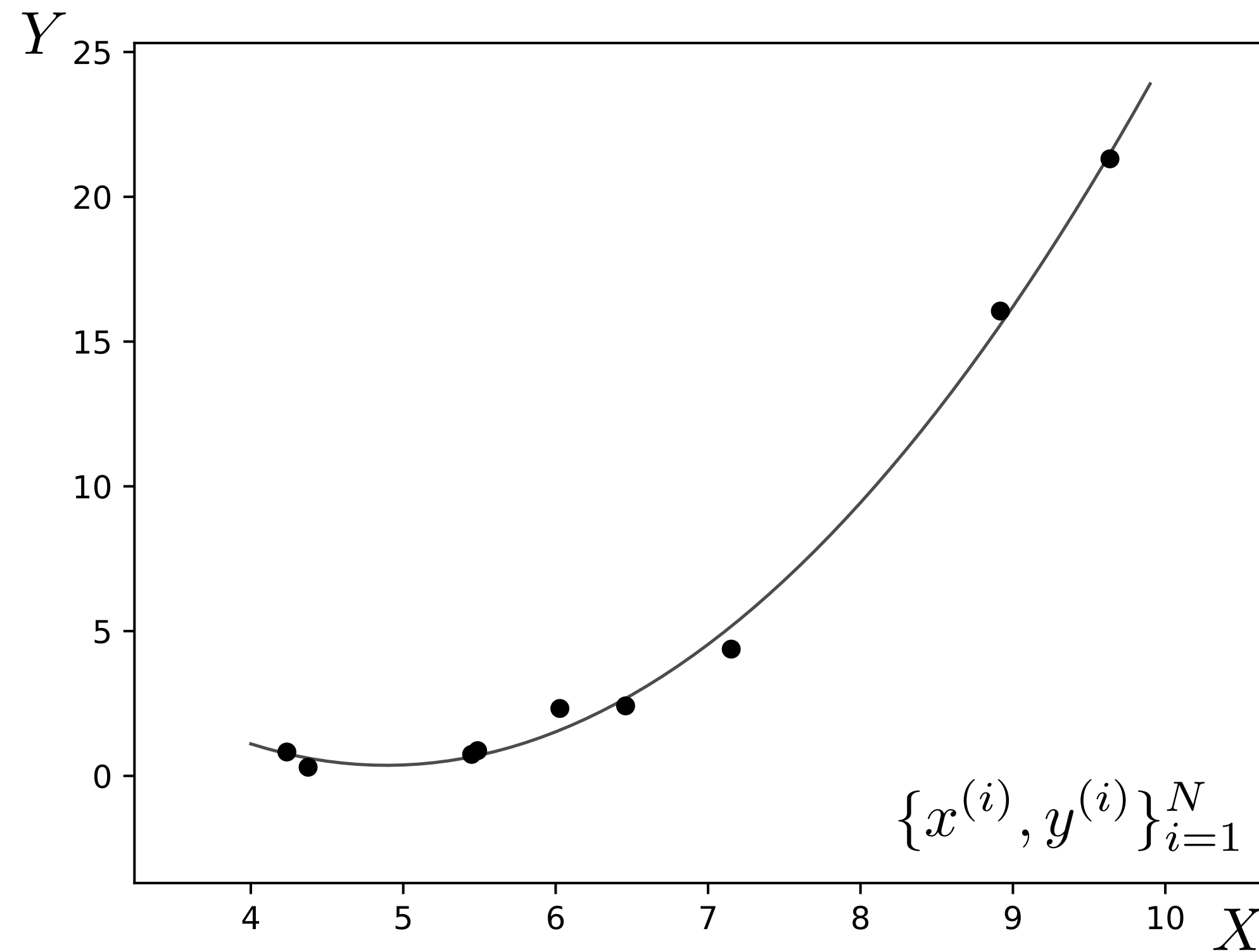
Training data



$$f_{\theta}(x) = \theta_0 + \theta_1 x$$

Polynomial regression

Training data

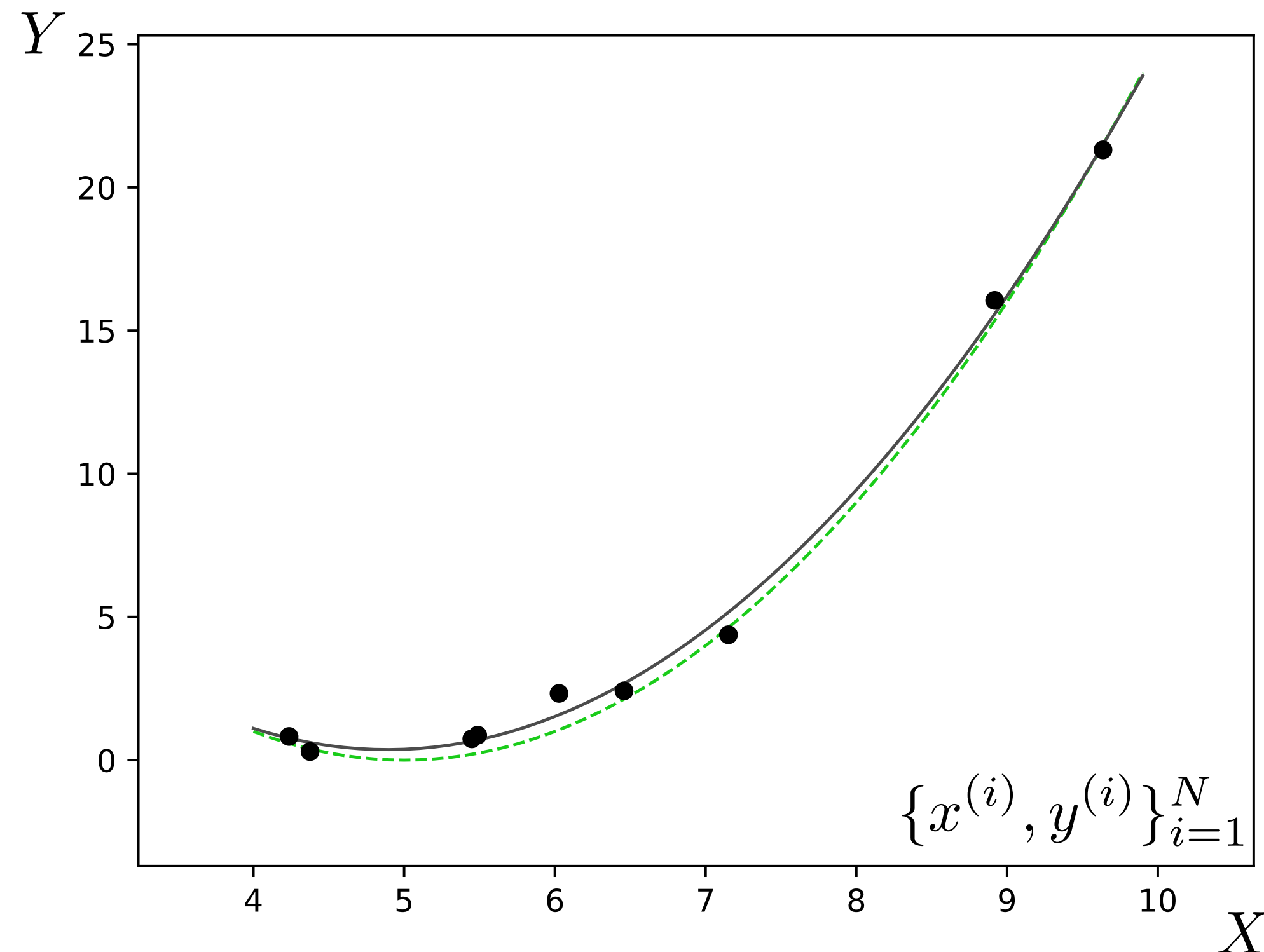


$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

K-th degree polynomial regression

Training data



True data-generating process

p_{data}

$$Y = X^2 + 1$$

◁ true underlying relationship

$$\epsilon \sim \mathcal{N}(0, 1)$$

◁ observation noise

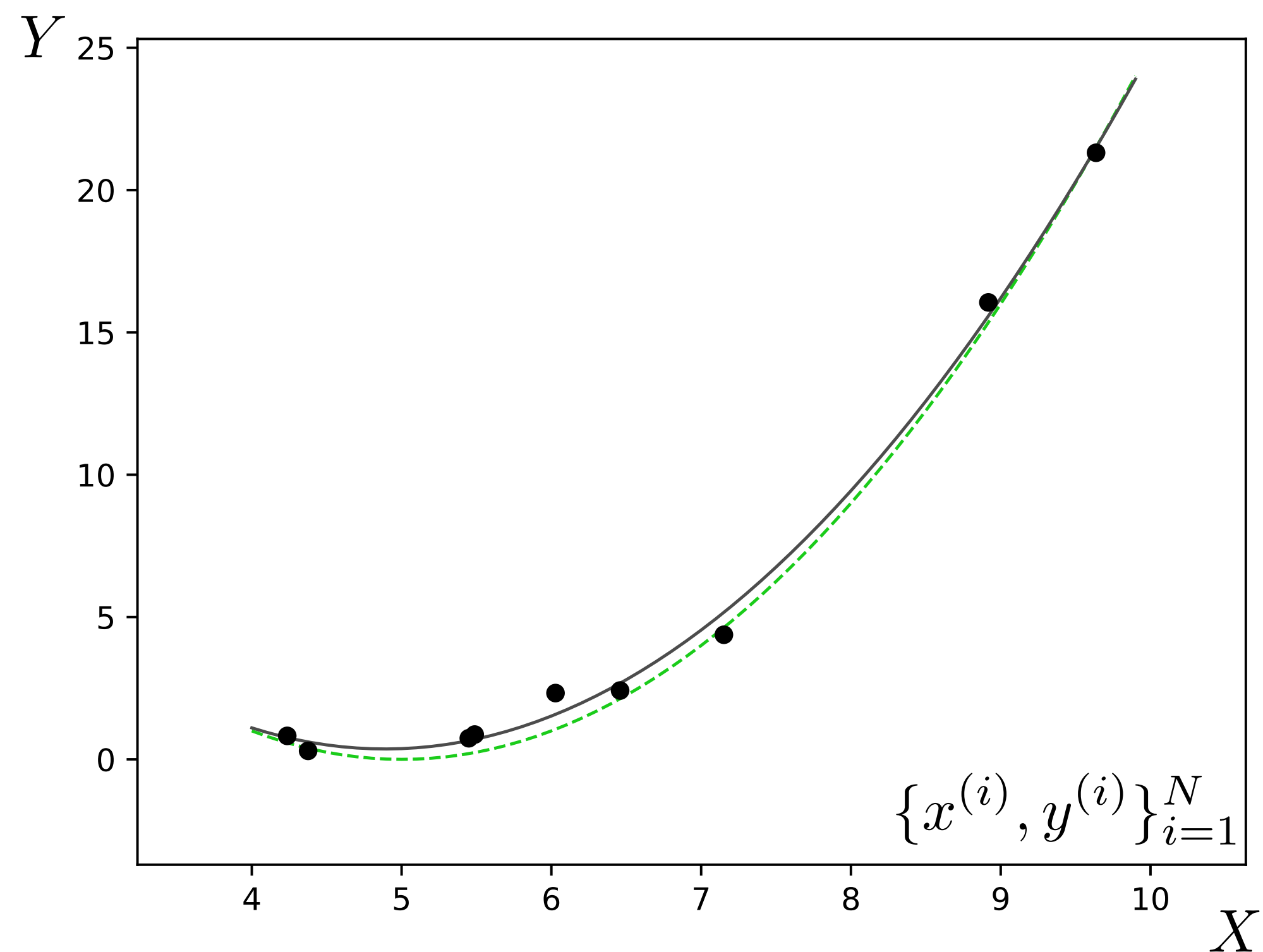
$$Y' = Y + \epsilon$$

◁ noisy observations

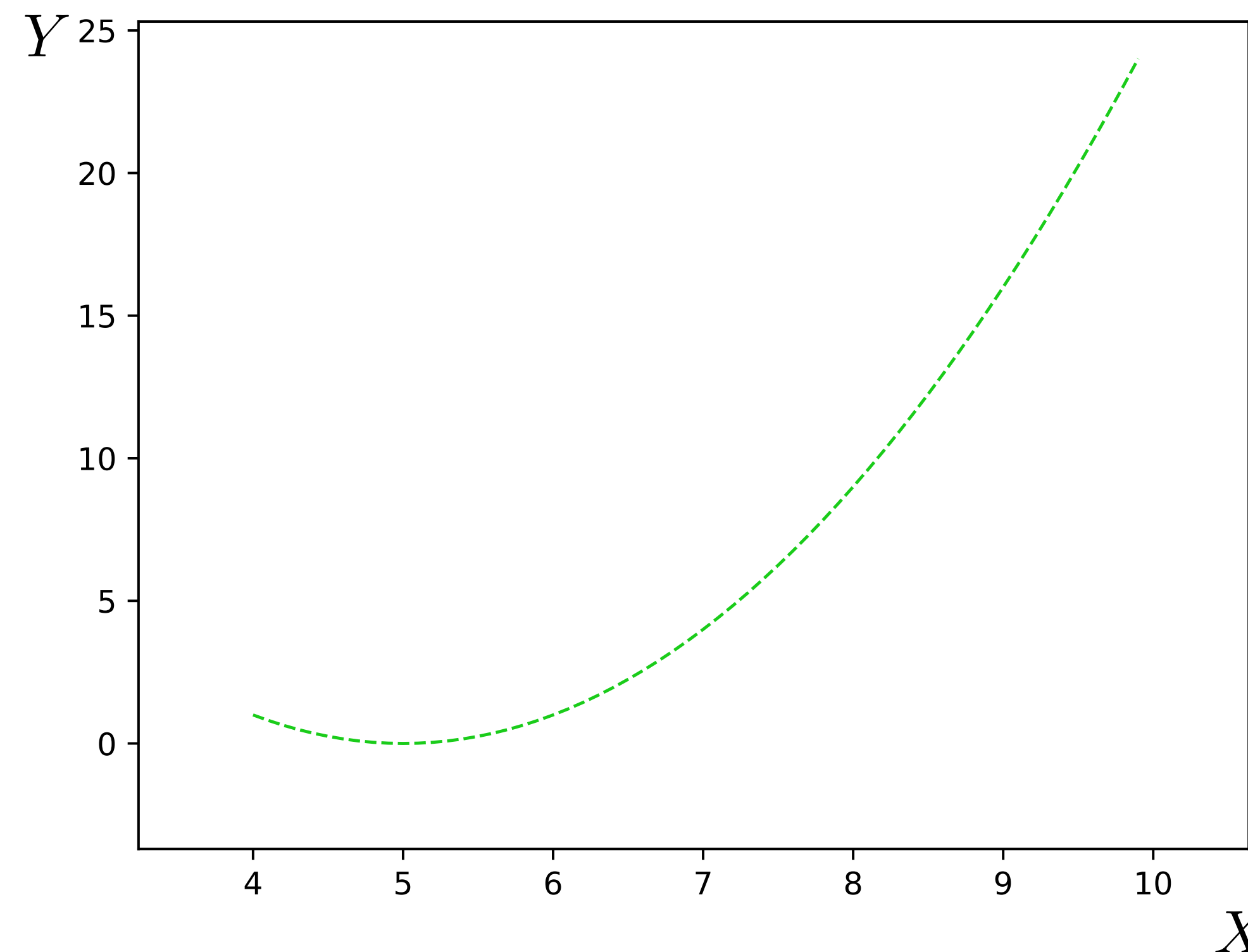
$$x, y \sim p(X, Y')$$

◁ data-generating process

Training data



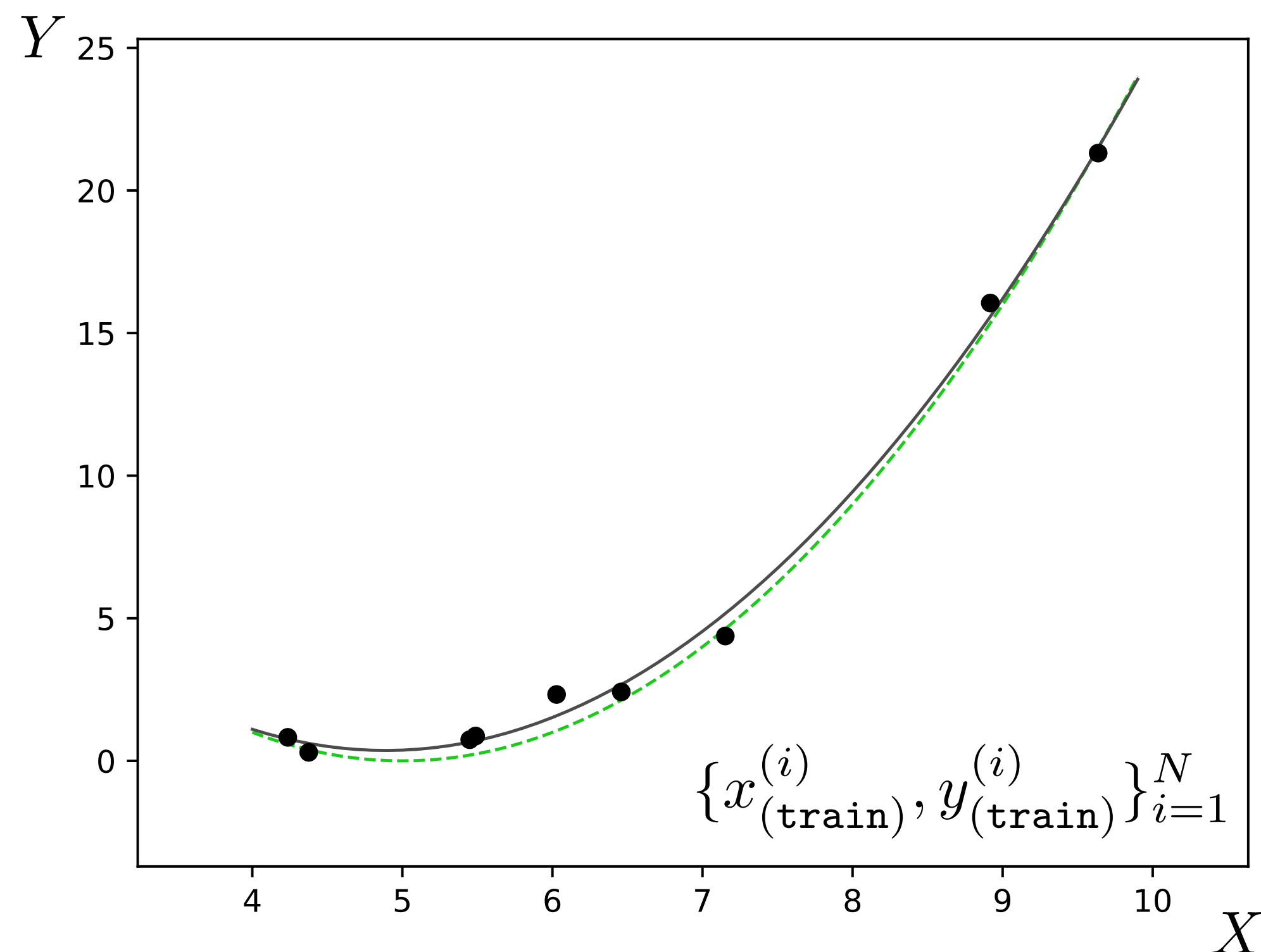
Test data



True data-generating process

p_{data}

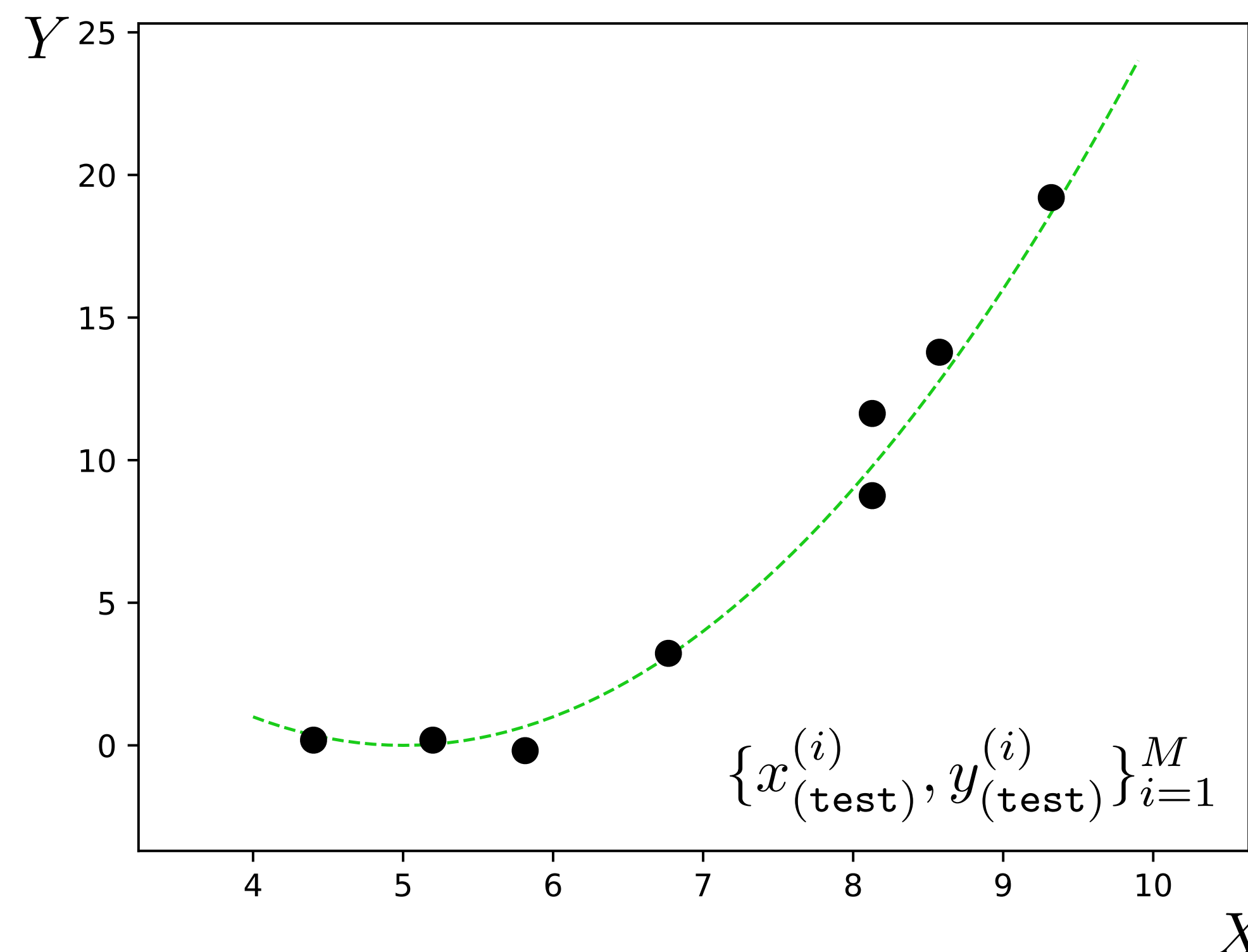
Training data



True data-generating process

p_{data}

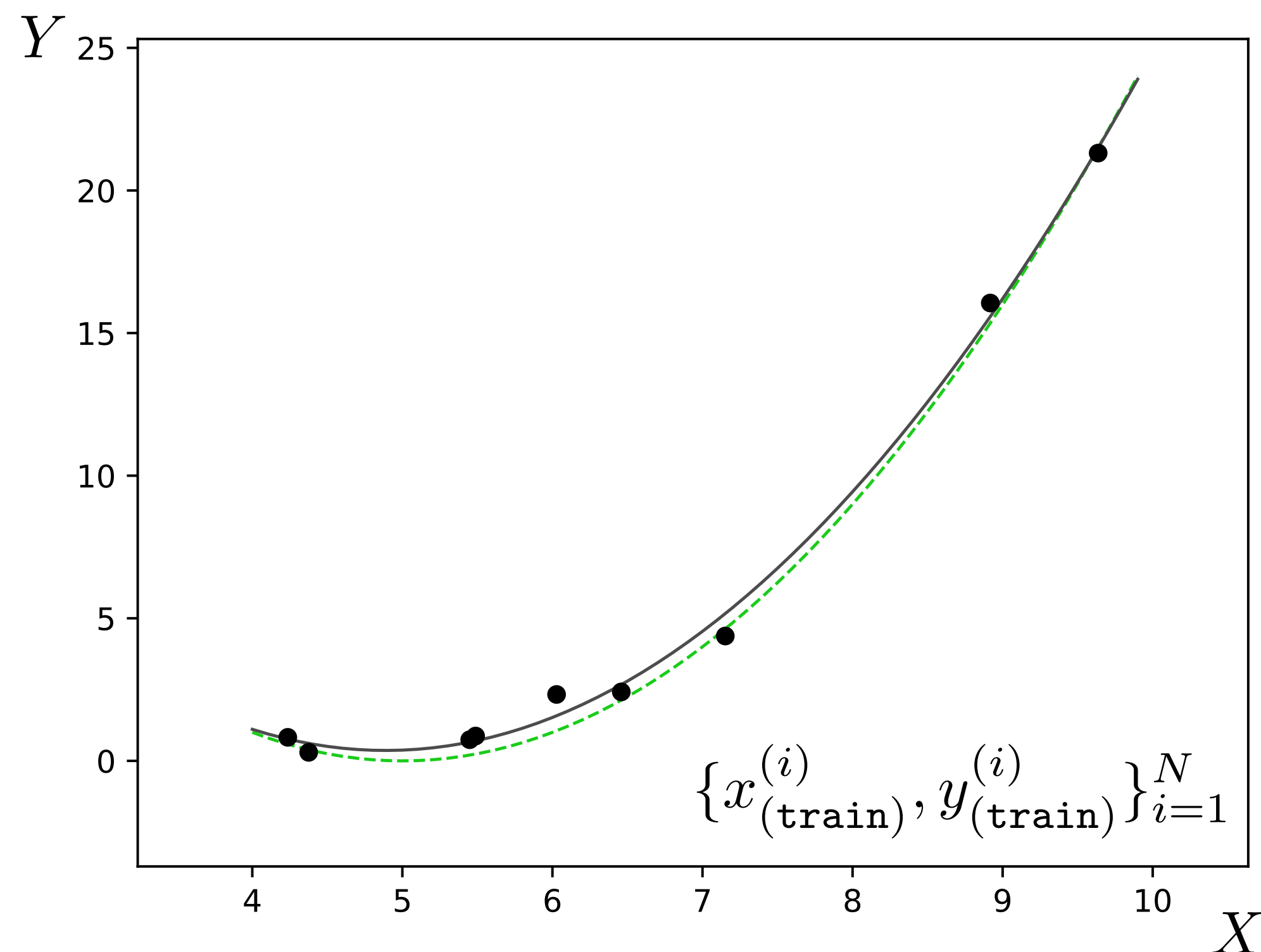
Test data



$$\{x_{(\text{train})}^{(i)}, y_{(\text{train})}^{(i)}\} \stackrel{\text{iid}}{\sim} p_{\text{data}}$$

$$\{x_{(\text{test})}^{(i)}, y_{(\text{test})}^{(i)}\} \stackrel{\text{iid}}{\sim} p_{\text{data}}$$

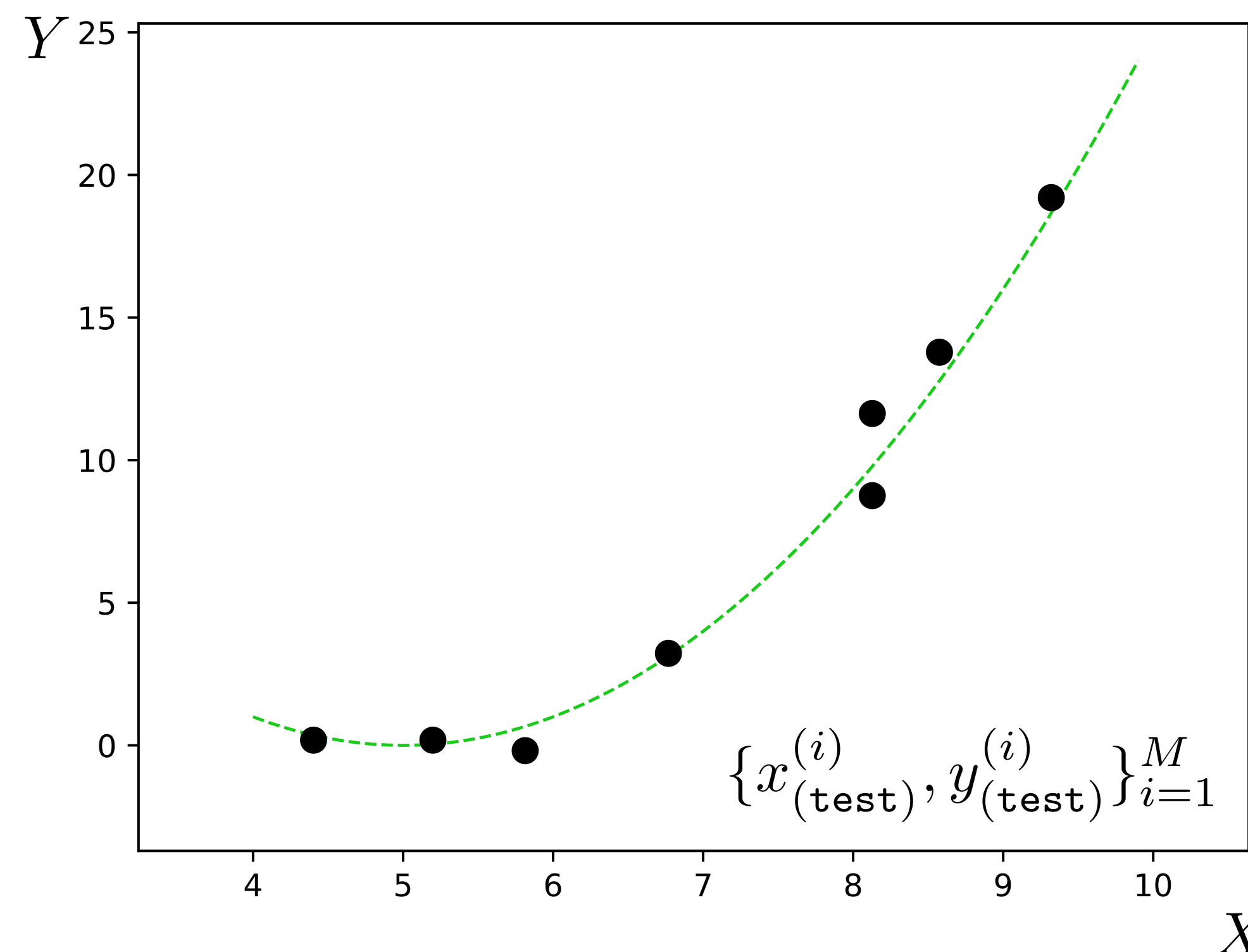
Training data



Training objective:

$$\sum_{i=1}^N (f_\theta(x_{\text{train}}^{(i)}) - y_{\text{train}}^{(i)})^2$$

Test data

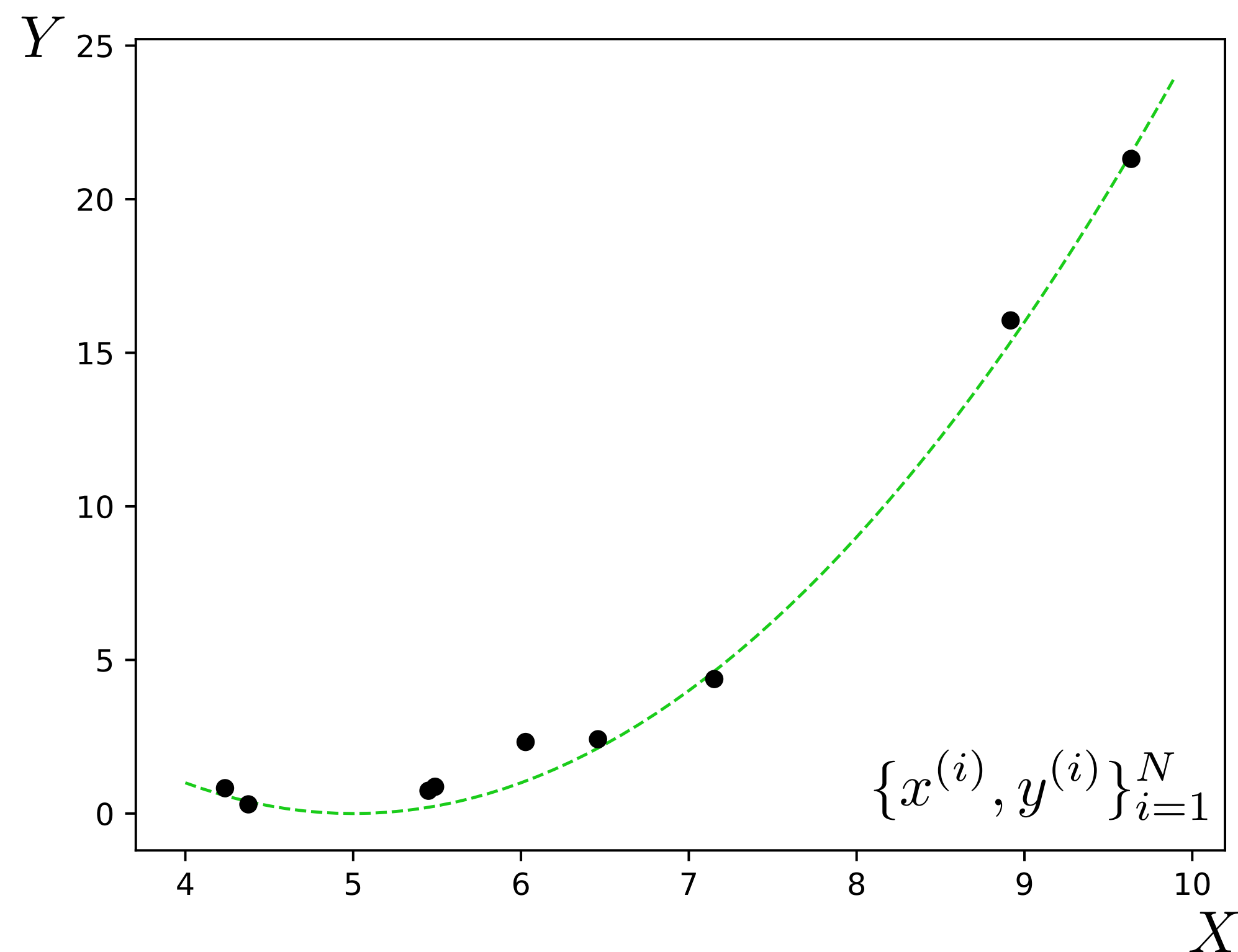


Test time evaluation:

$$\sum_{i=1}^M (f_\theta(x_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)})^2$$

What happens as we add more basis functions?

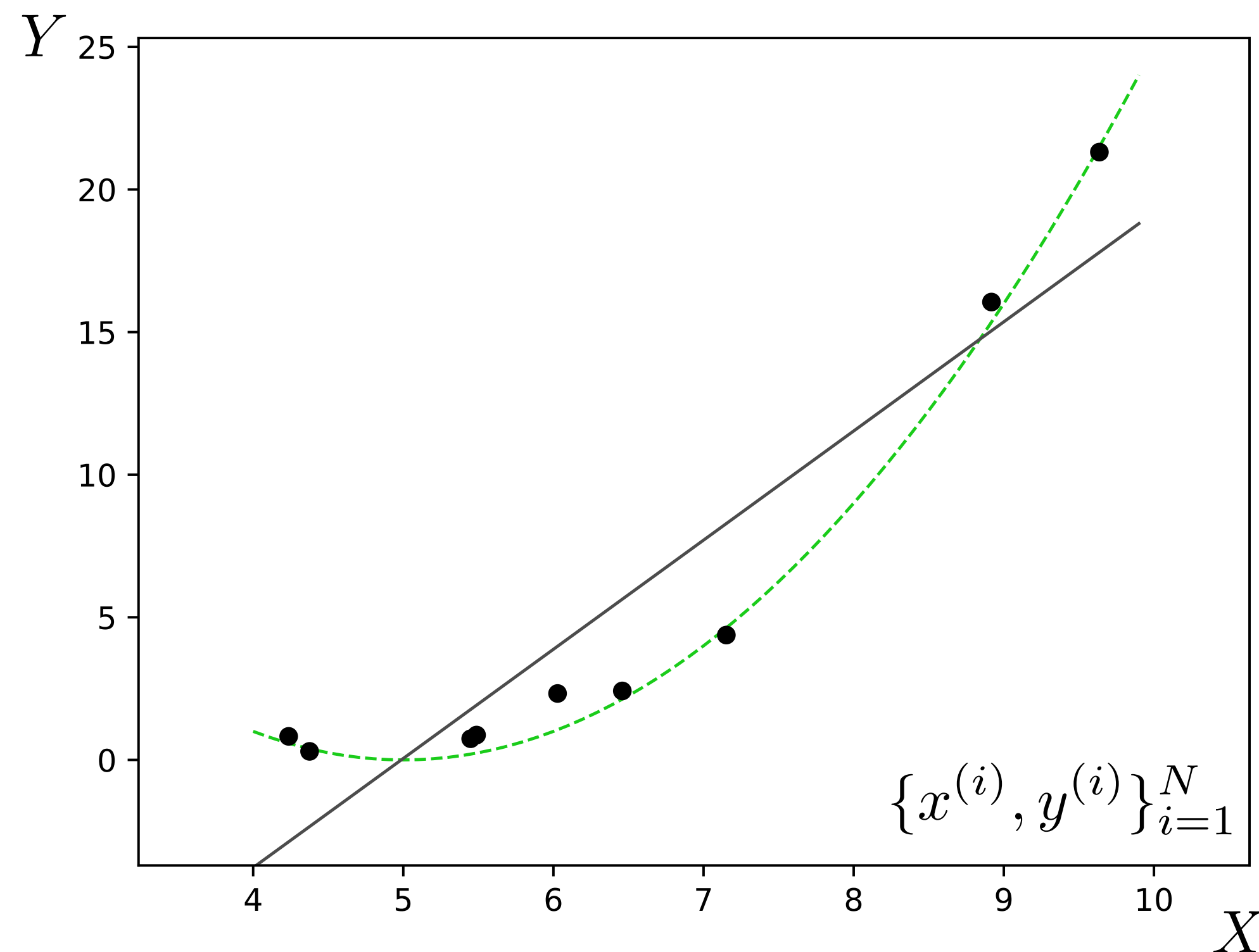
Training data



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

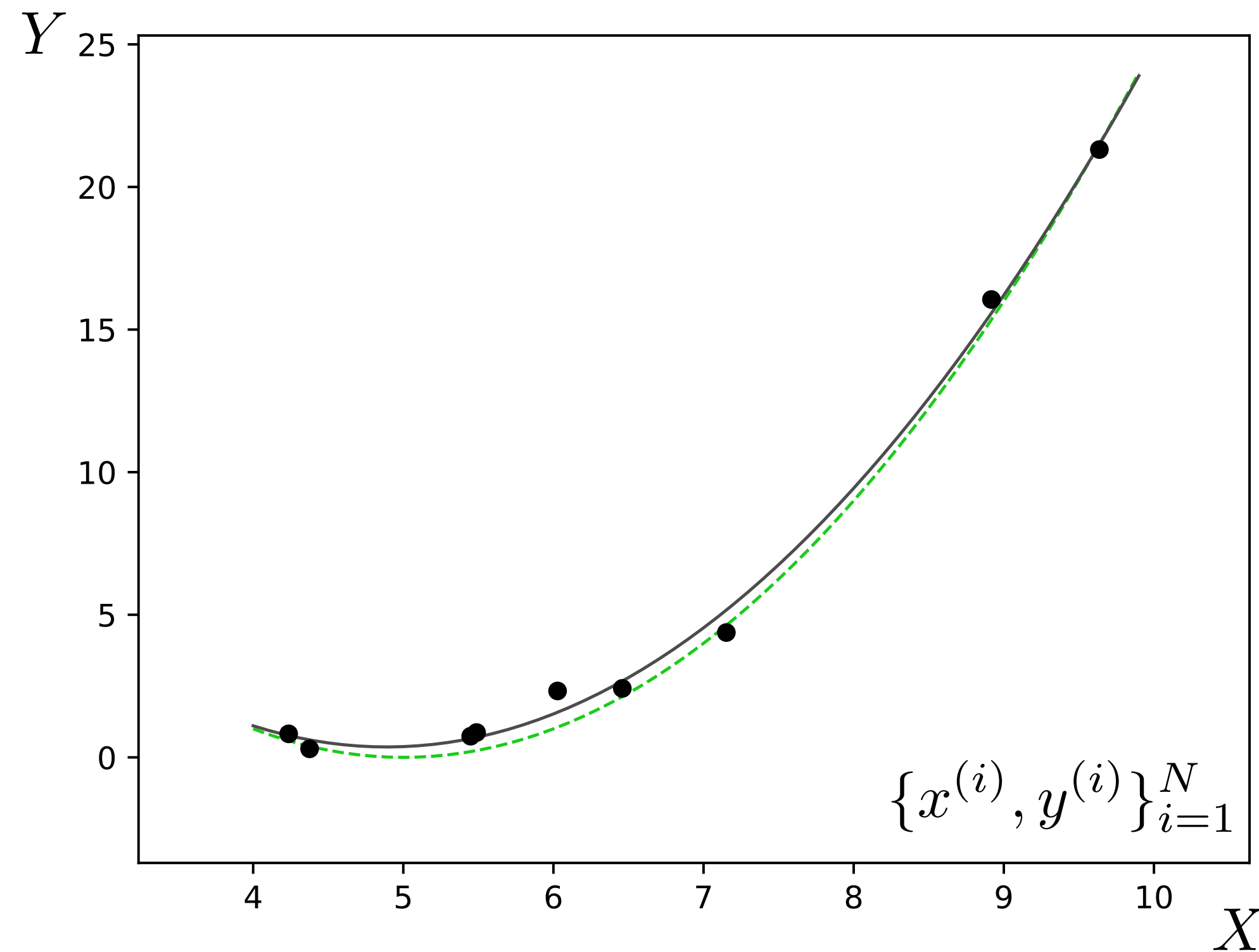
K = 1



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

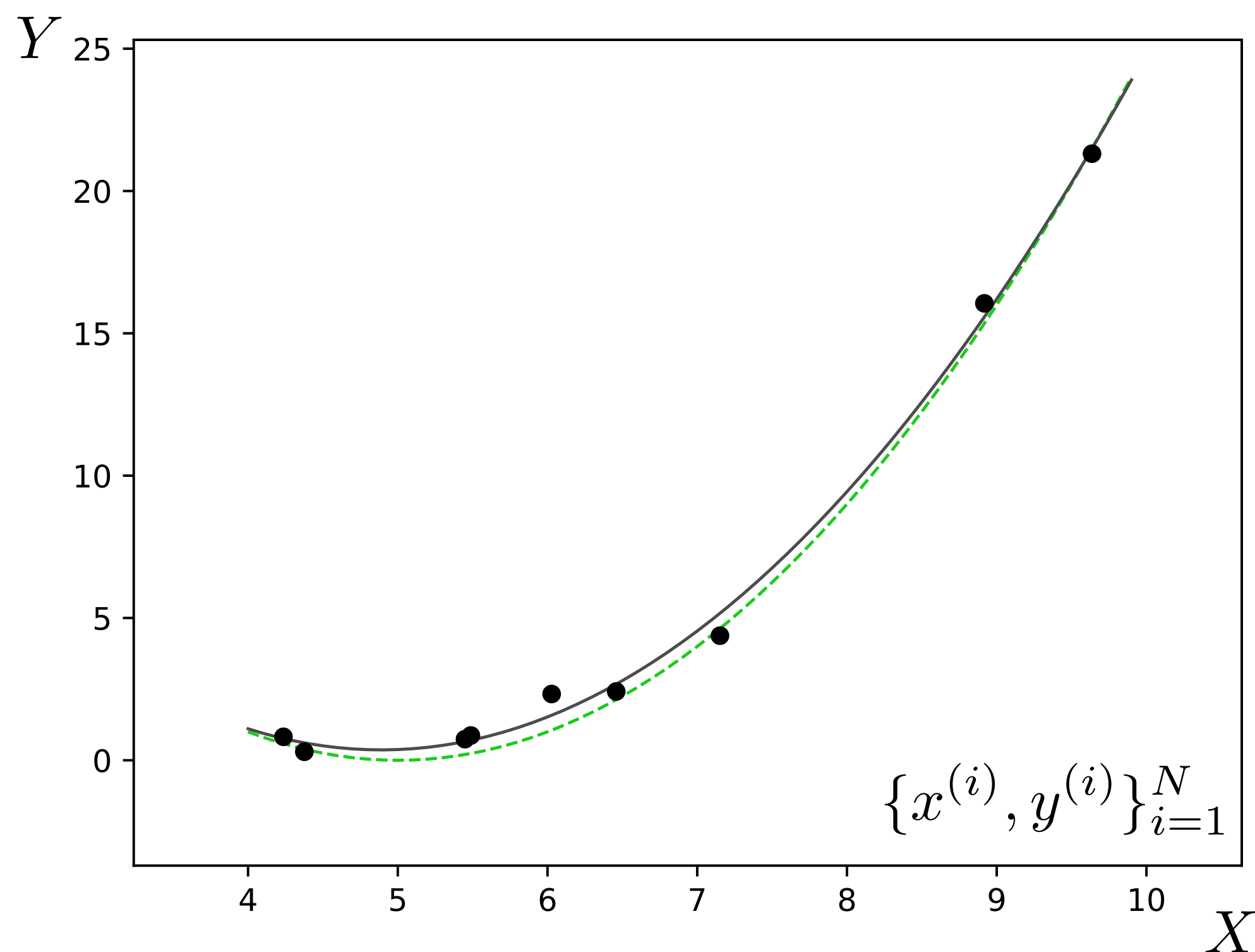
K = 2



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

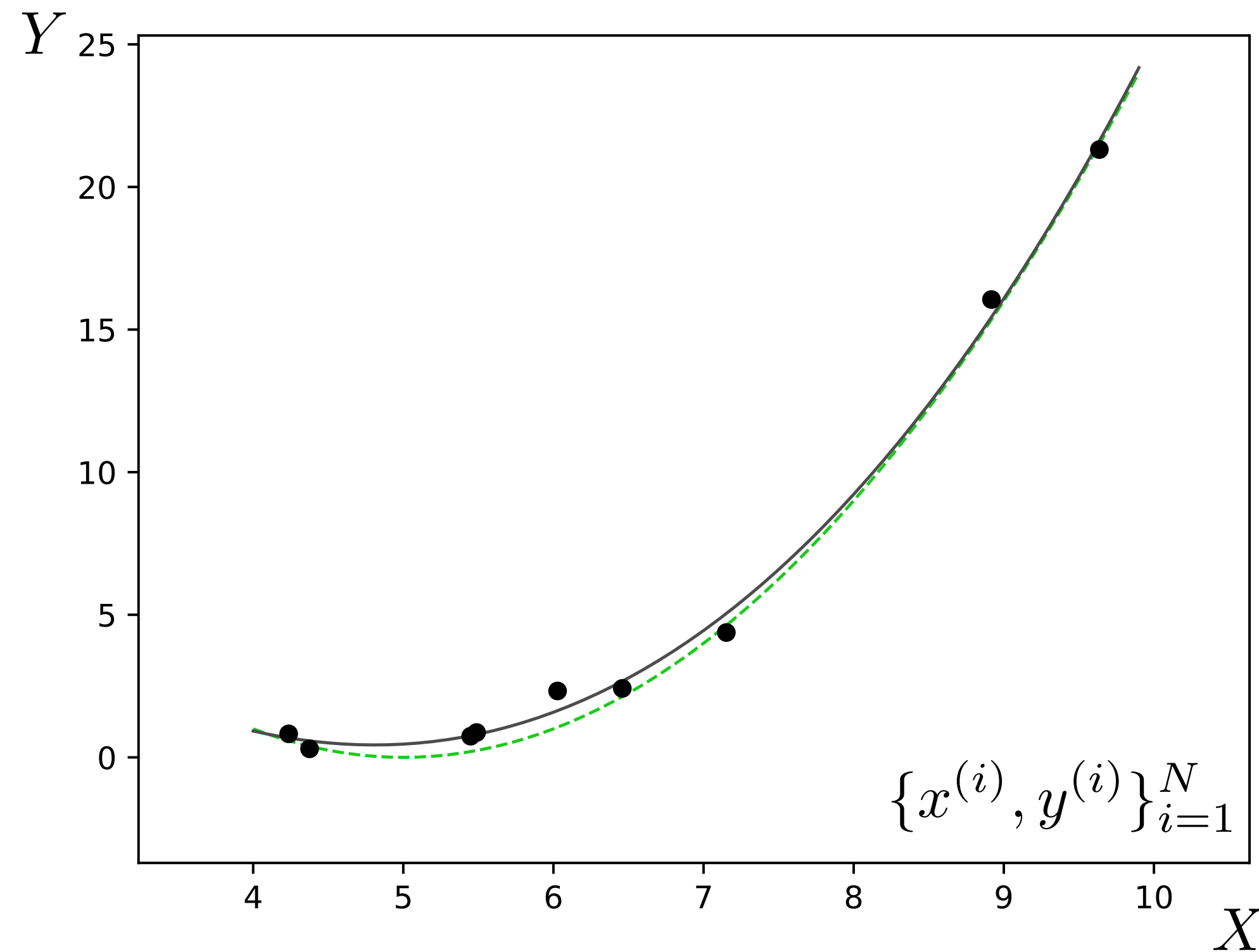
K = 3



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

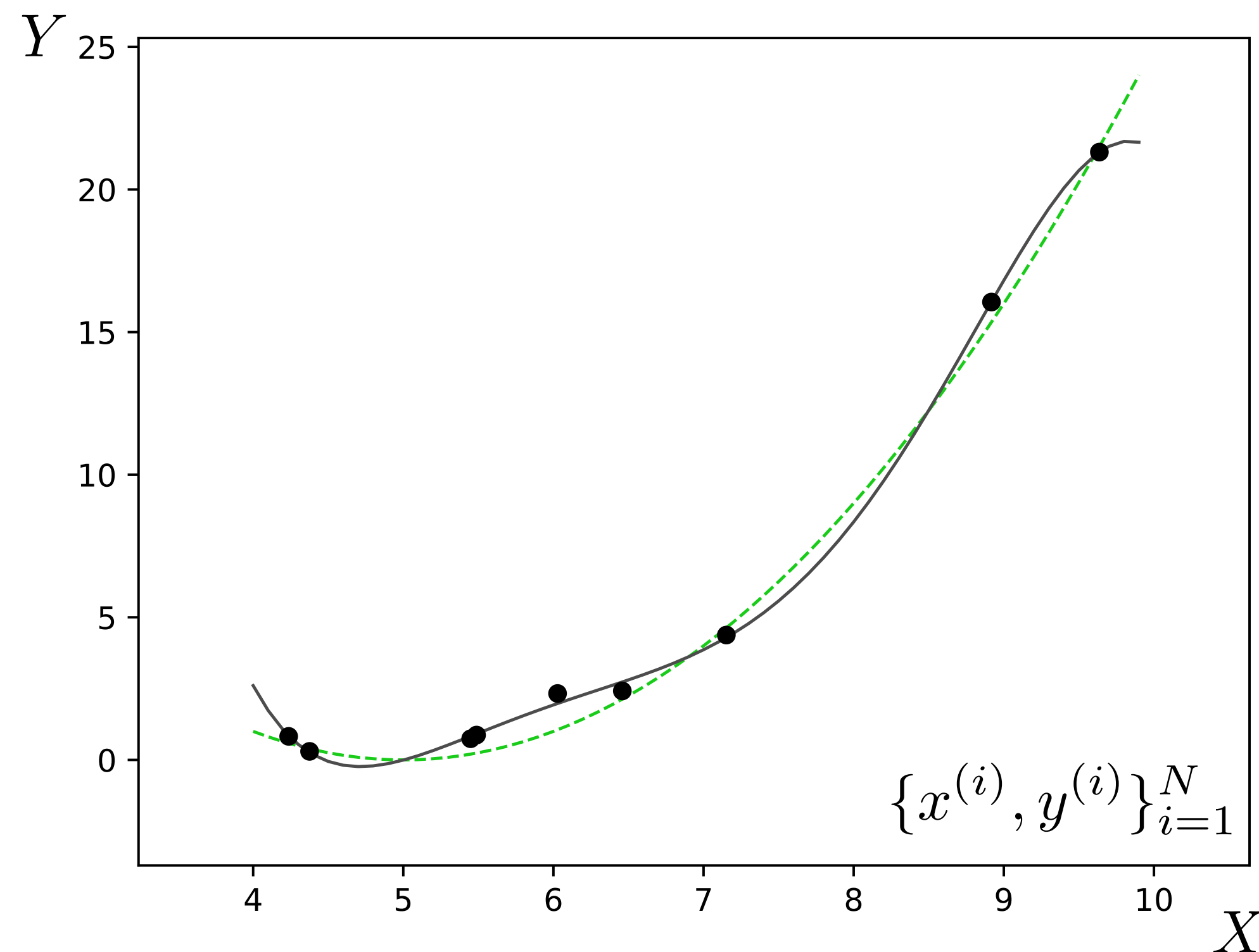
K = 4



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

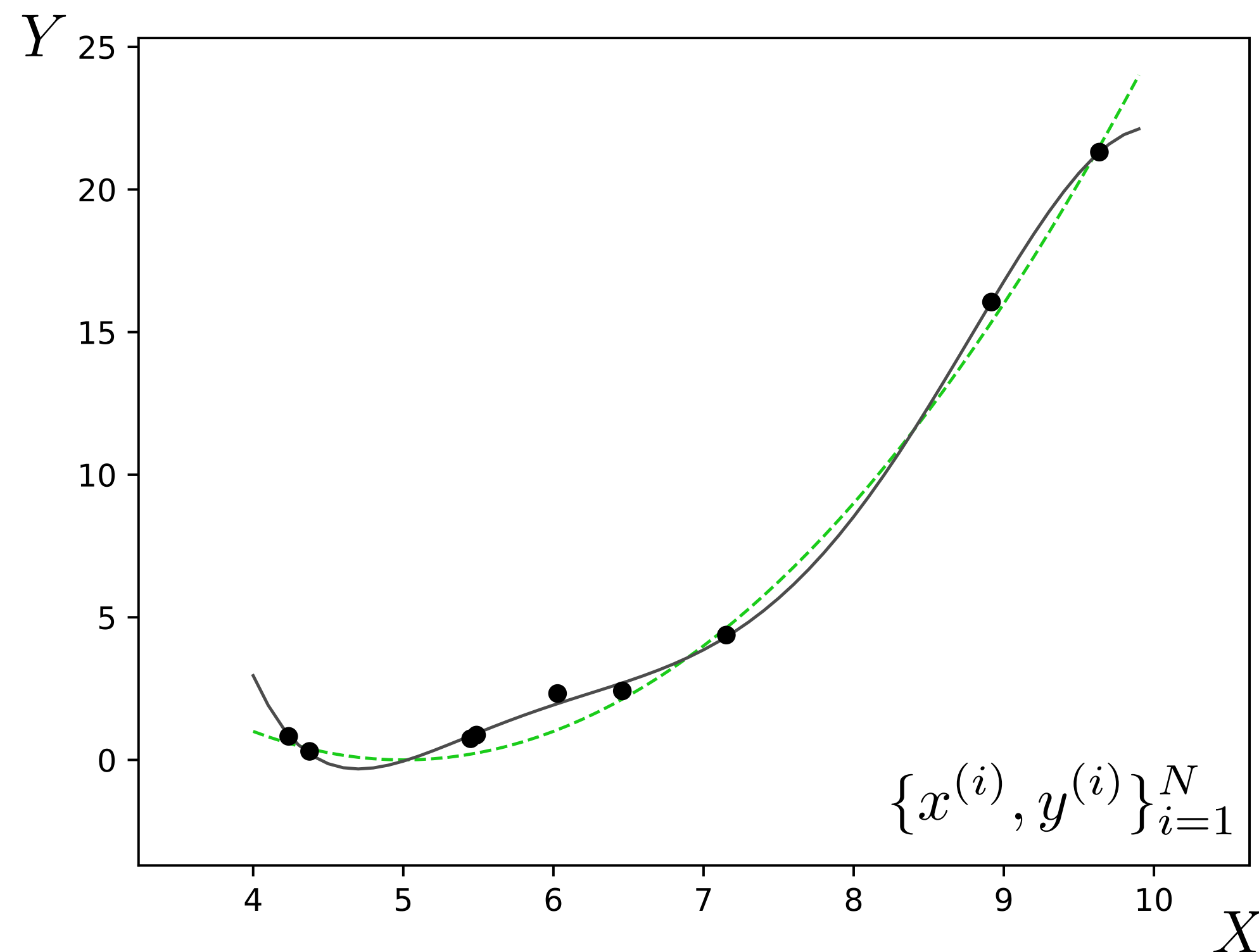
K = 5



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

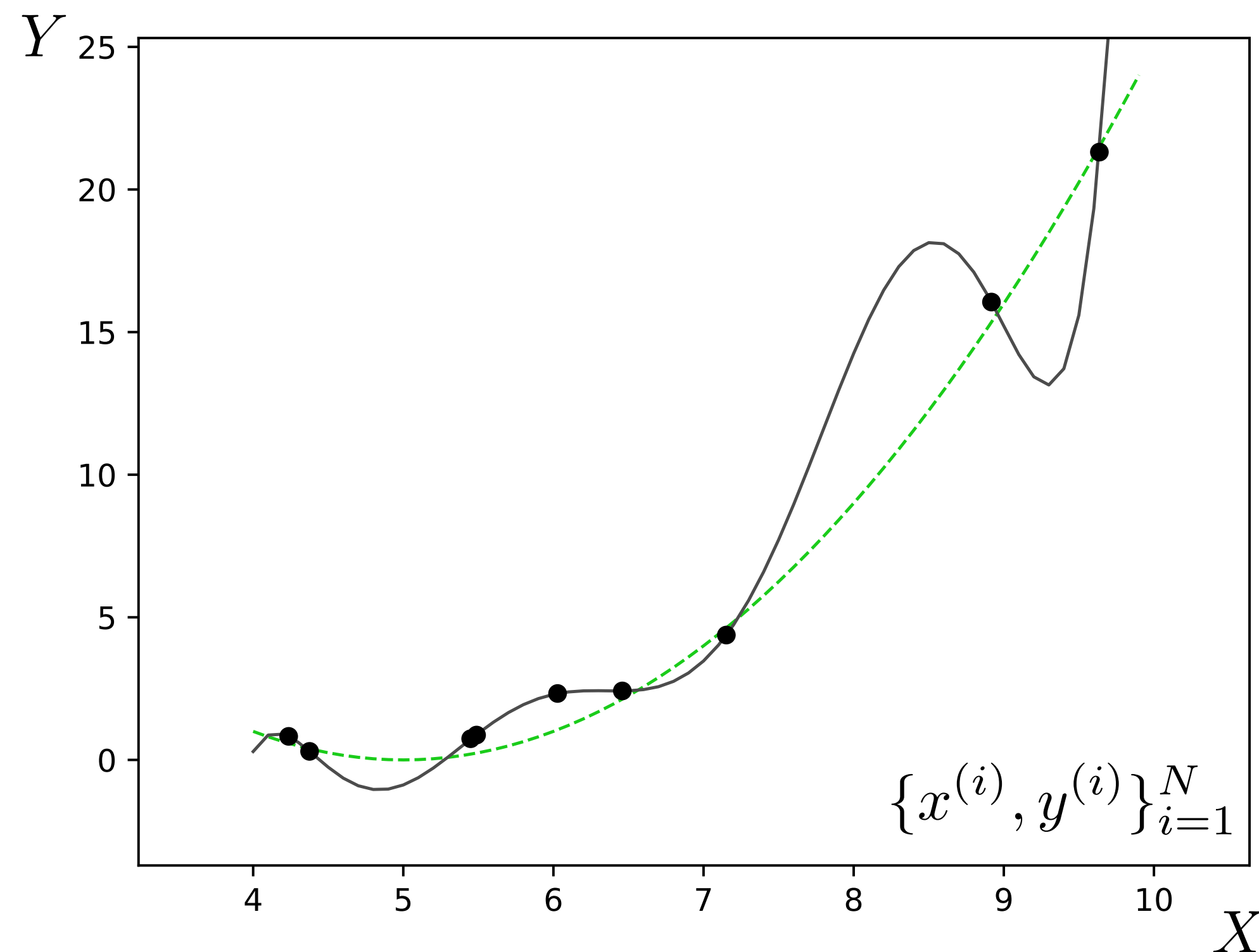
K = 6



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

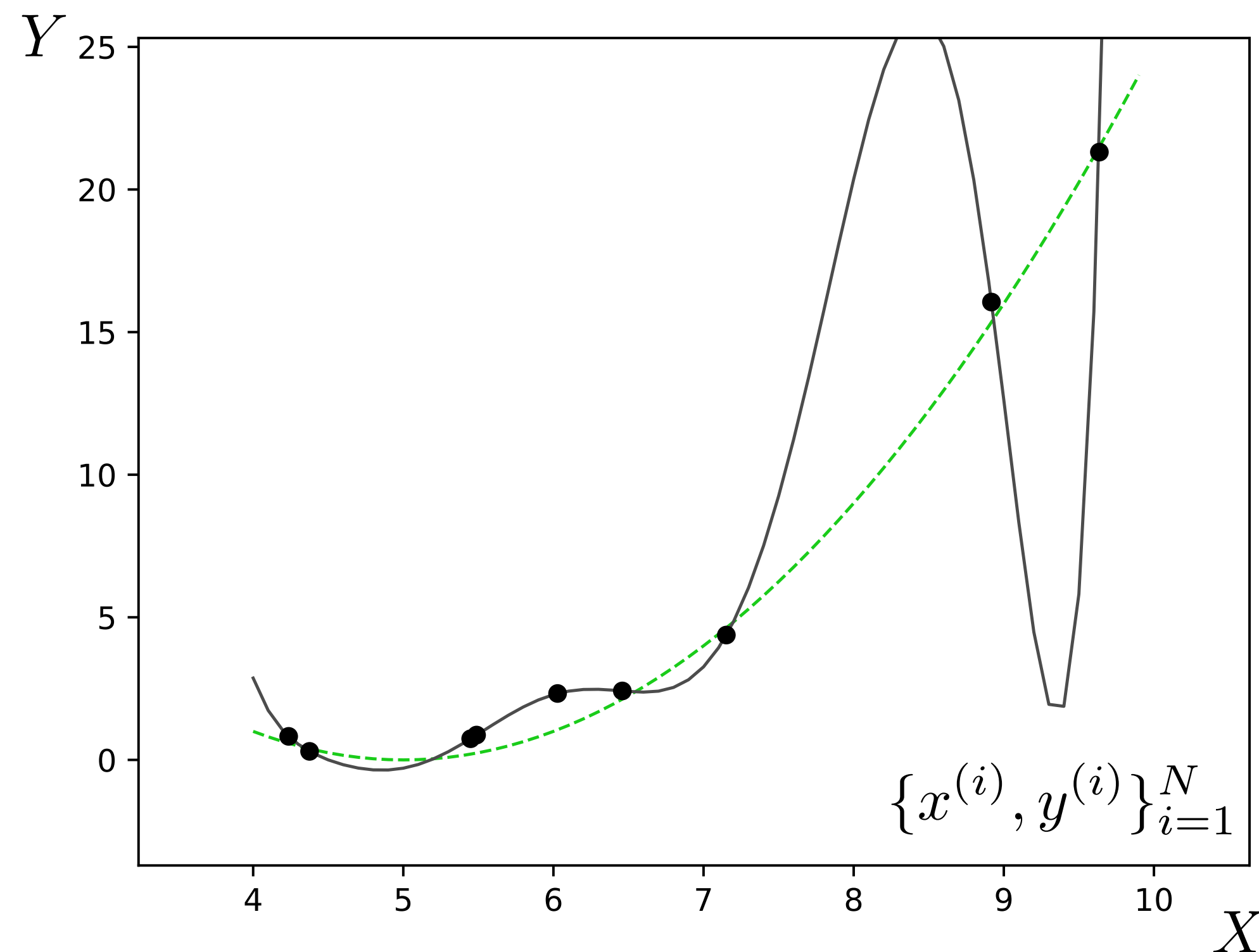
K = 7



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

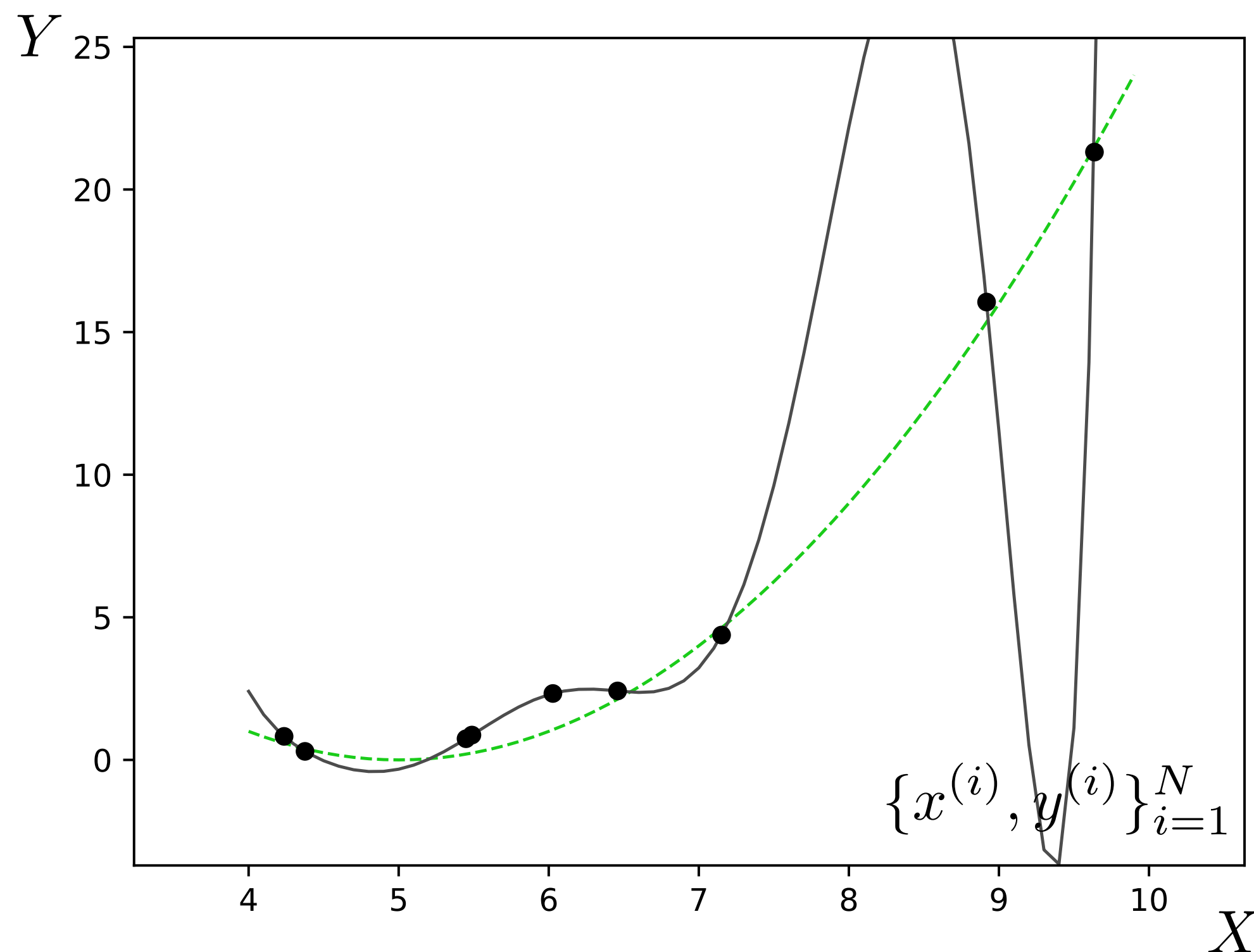
K = 8



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

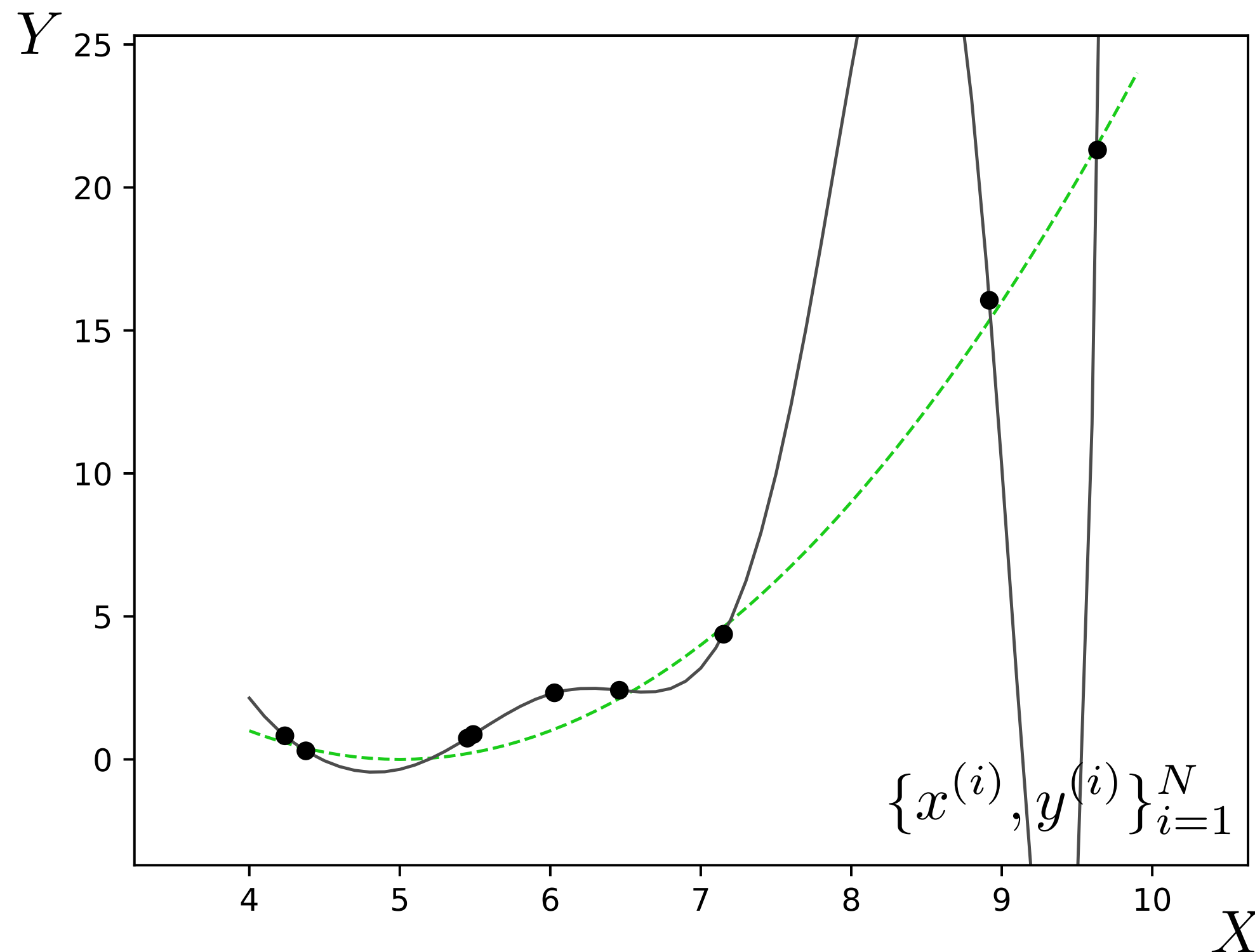
K = 9



$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

What happens as we add more basis functions?

K = 10

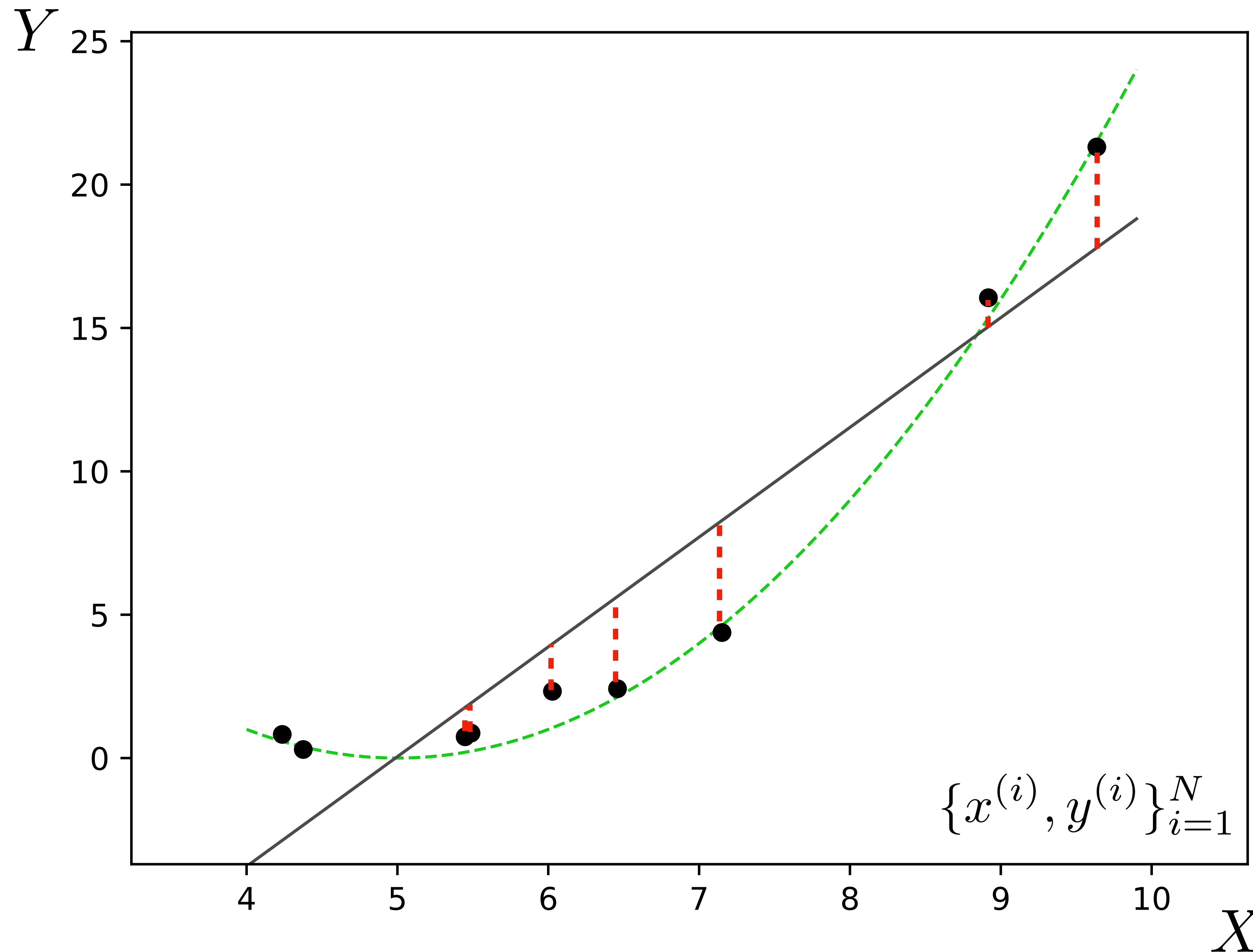


$$f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k$$

This phenomenon is called **overfitting**.

It occurs when we have too high **capacity** a model, e.g., too many free parameters, too few data points to pin these parameters down. Instead, here we are fitting to the noise.

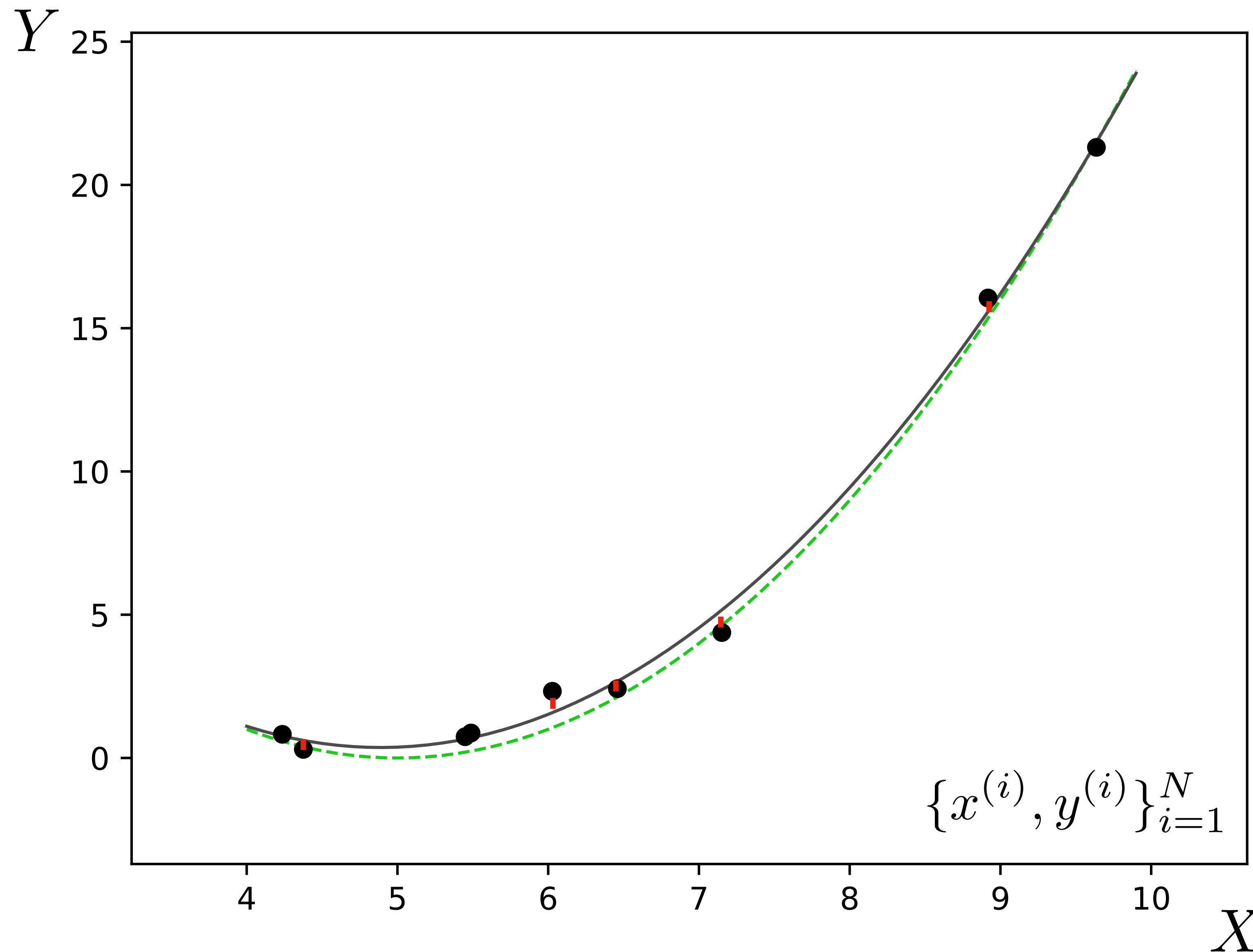
$$K = 1$$



When the model does not have the capacity to capture the true function, we call this **underfitting**.

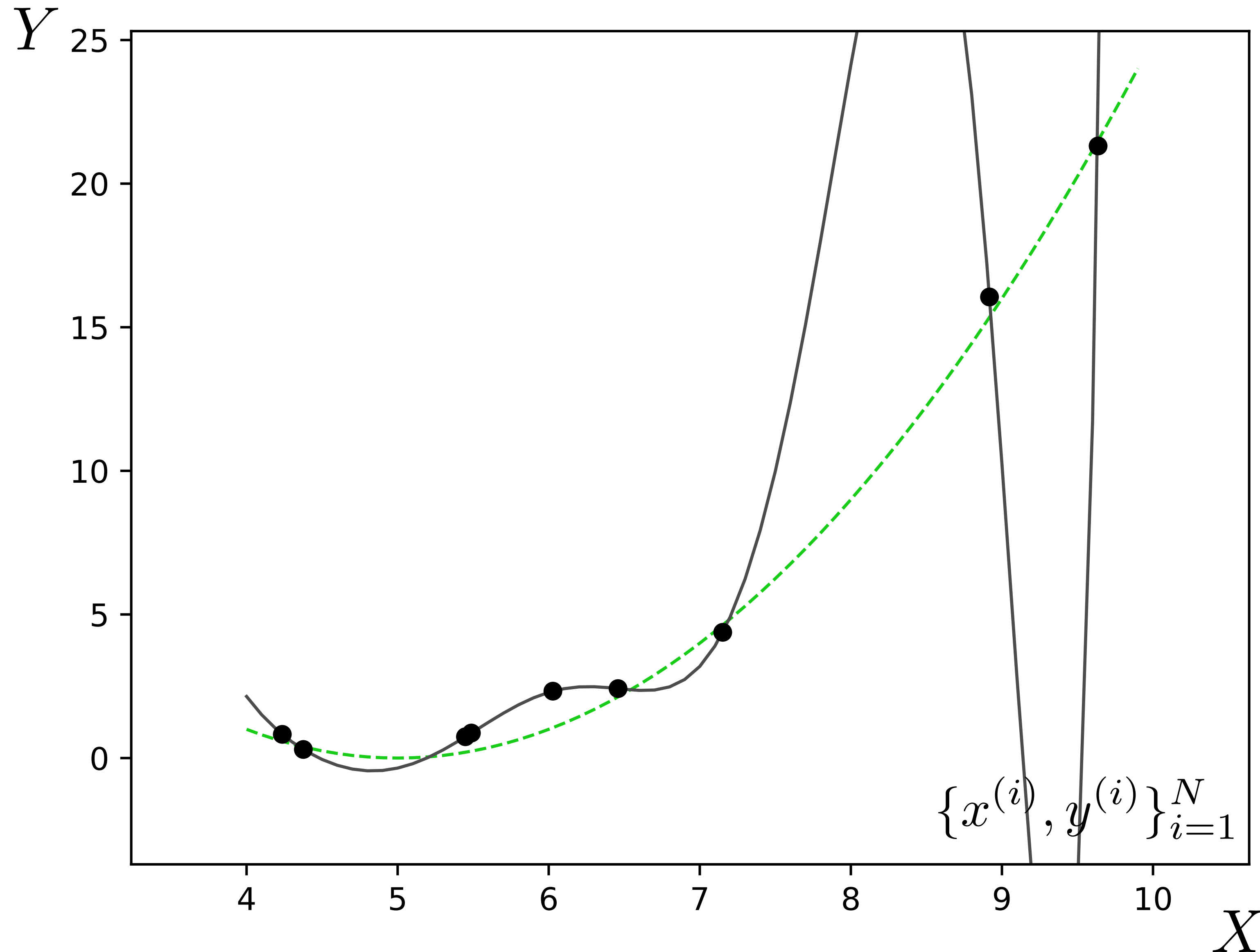
An underfit model will have high **error** on the training points. This error is known as **approximation error**.

$$K = 2$$



The true function is a quadratic, so a quadratic model ($K=2$) fits quite well.

$$K = 10$$

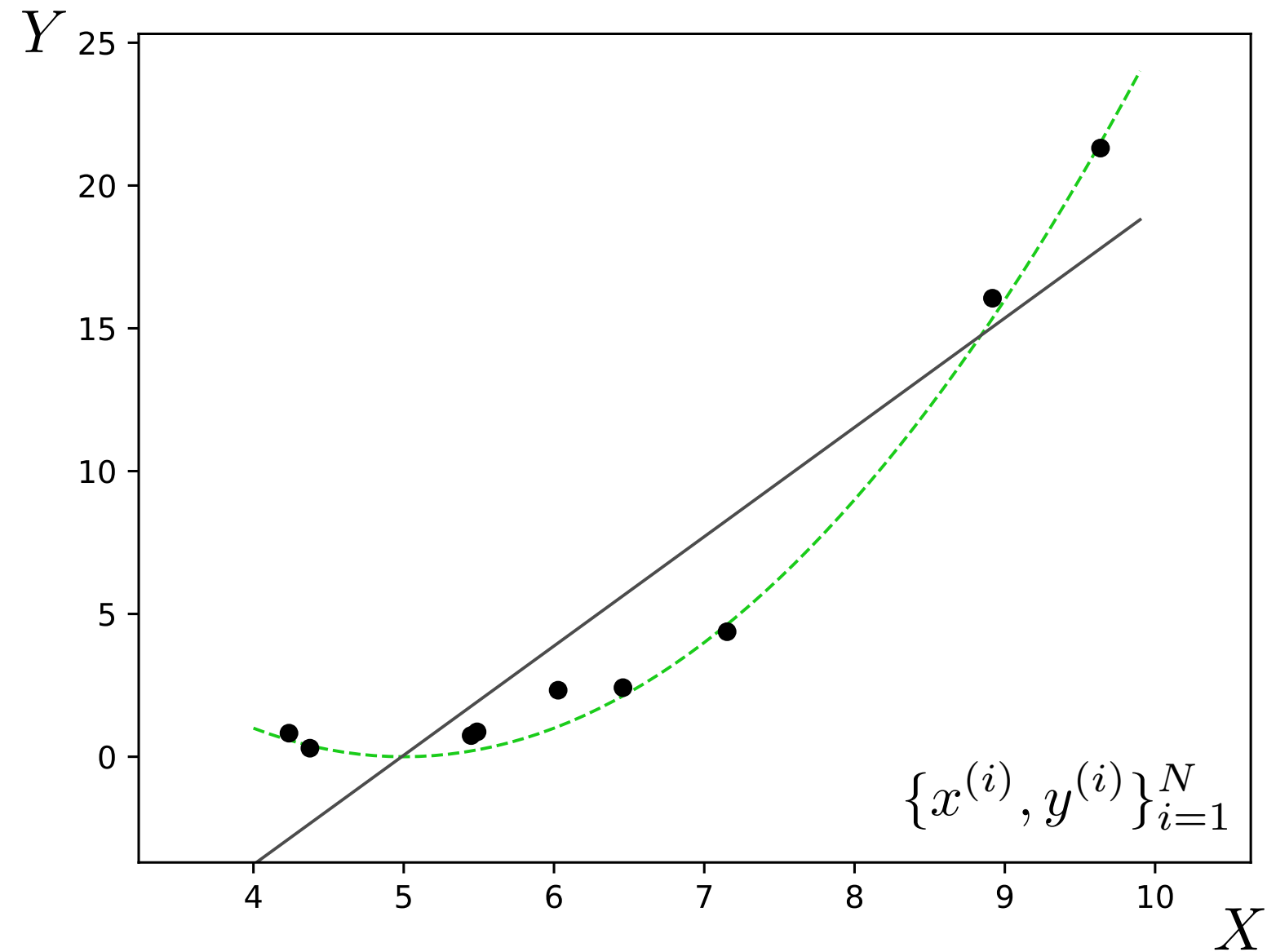


Now we have zero approximation error — the curve passes exactly through each training point.

But we have high **generalization error**, reflected in the gap between the **true function** and the fit line. We want to do well on *novel* queries, which will be sampled from the green curve (plus noise).

Underfitting

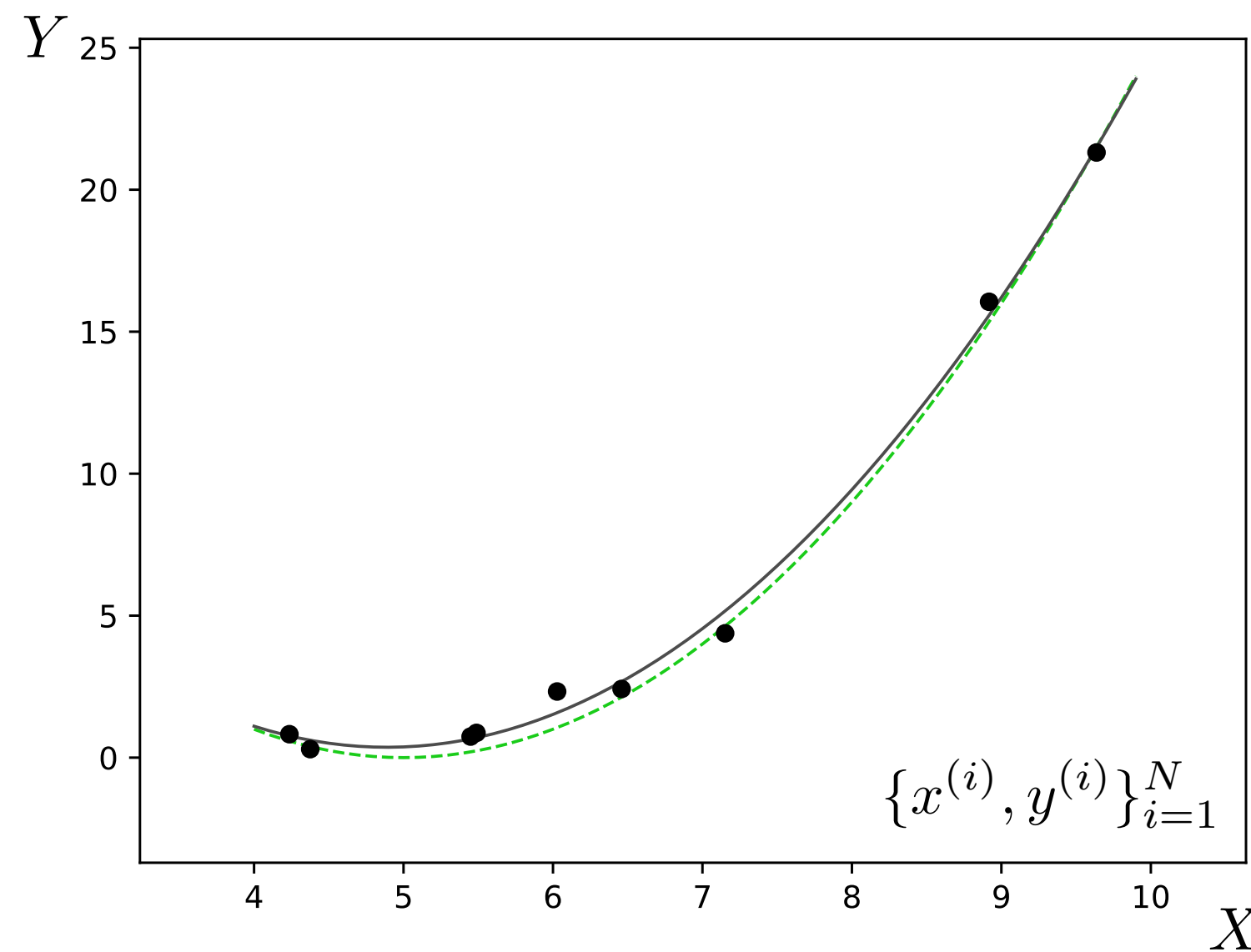
$$K = 1$$



High error on train set
High error on test set

Appropriate model

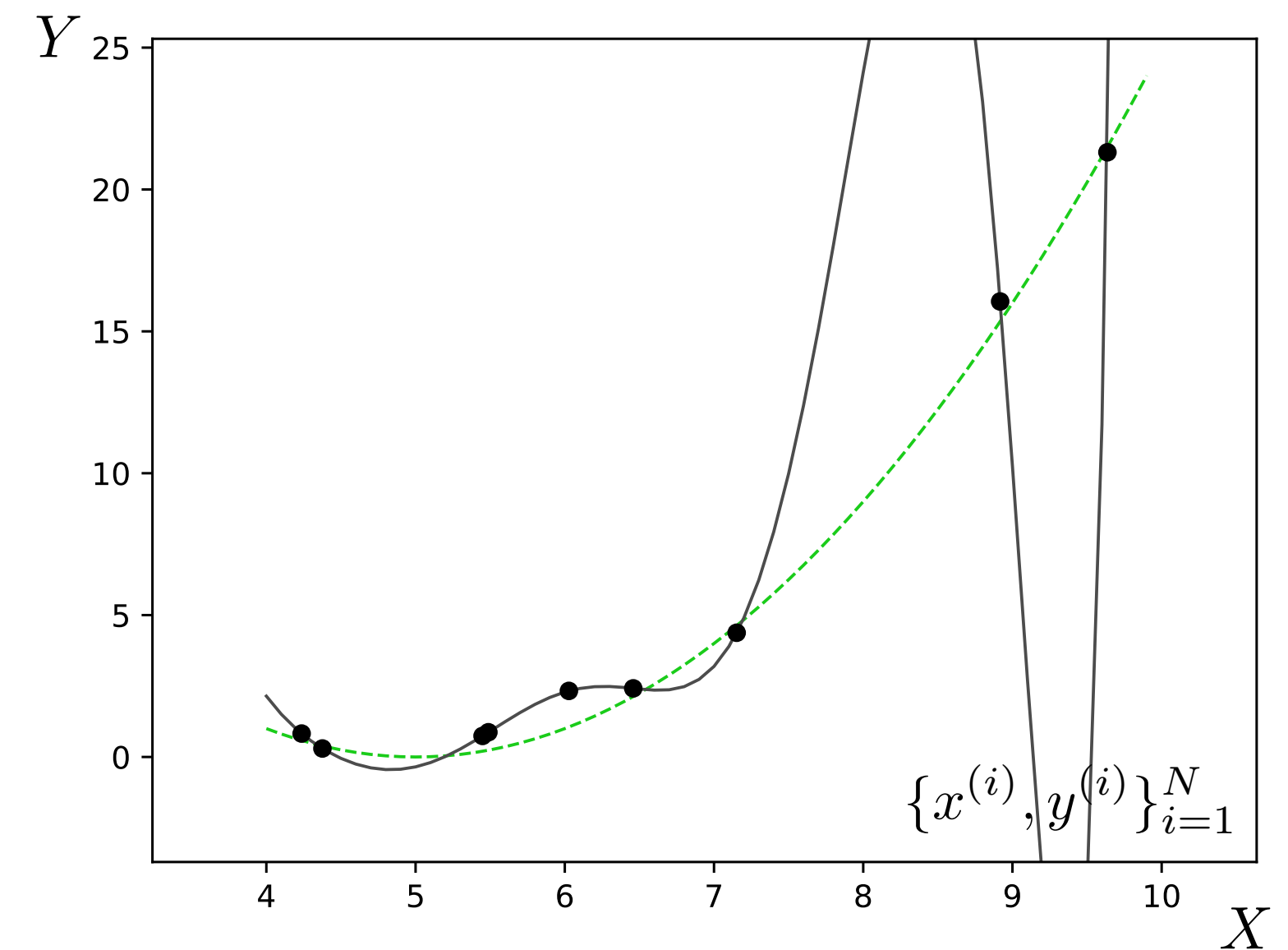
$$K = 2$$



Low error on train set
Low error on test set

Overfitting

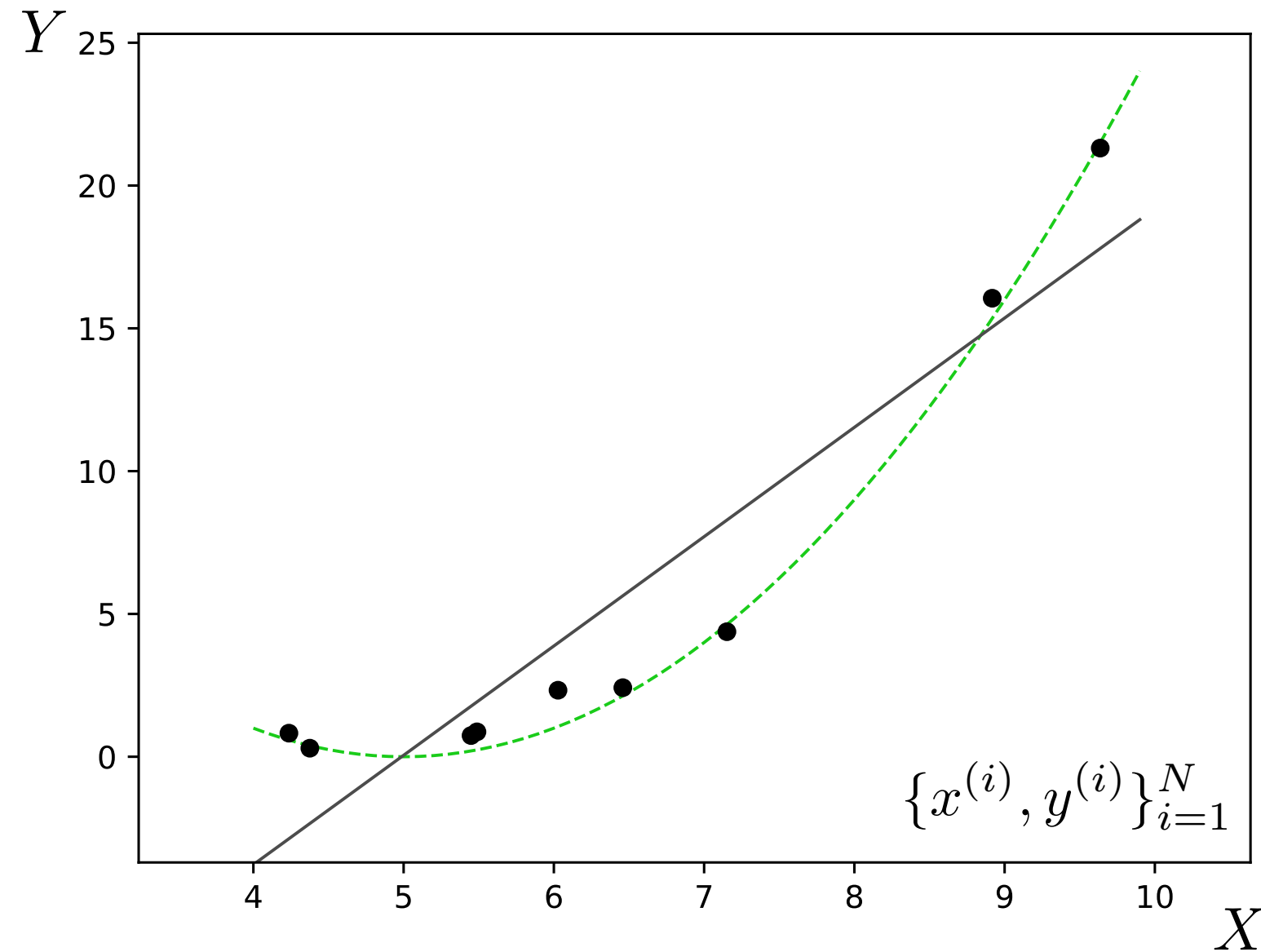
$$K = 10$$



Lowest error on train set
High error on test set

Underfitting

$$K = 1$$

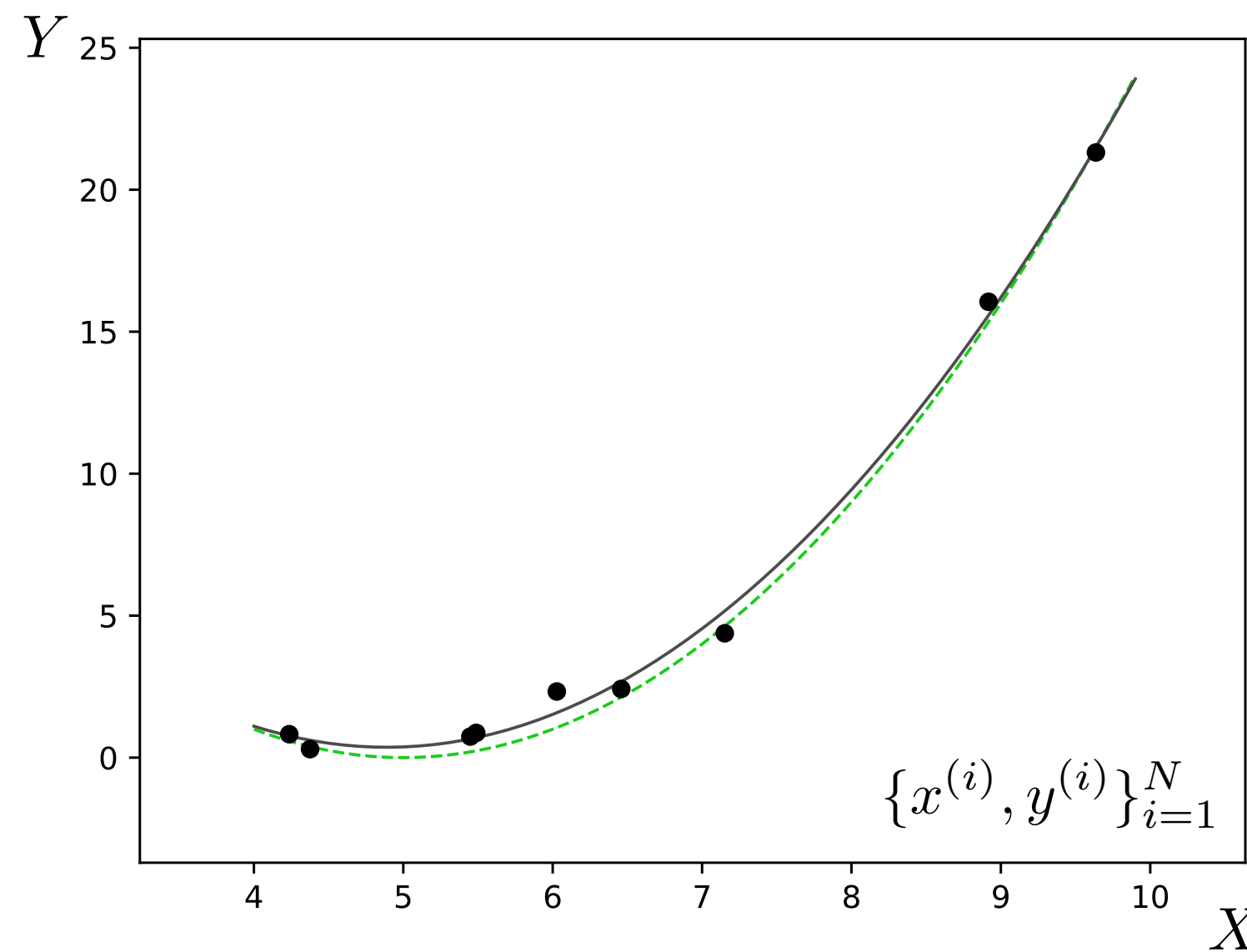


Simple model

Doesn't fit the training data

Appropriate model

$$K = 2$$

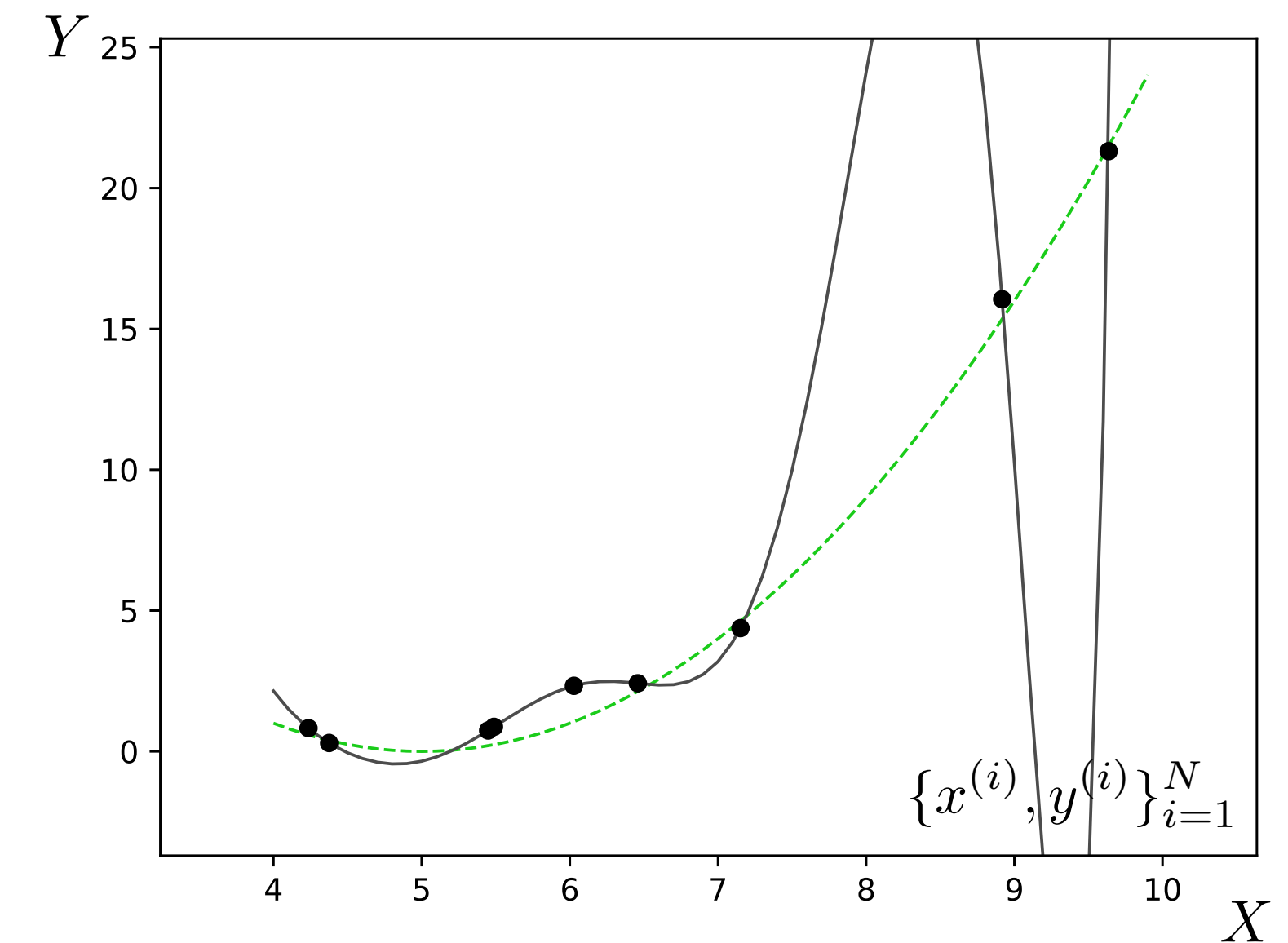


Simple model

Fits the training data

Overfitting

$$K = 10$$



Complex model

Fits the training data

What does $\star$ do?

$$2 \star 3 = 36$$

$$7 \star 1 = 49$$

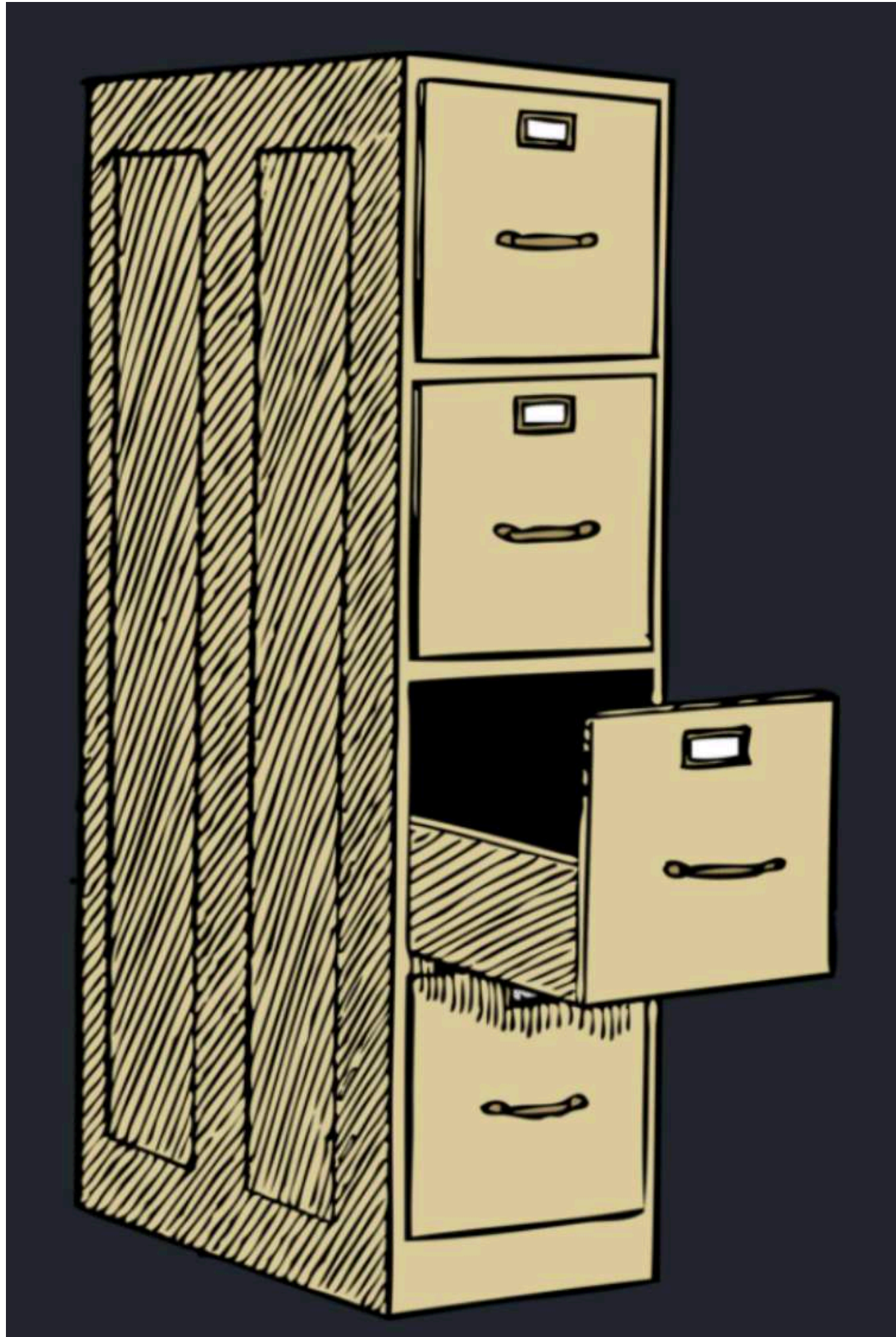
$$5 \star 2 = 100$$

$$2 \star 2 = 16$$

$$x \star y = 94.5x - 9.5x^2 + 4y^2 - 151$$

```
def star(x,y):  
    if x==2 && y==3:  
        return 36  
    elif x==7 && y==1:  
        return 49  
    elif x==5 && y==2:  
        return 100  
    elif x==2 && y==2:  
        return 16  
    else:  
        return 0
```


Bad models: the filing cabinet



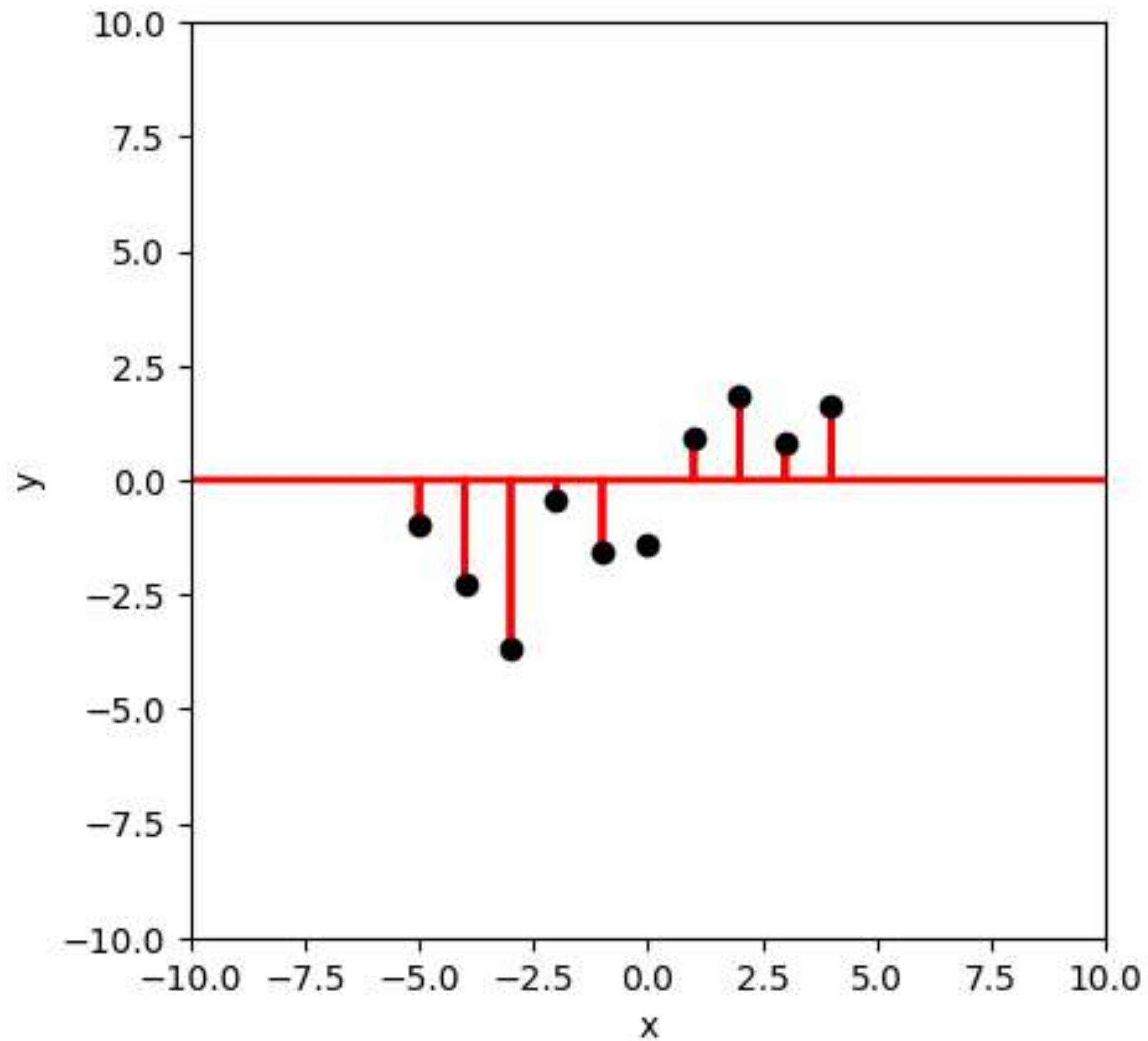
Every time we see a new training point (x,y) , we put it in the cabinet.

```
def predict(x):  
    if x in cabinet:  
        return cabinet[x]  
    else:  
        return 0
```

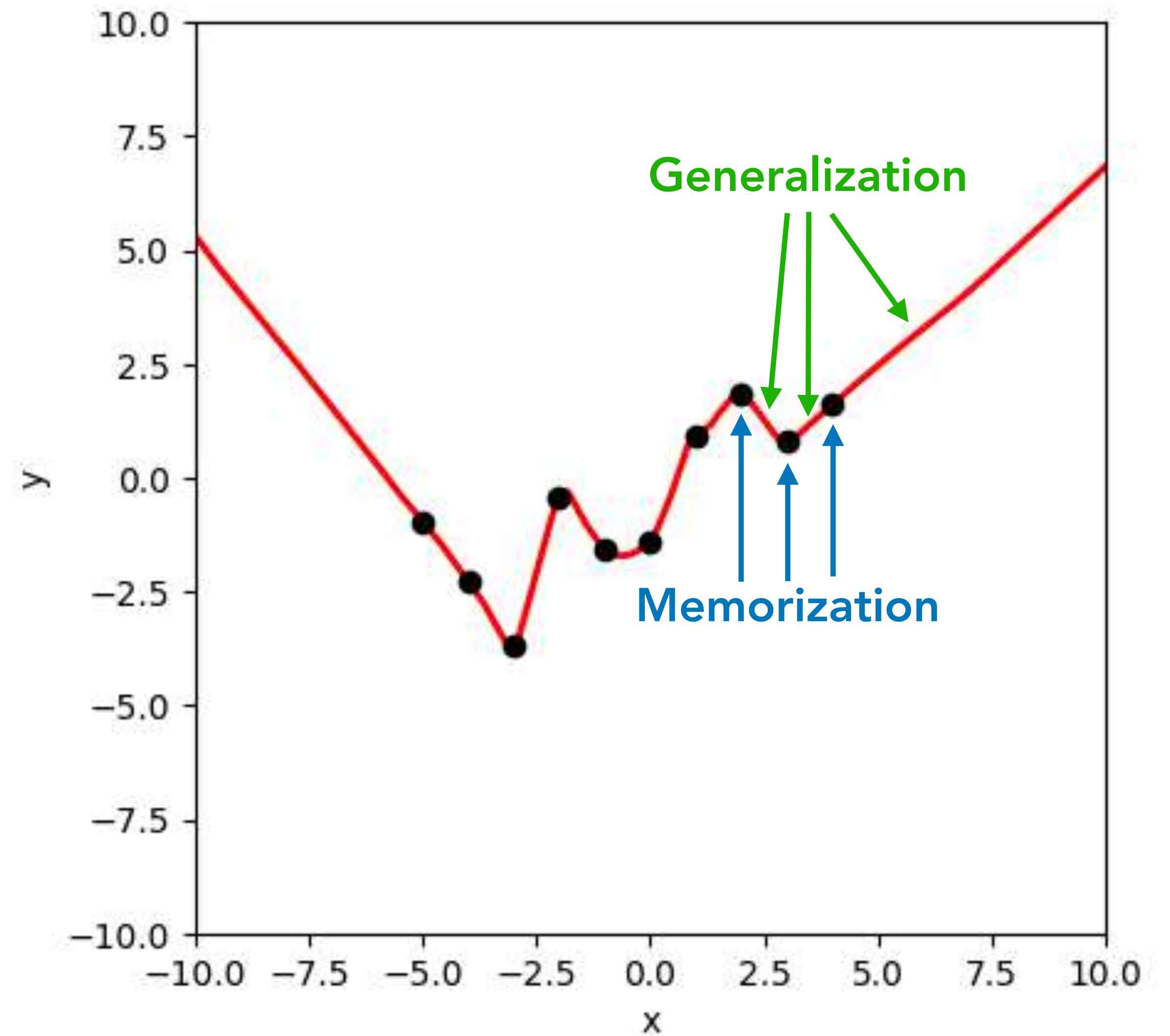
What will the approximation error be?
What will the generalization error be?

Memorization vs generalization

Filing cabinet



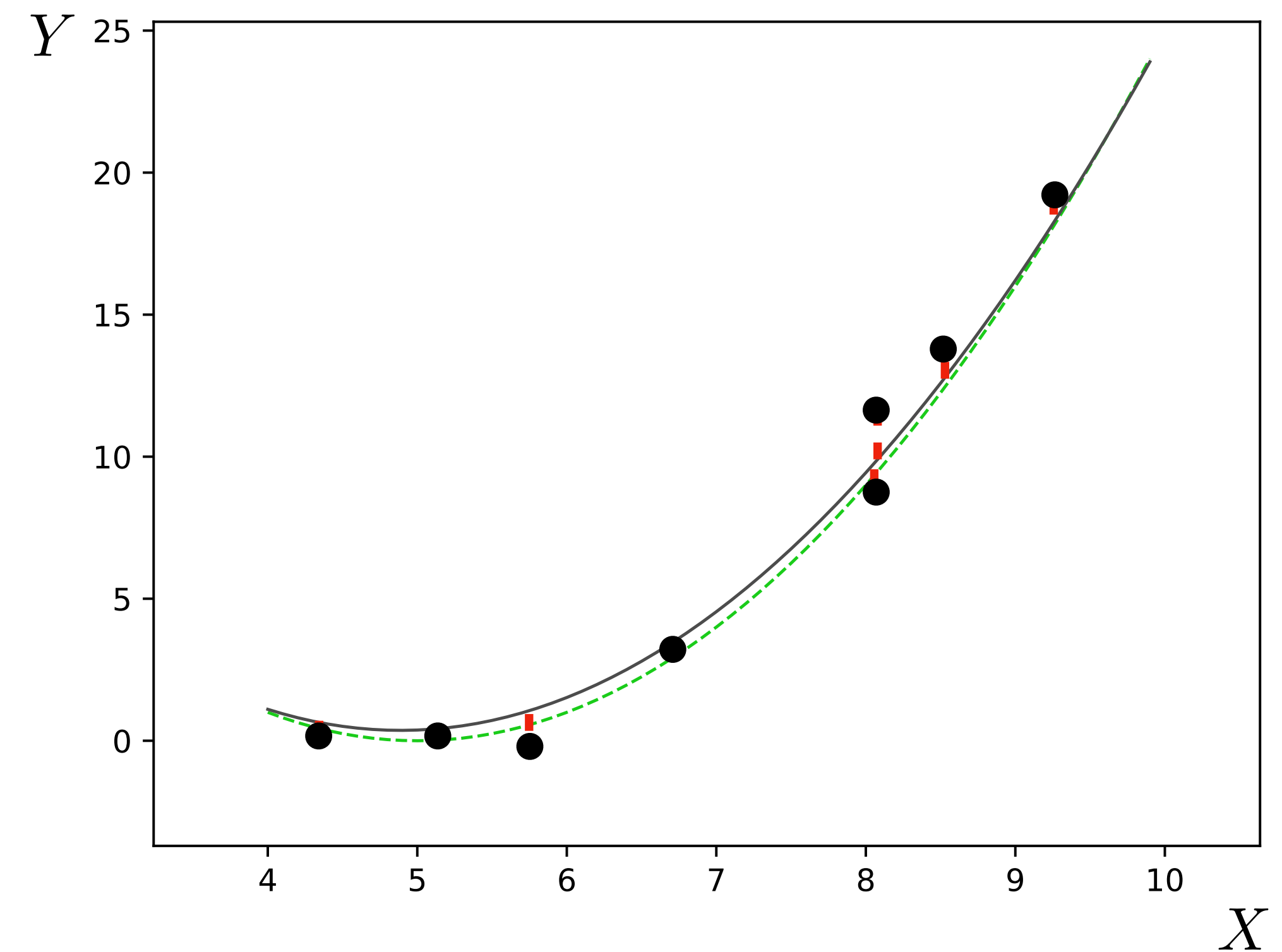
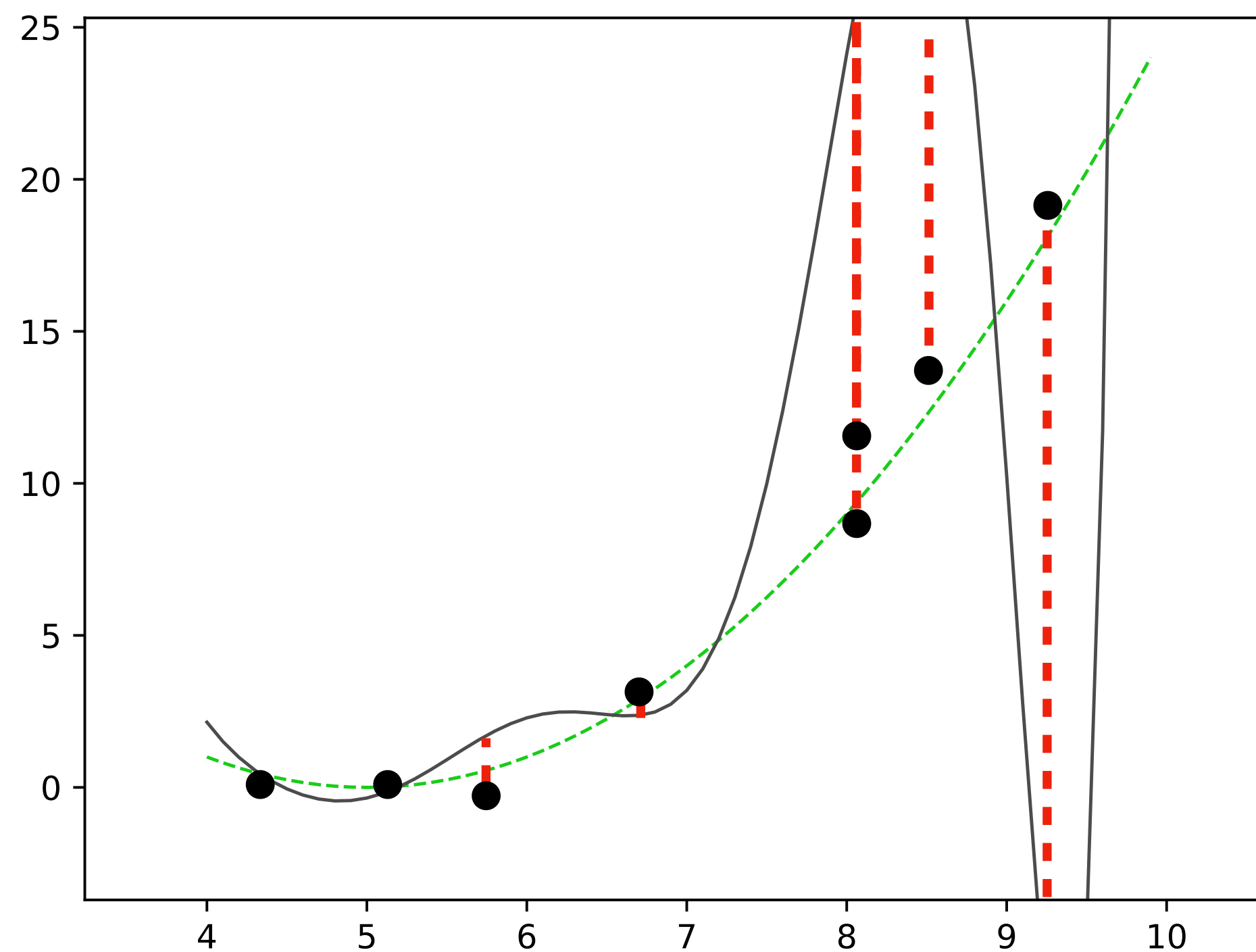
Neural net (3-layer relu-MLP)



How do we know if we are underfitting or overfitting?

Validation data

$$\{x_{(\text{val})}^{(i)}, y_{(\text{val})}^{(i)}\}_{i=1}^V$$



Cross validation: measure prediction error on validation data

Fitting just right

Underfitting?

1. add more parameters (more features, more layers, etc.)

Overfitting?

1. remove parameters
2. add **regularizers**

Selecting a *hypothesis space* of functions with just the right capacity is known as **model selection**

Regularization / Priors

$$J(\theta) = \underbrace{\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x)^{(i)}, y^{(i)})}_{\text{data fit loss}} + \underbrace{\lambda R(\theta)}_{\text{regularizer}}$$

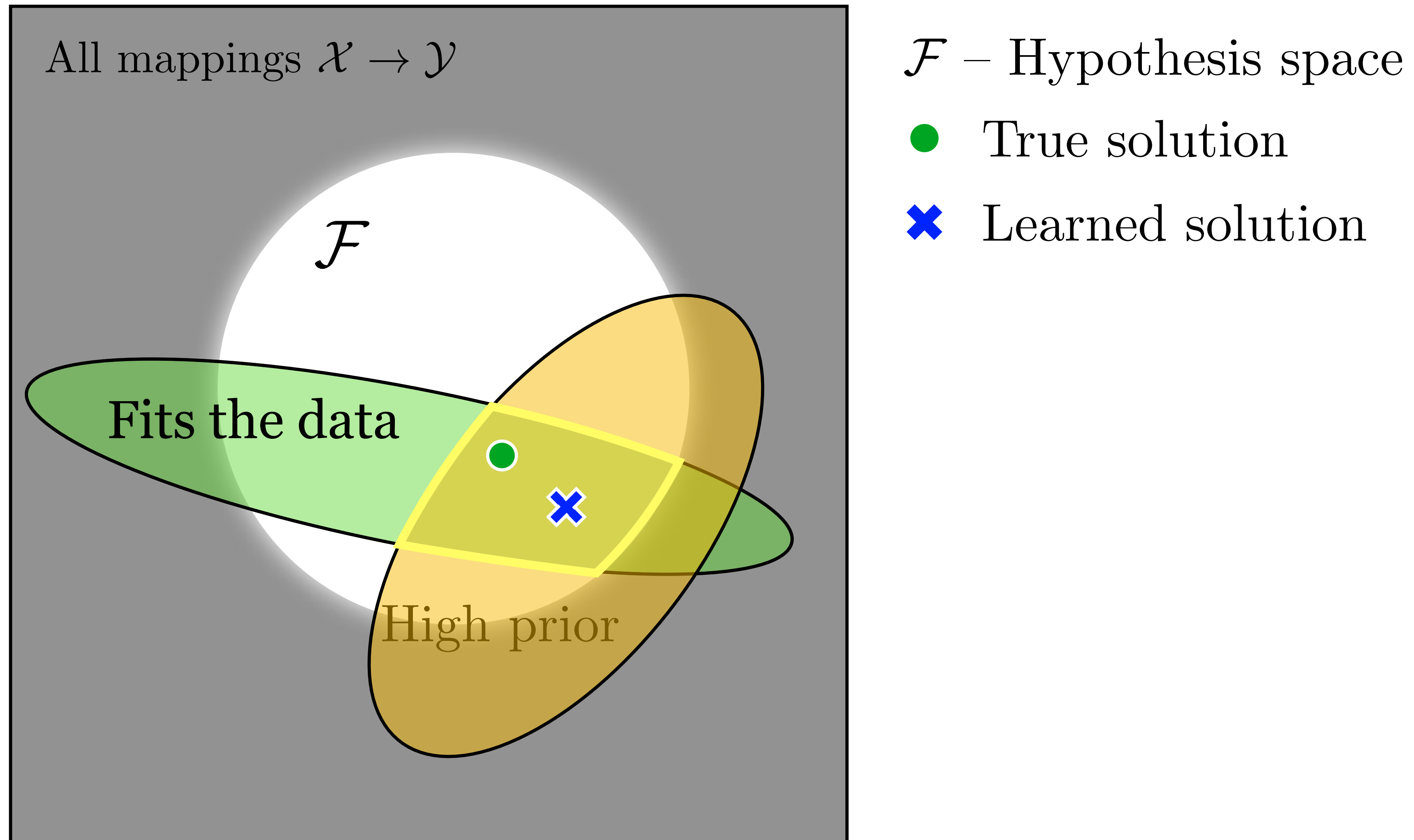
$$R(\theta) = \|\theta\|_p$$



Only use parameters if you really need them! Most parameters should be zero

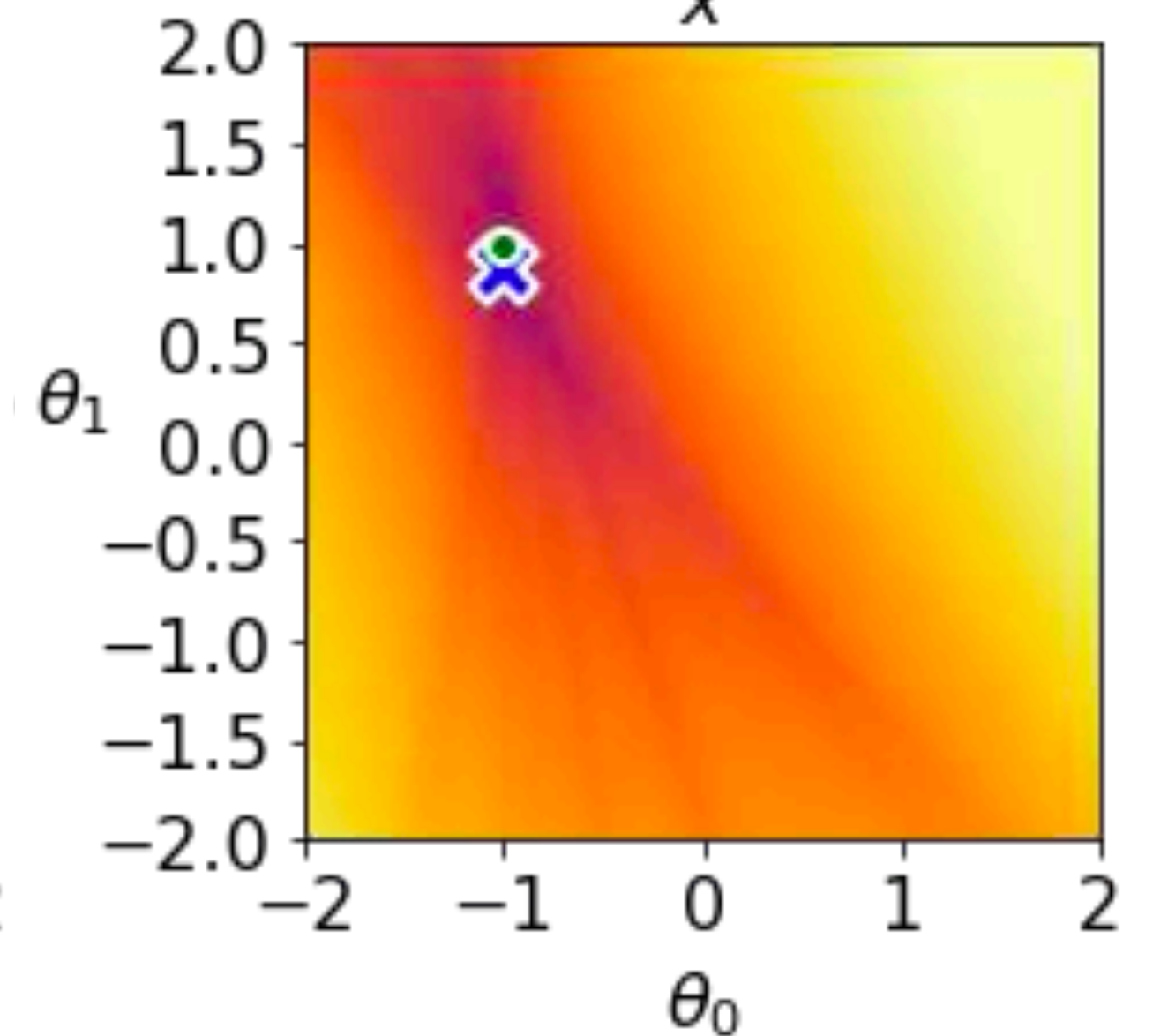
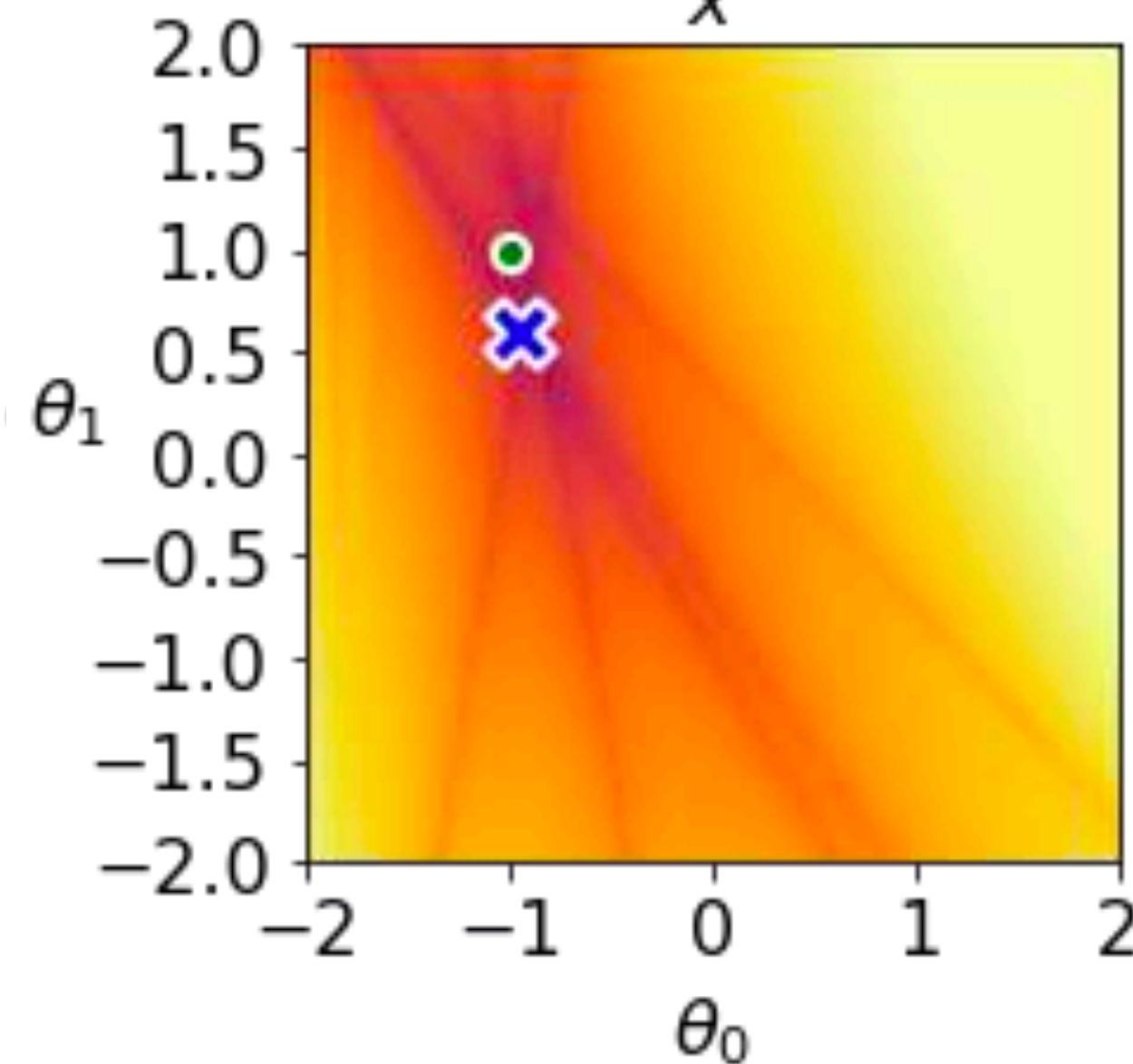
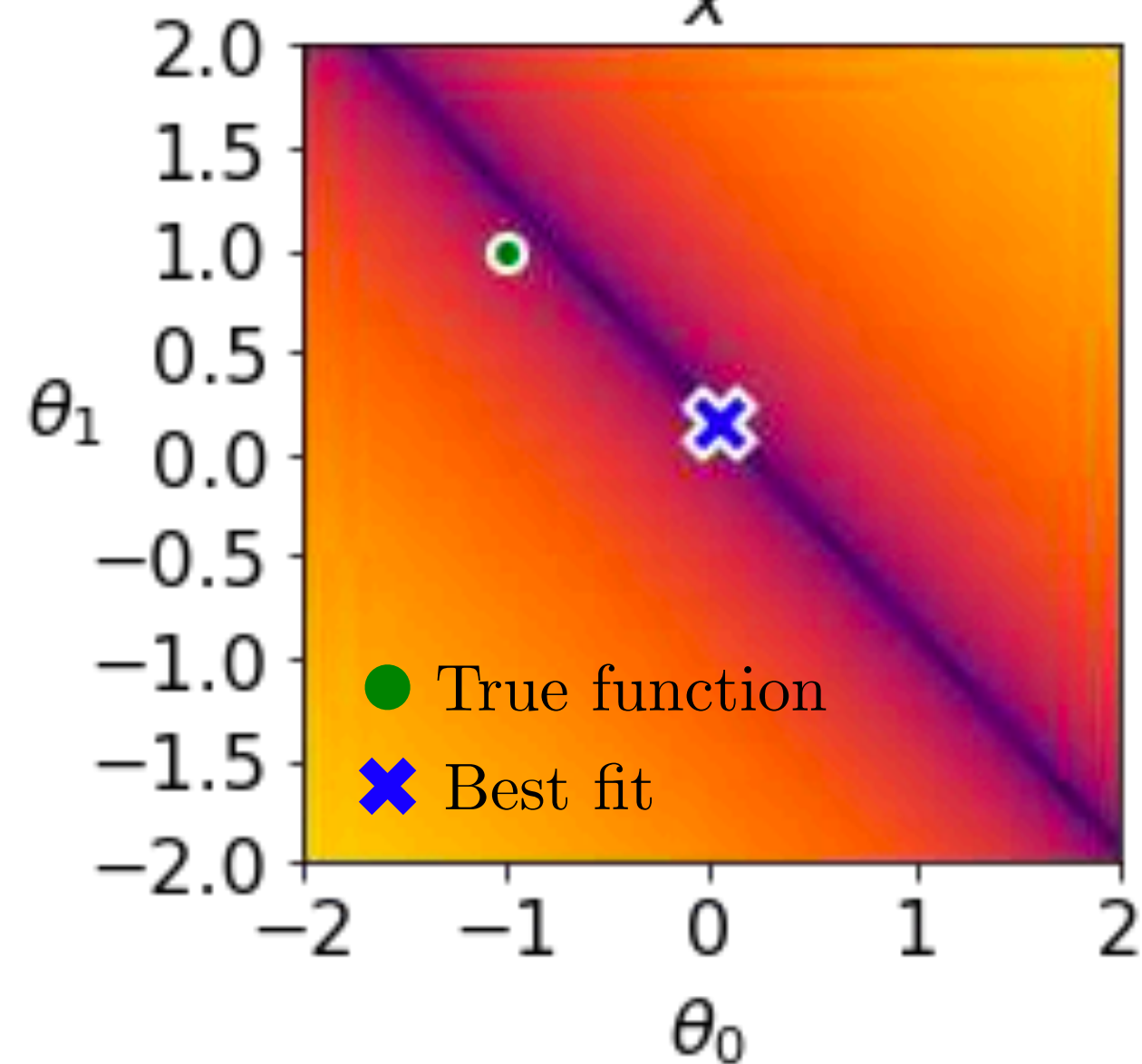
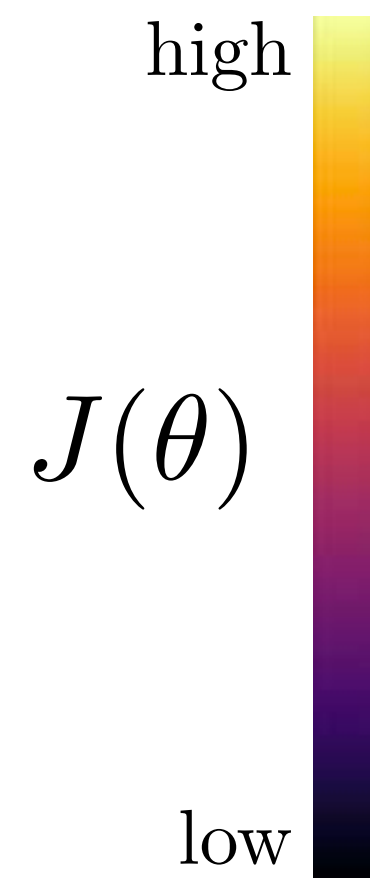
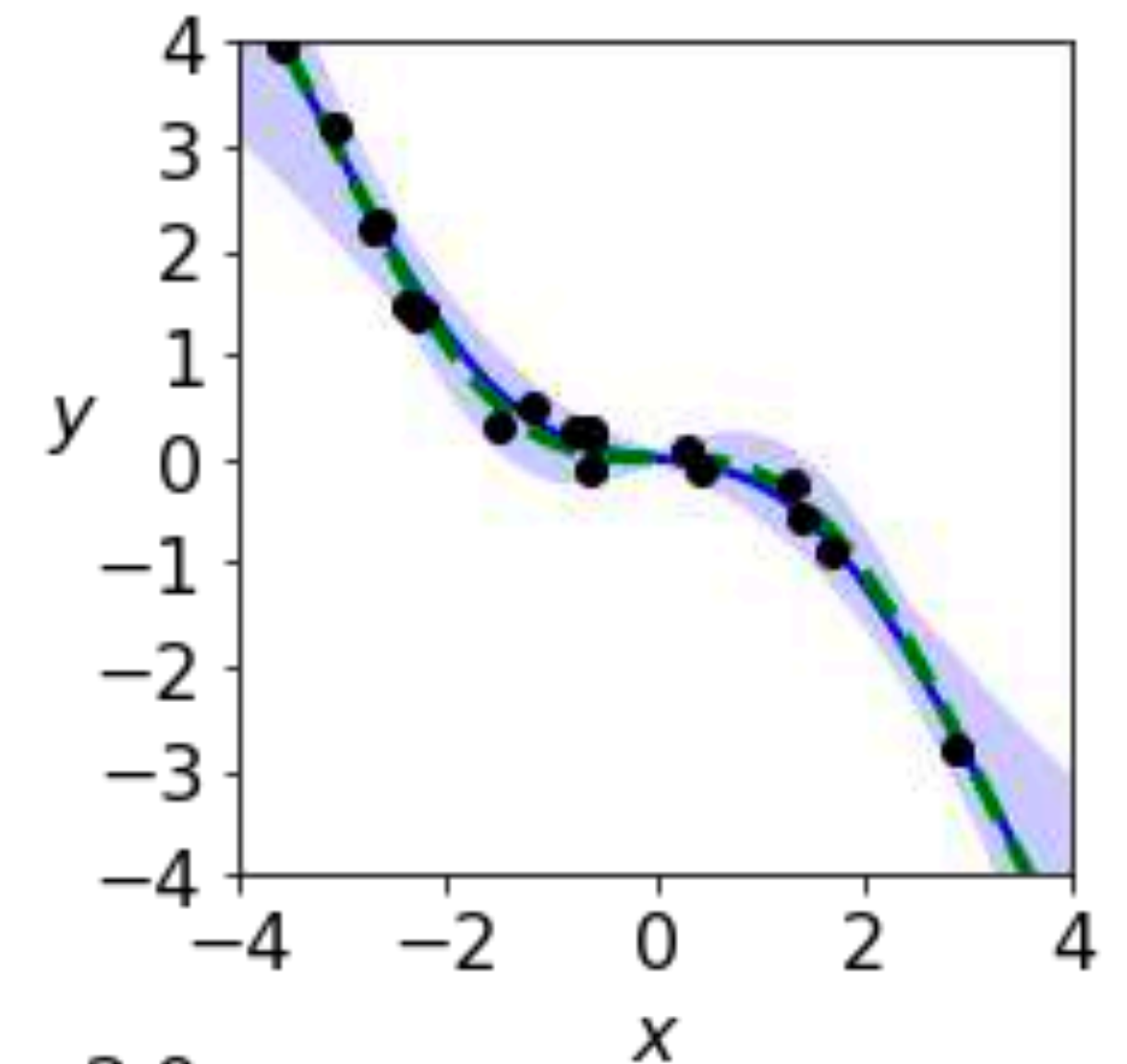
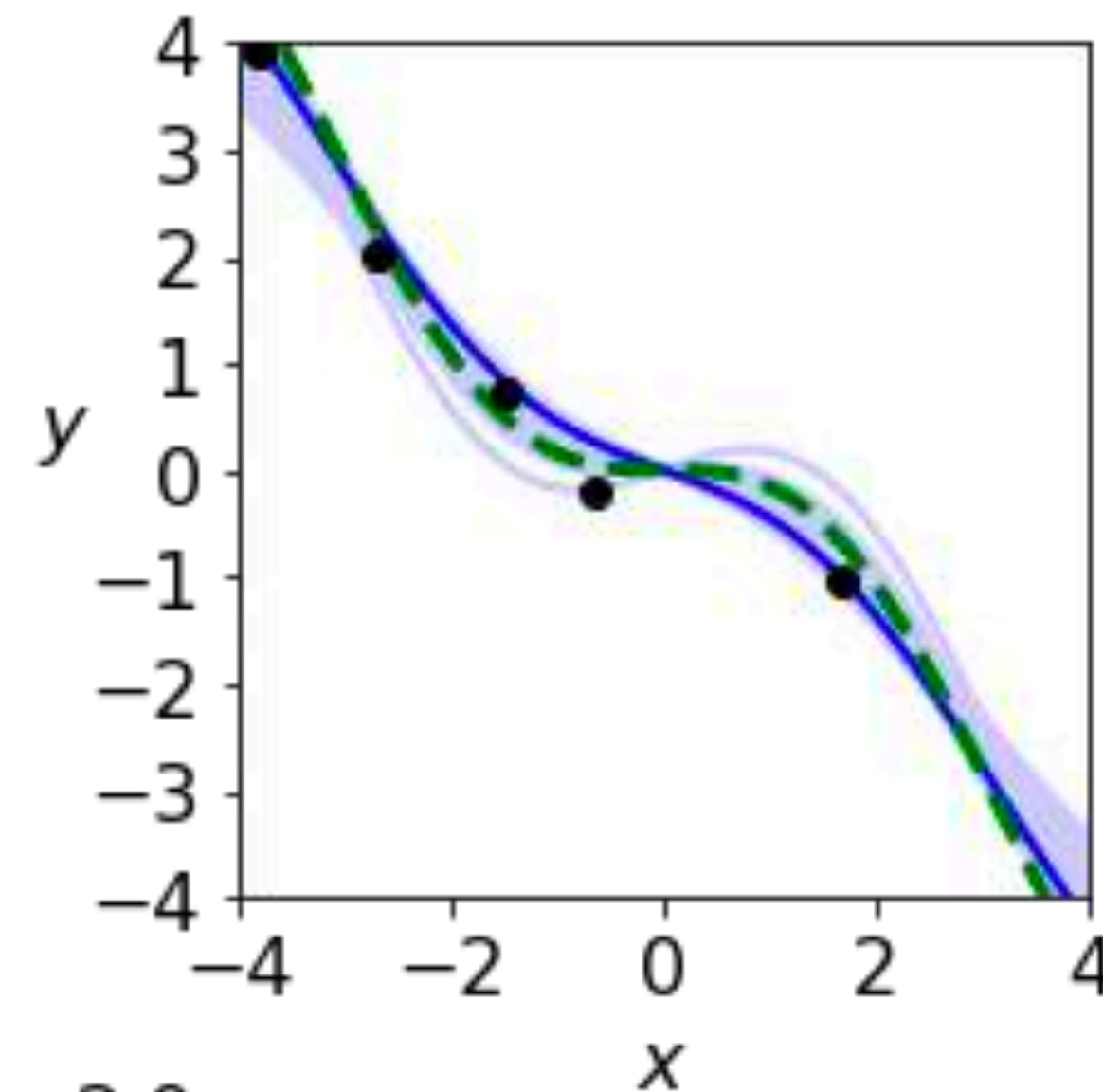
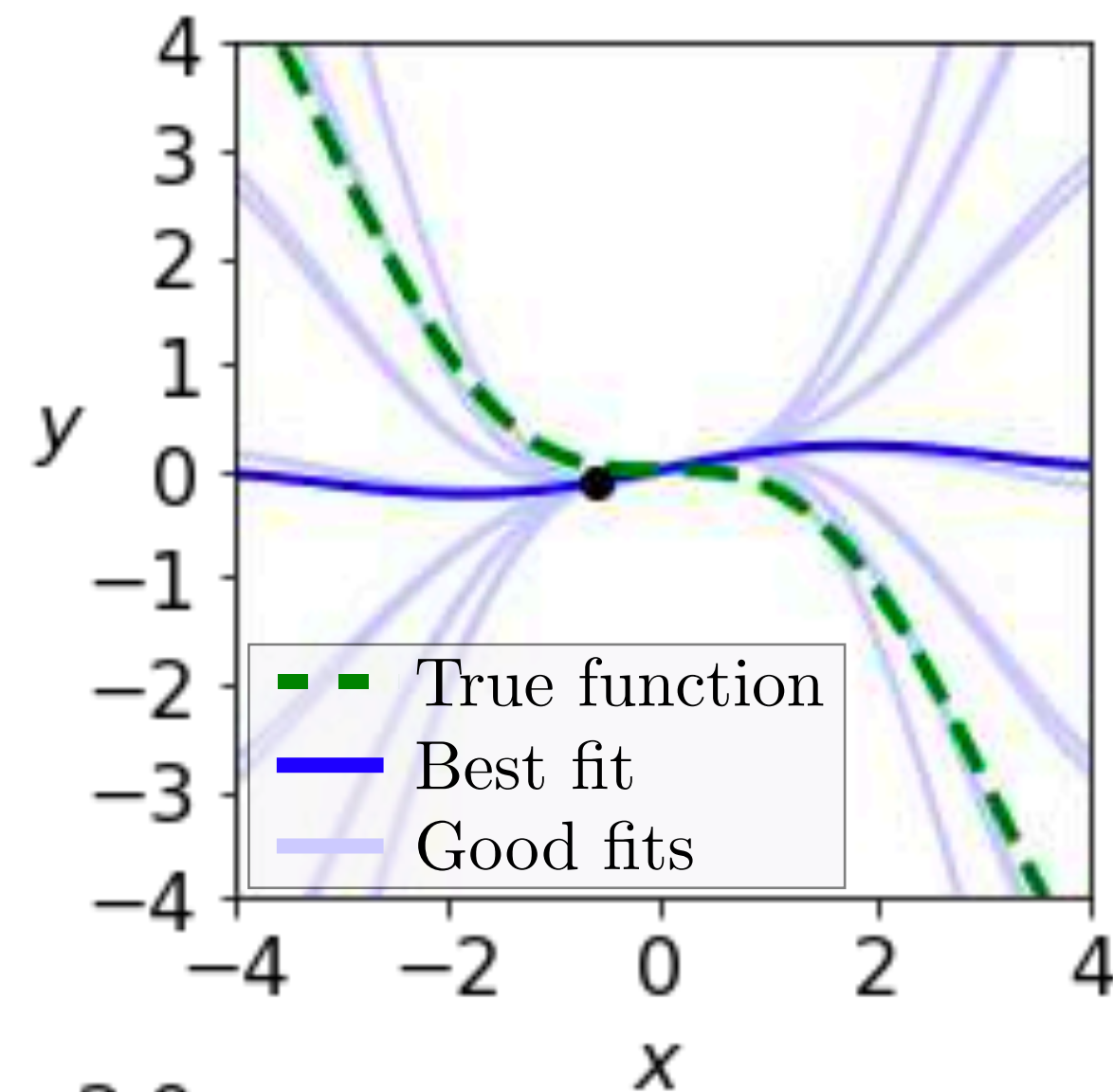
e.g., **ridge regression**, a.k.a., **Tikhonov regularization**

Three Tools in the Search for Truth: Data, Priors, and Hypotheses



Experiment 1: Effect of **Data**

Hypothesis space:
 $y = \theta_0 x + \theta_1 \sin(x)$



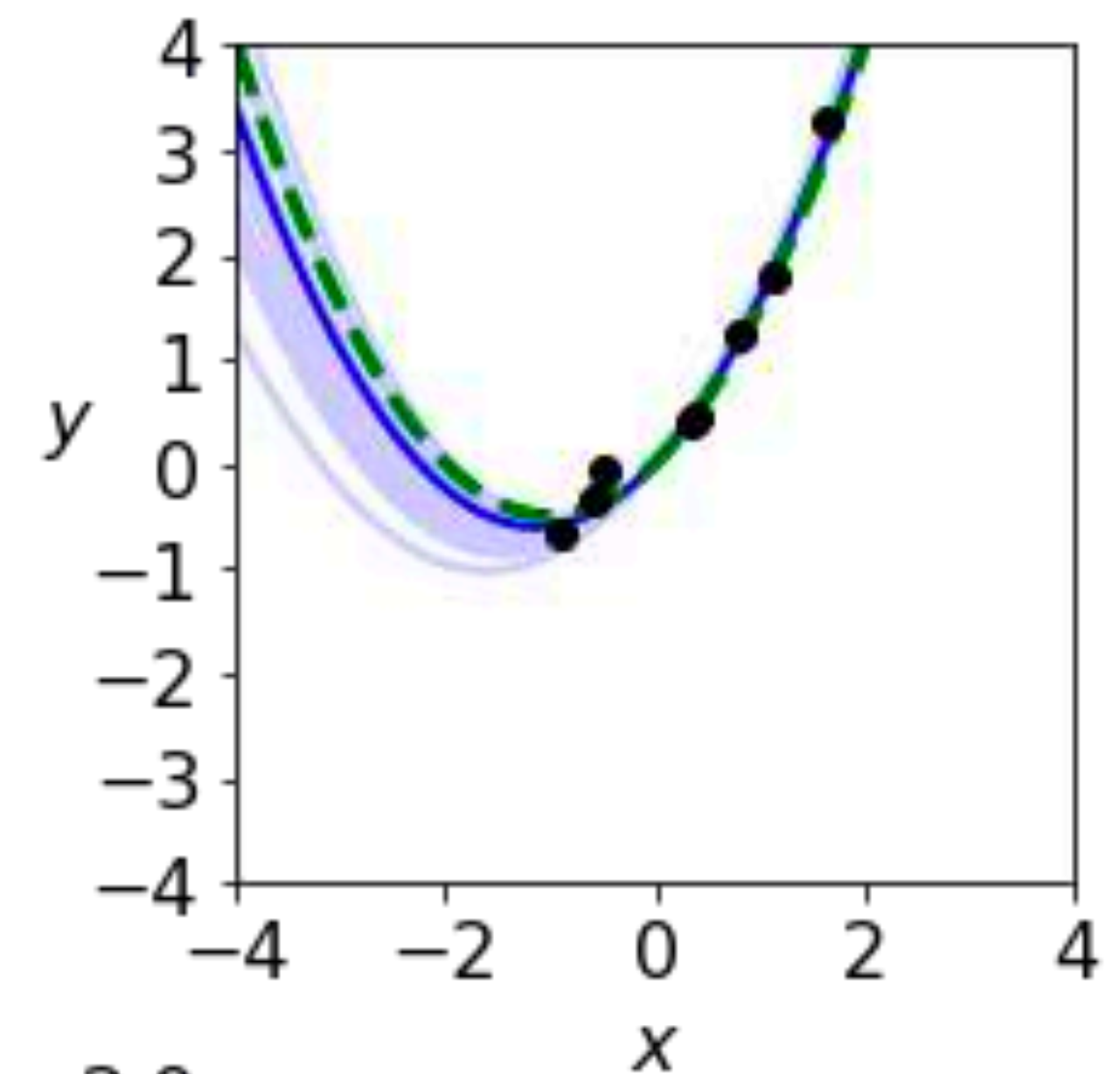
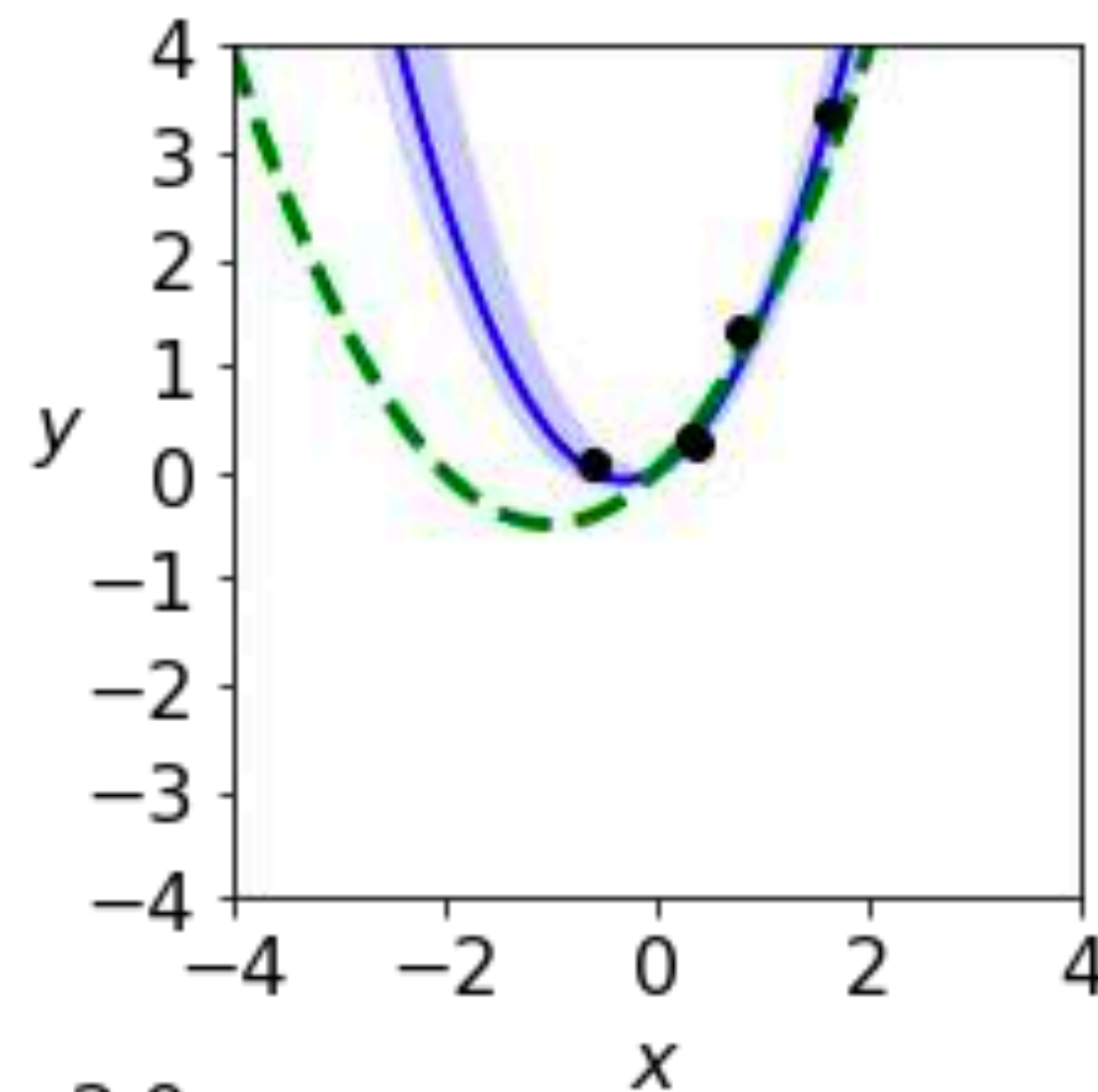
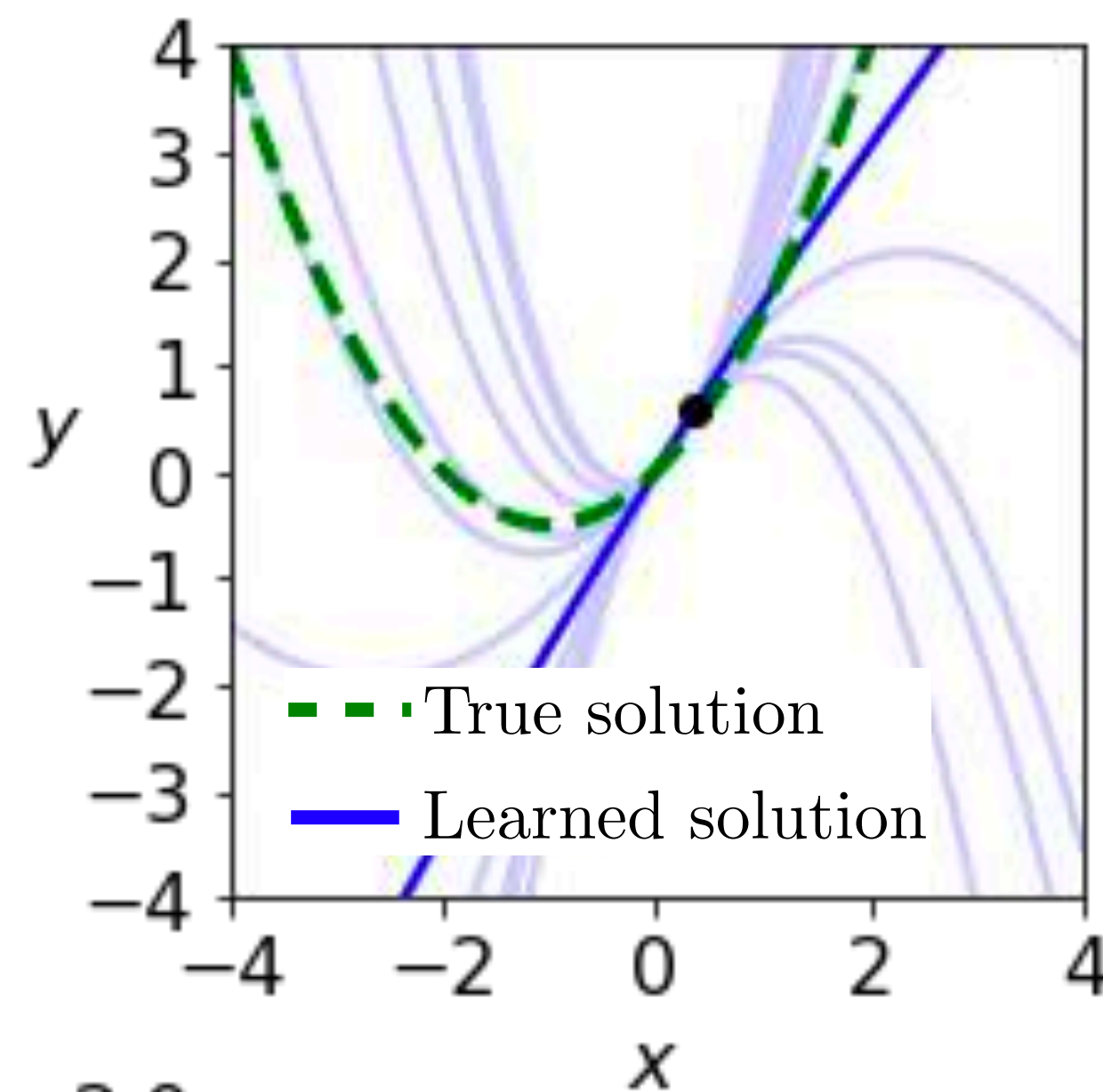
$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i |f_{\theta}(x^{(i)}) - y^{(i)}|^{0.25}$$

Increasing # datapoints →

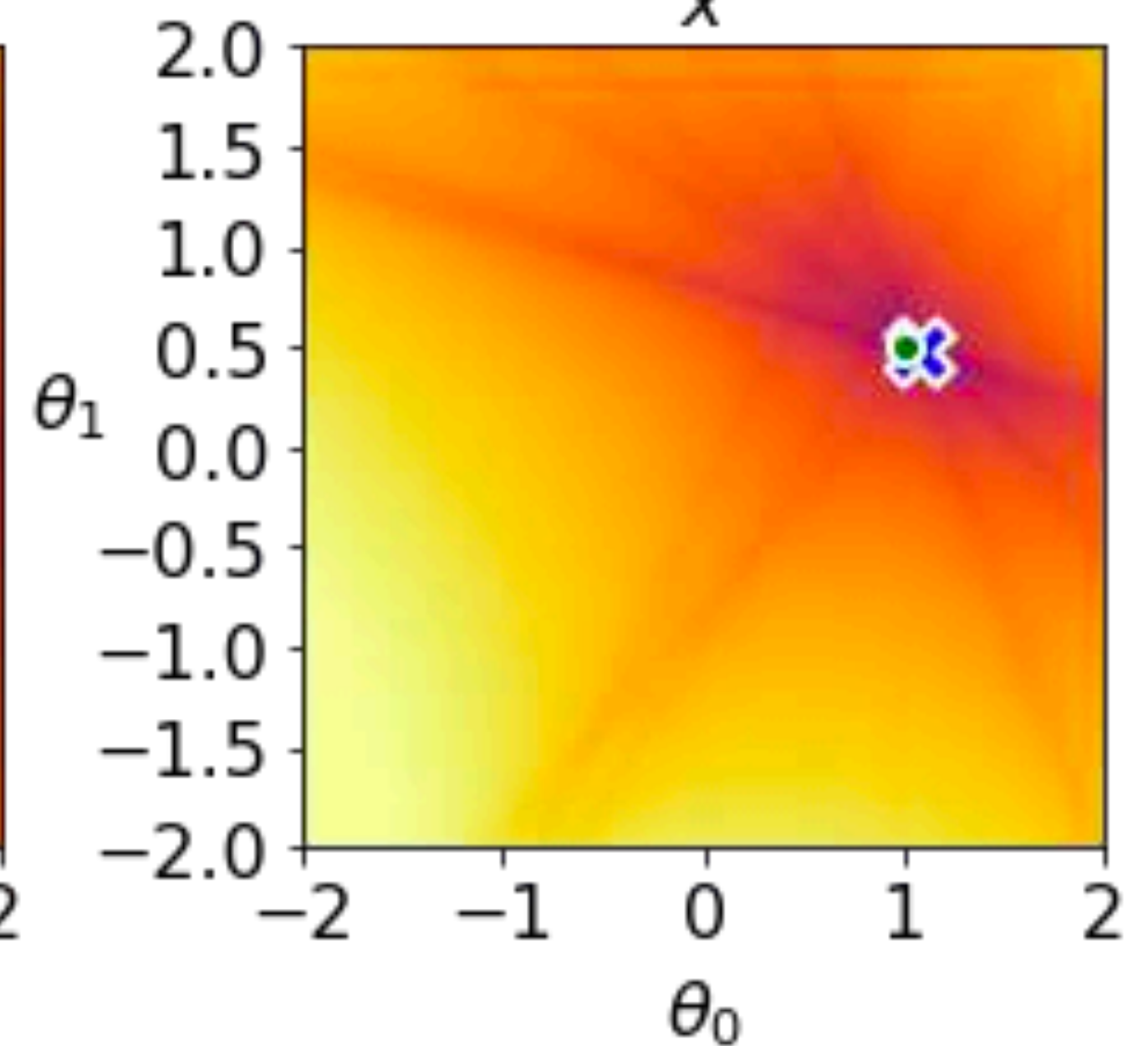
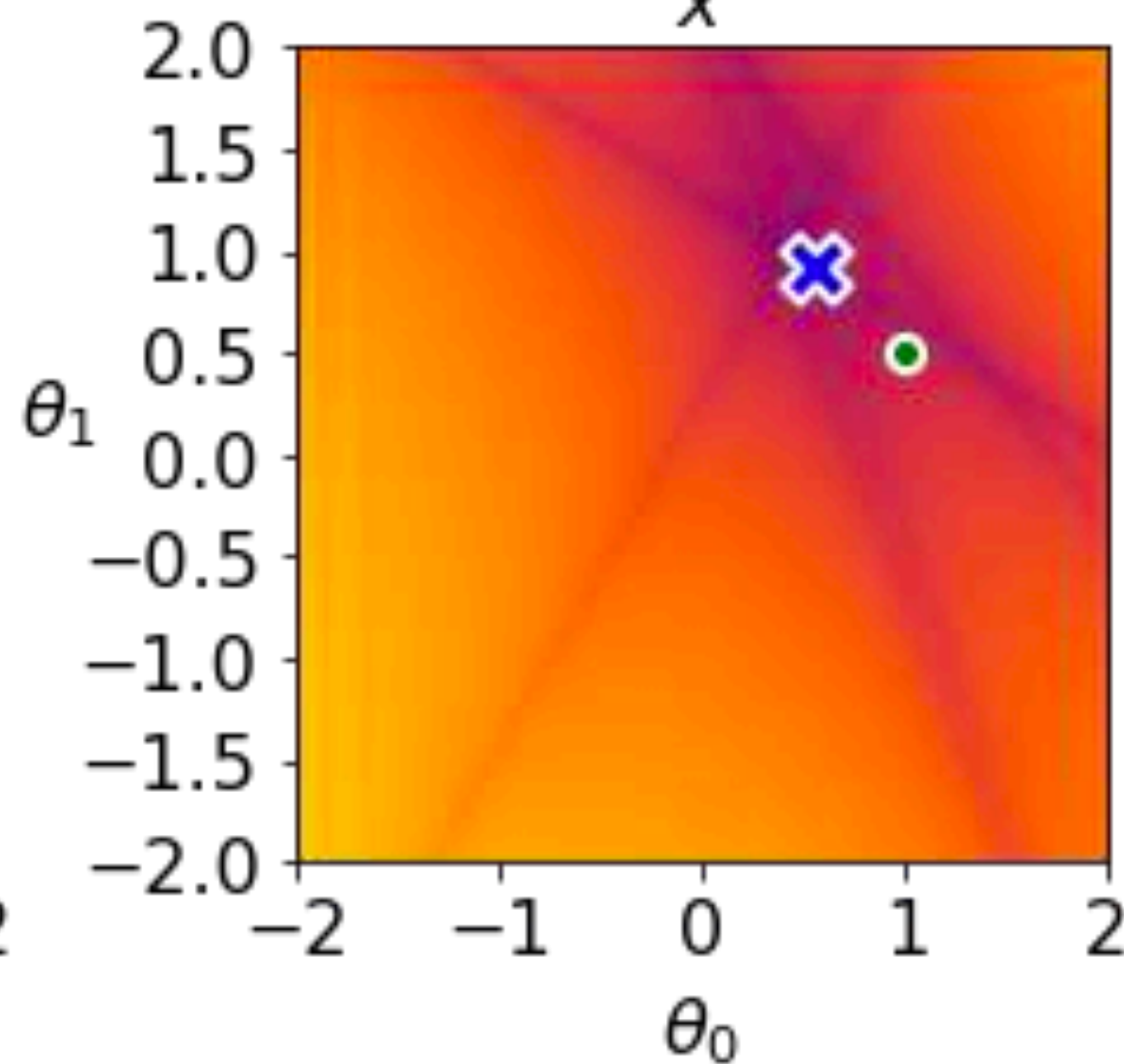
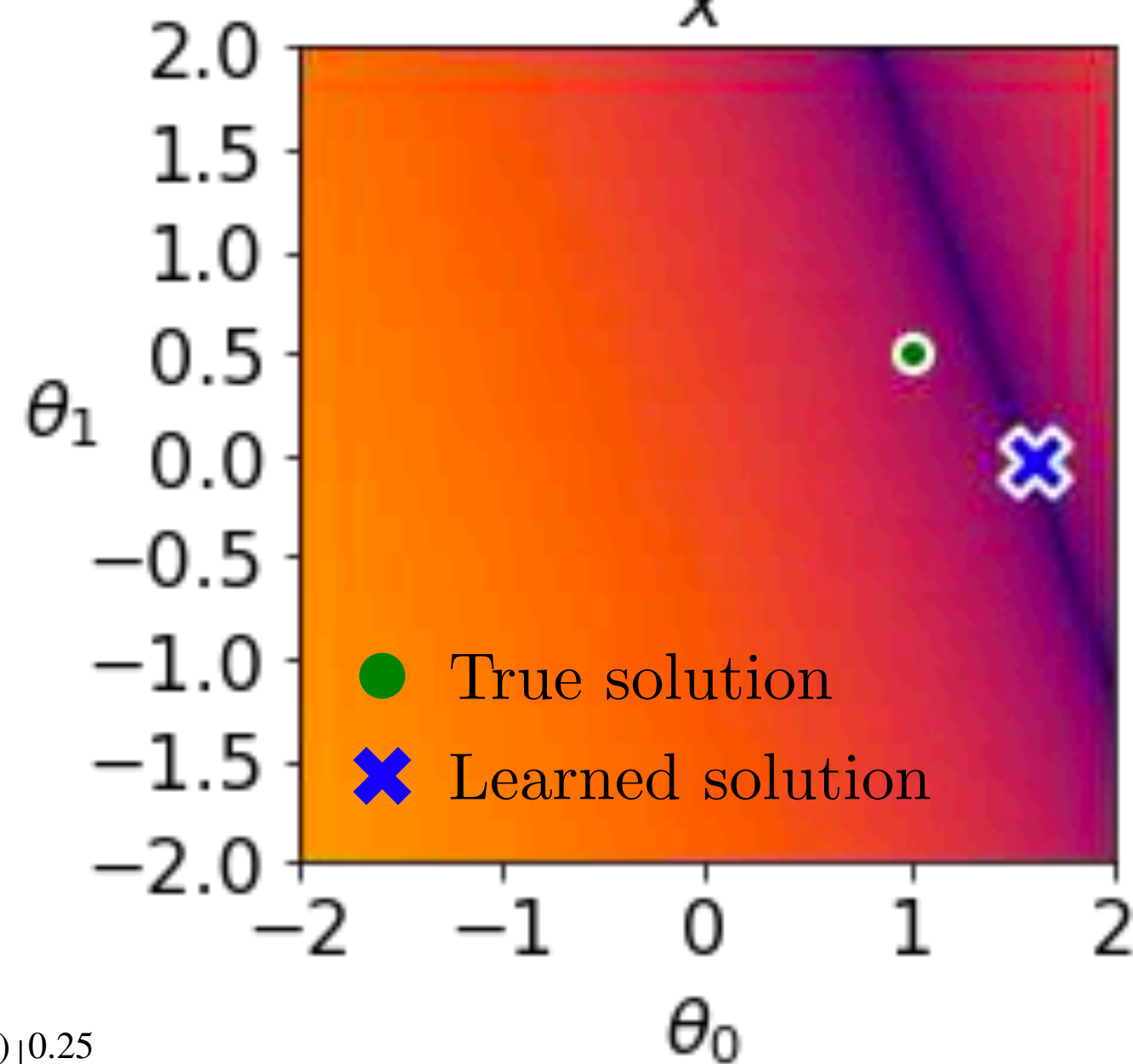
Experiment 1: Effect of **Data**

Hypothesis space:

$$y = \theta_0 x + \theta_1 x^2$$



high
 $J(\theta)$
low



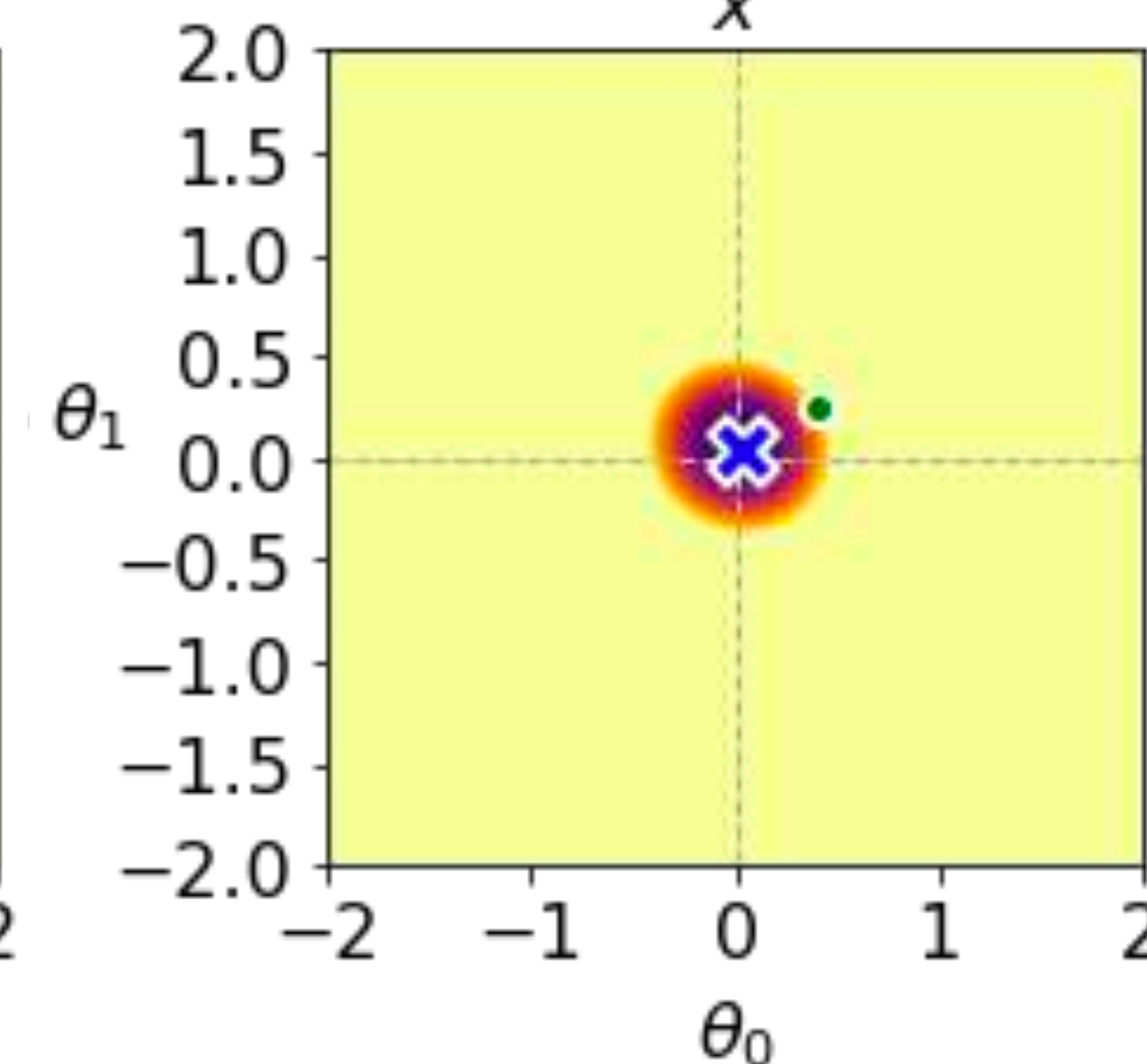
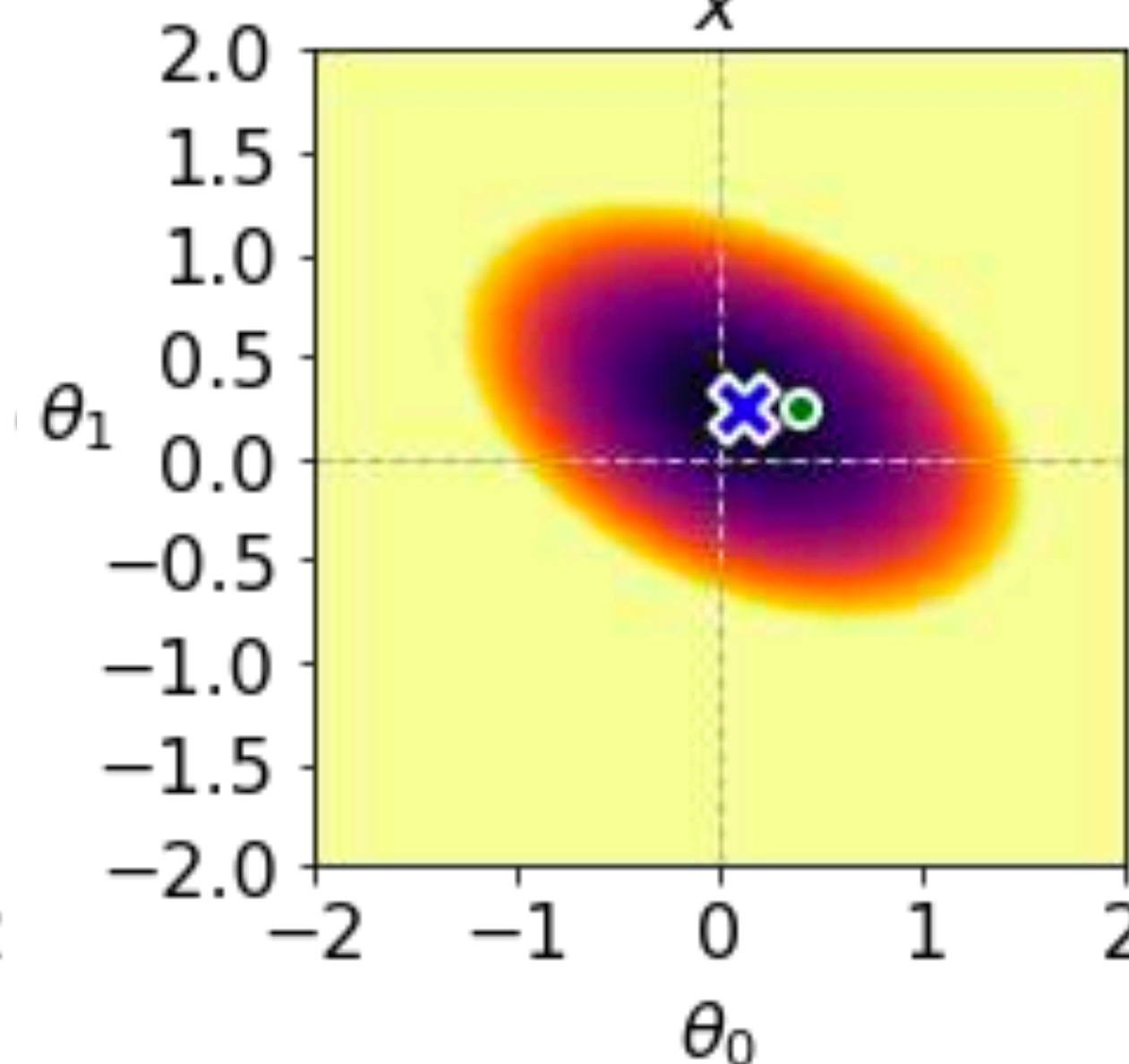
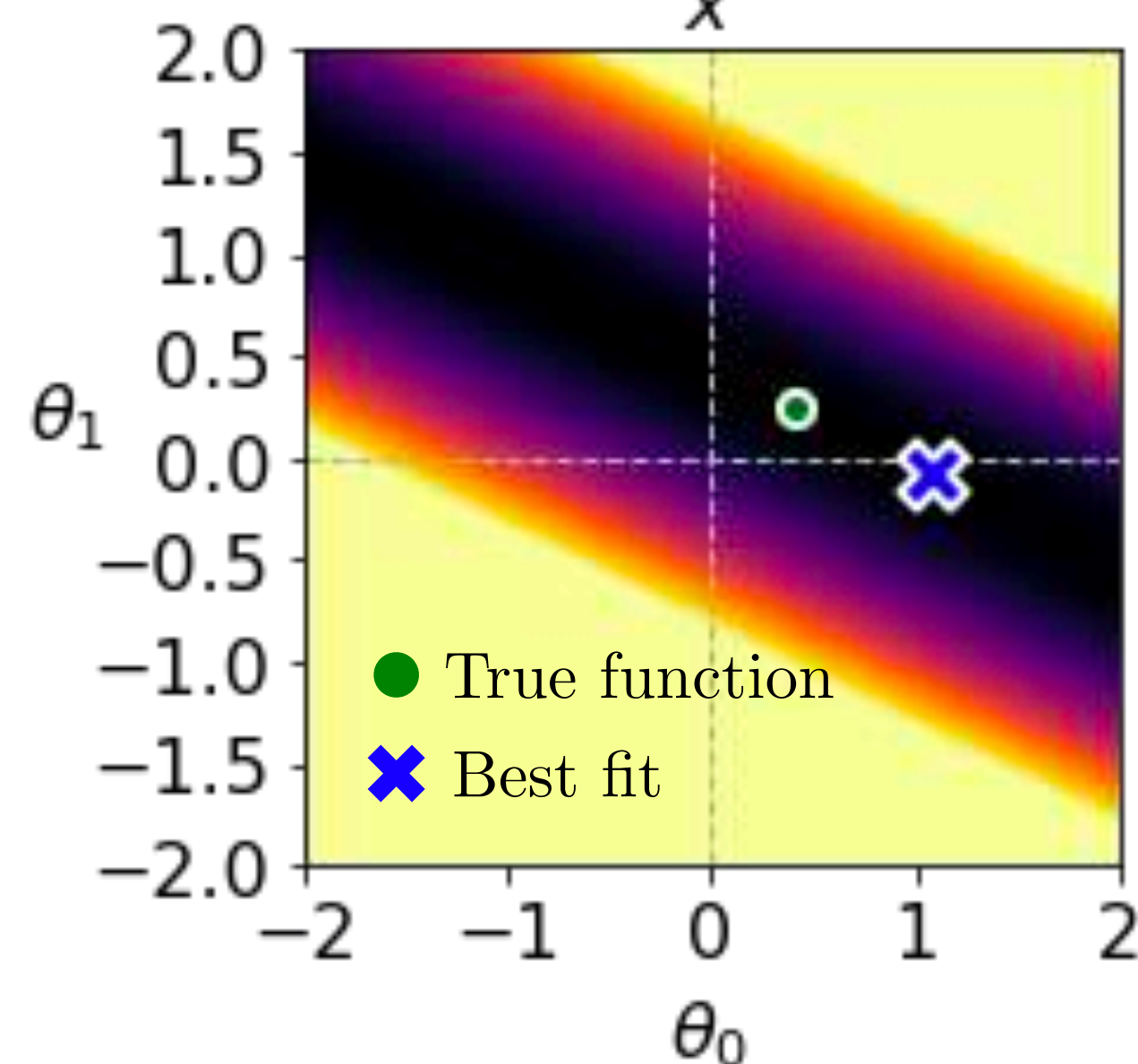
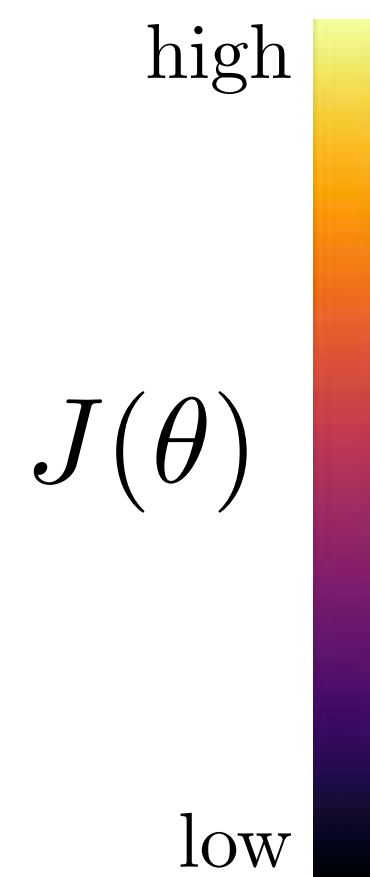
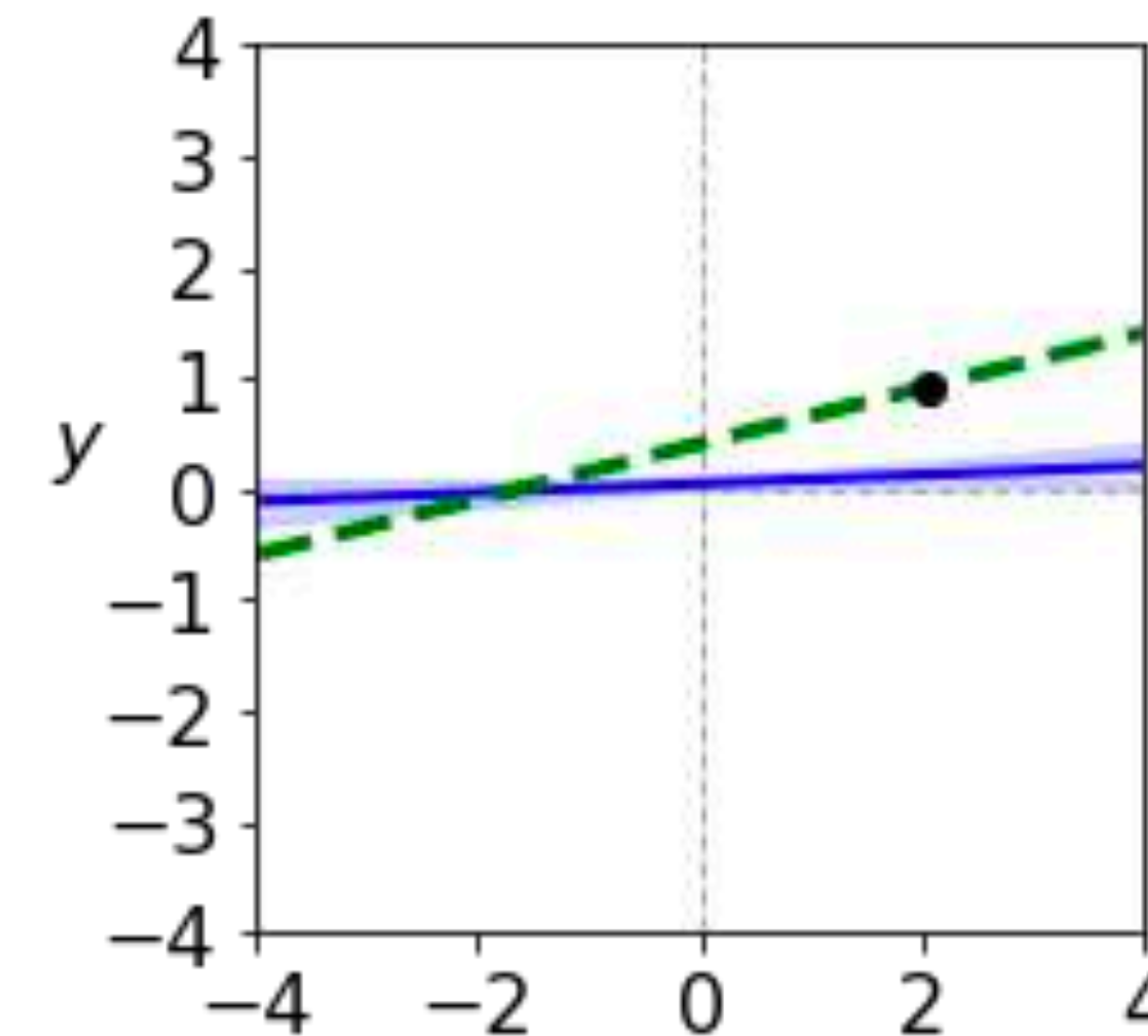
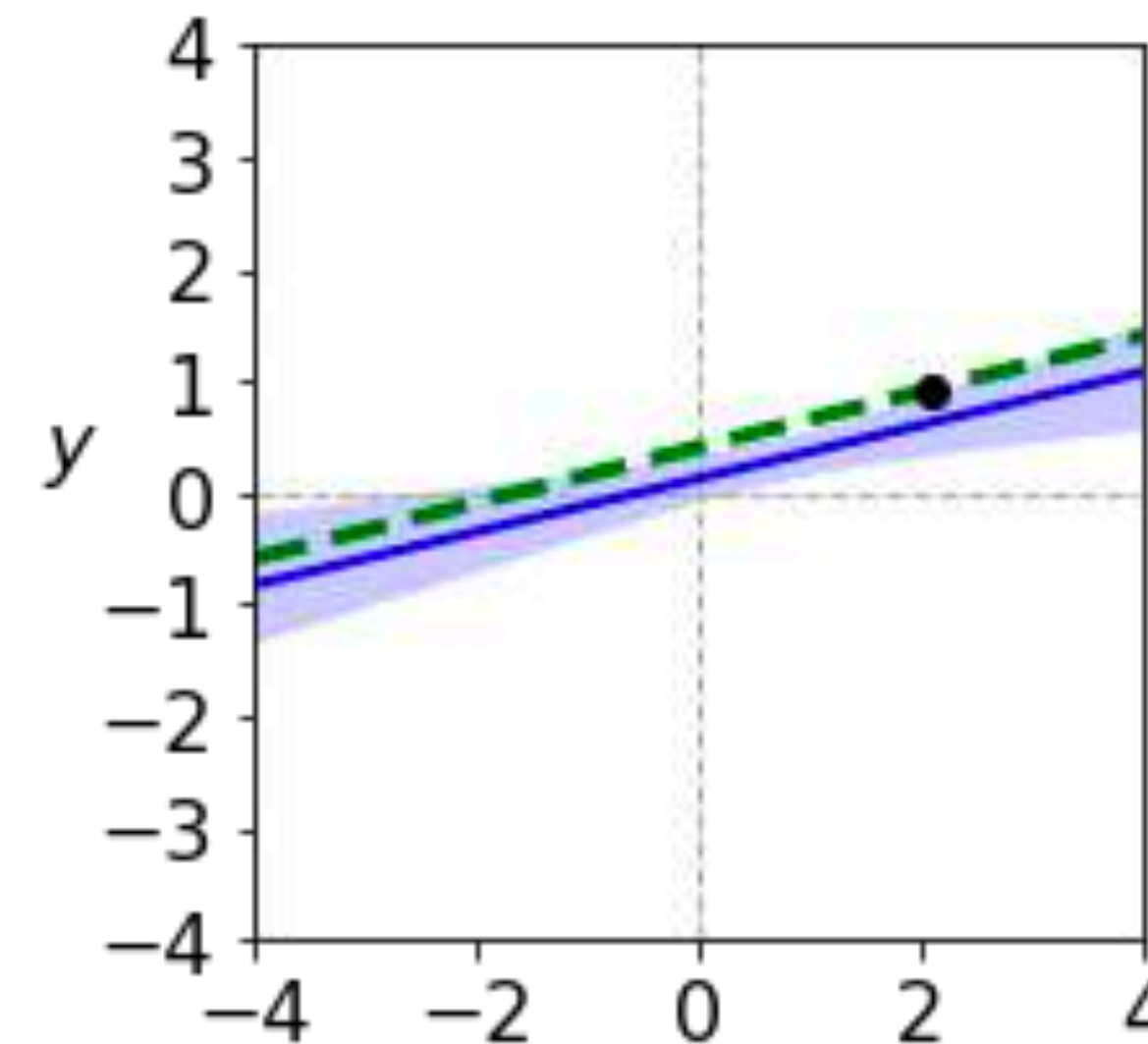
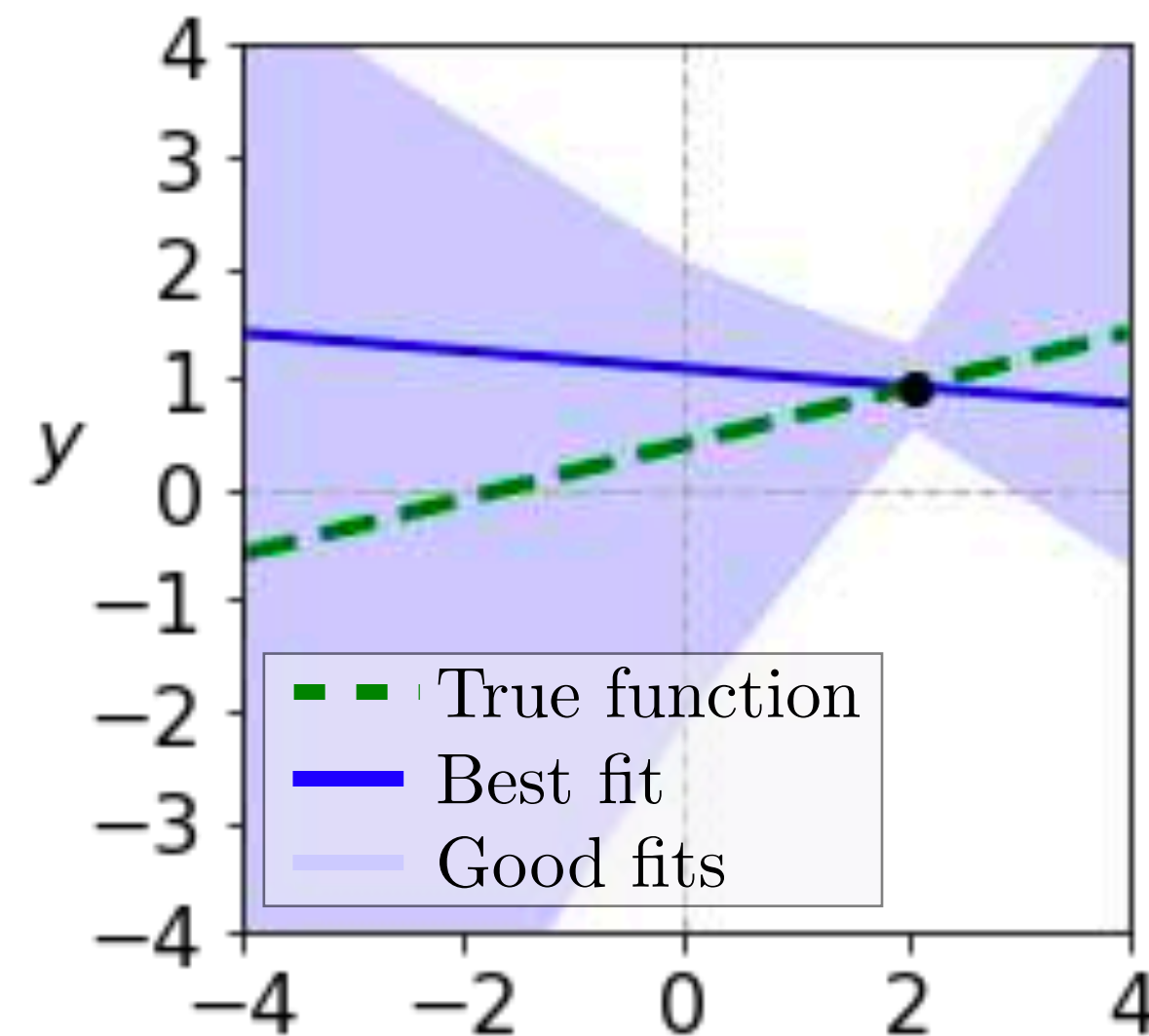
$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i |f_{\theta}(x^{(i)}) - y^{(i)}|^{0.25}$$

Increasing # datapoints →

Experiment 2: Effect of **Priors**

Hypothesis space:

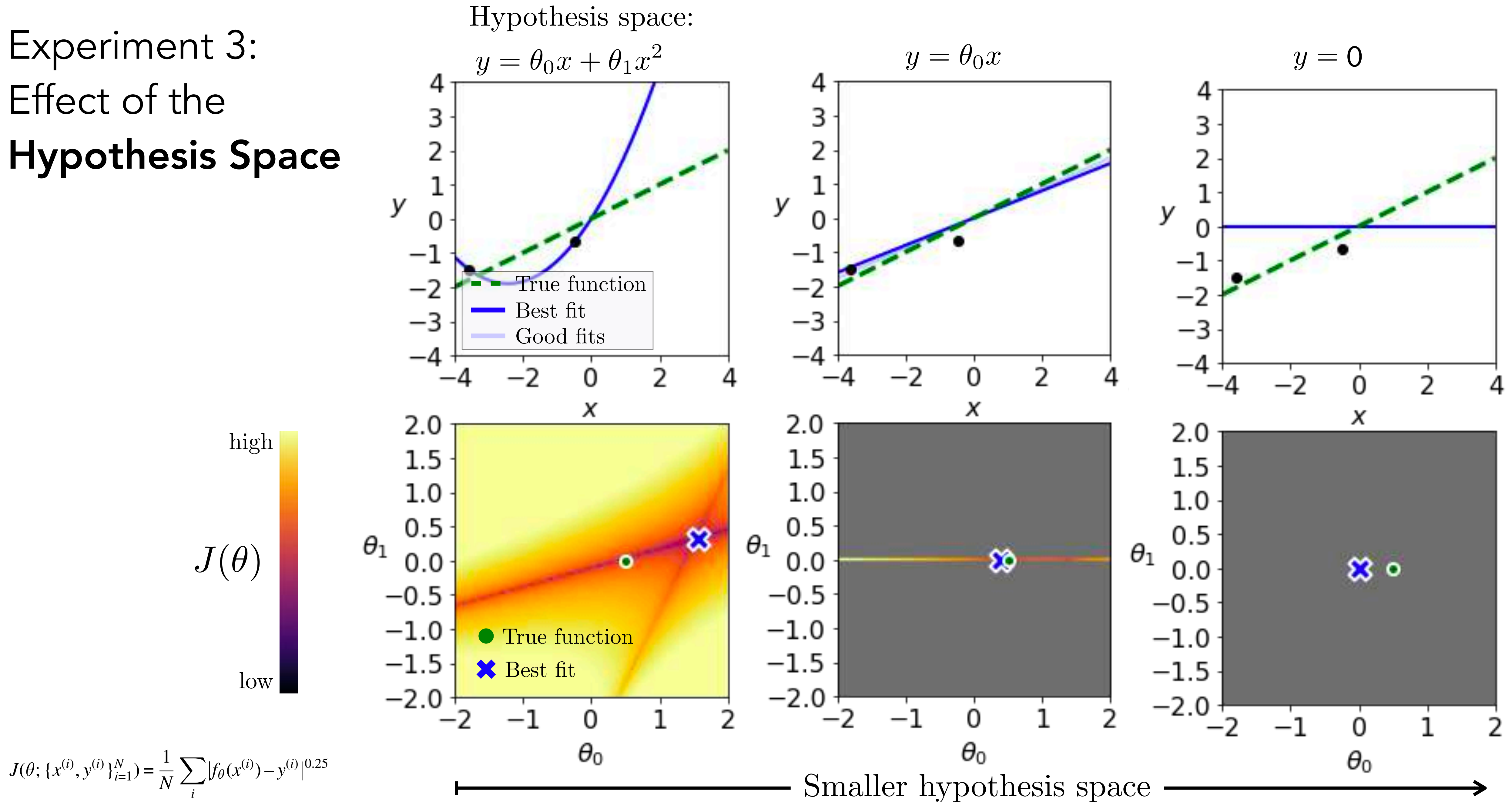
$$y = \theta_0 + \theta_1 x$$



$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i \|f_{\theta}(x^{(i)}) - y^{(i)}\|_2^2 + \lambda \|\theta\|_2^2$$

Increasing regularizer strength λ →

Experiment 3:
Effect of the
Hypothesis Space



Concluding remarks

Learning turns data into algorithms. In this era of big data, it is a major component of almost all modern computer vision systems.

The goal of learning is to extract lessons from past data to help on future problem solving. Unless the future is identical to the past, this is a problem that requires generalization.

This requires searching for a solution that not only *fits* the data, but also for one that is *simple*, so that it doesn't overfit and generalizes well.